Teoria dell'Informazione

Appunti del corso, Anno Accademico 2025/2026

Marco Luisi

Prefazione

Questi appunti sono basati sul materiale e sulle lezioni del corso di Teoria dell'Informazione, tenuto dalla Professoressa Veronica Guerrini nell'anno accademico 2025/2026 per il corso di laurea in Informatica presso l'Università di Pisa.

Sono stati redatti principalmente come strumento personale per lo studio e per familiarizzare con LaTeX con l'ausilio di LLM. Presentano alcune incongruenze: alcuni argomenti sono stati trattati in modo più approfondito di altri, e alcune sezioni risultano maggiormente rielaborate rispetto al materiale originale del docente.

Il testo presenta ancora delle mancanze e sicuramente qualche errore; inoltre, alcuni grafici potrebbero essere assenti o da correggere.

Ho deciso di rendere pubblici questi appunti per lasciarli come base di partenza per chiunque voglia utilizzarli, migliorarli o integrarli in futuro. Gli appunti sono resi disponibili come open source, pensati per fungere da base di partenza per chiunque voglia utilizzarli, perfezionarli o arricchirli.

Prefazione

Questi appunti nascono dalla rielaborazione del materiale e delle lezioni del corso di Teoria dell'Informazione, tenuto dalla Professoressa Veronica Guerrini nell'anno accademico 2025/2026 per il corso di laurea in Informatica presso l'Università di Pisa.

Il progetto ha preso forma inizialmente come strumento di studio e di pratica con LaTeX (supportato dall'uso di LLM). Per questo motivo, la stesura può presentare alcune disomogeneità: determinati argomenti sono trattati in modo più approfondito di altri e diverse sezioni risultano maggiormente rielaborate rispetto al materiale originale della docente.

È possibile che il testo contenga ancora delle mancanze o imprecisioni, così come alcuni grafici potrebbero risultare assenti o necessitare di revisione.

Tutto il materiale viene rilasciato in formato open-source allo scopo di fornire una risorsa didattica condivisa e collaborativa. L'obiettivo è offrire una solida base di partenza a disposizione della community, per chiunque desideri consultarla, perfezionarla o arricchirla nel tempo.

Indice

I Fondamenti della Teoria dell'Informazione **2**

1 Informazione ed Entropia **3**
1.1 Informazione e contenuto informativo . . . 3
1.1.1 Cenni storici: L'intuizione di Hartley . . . 3
1.1.2 Cosa è l'*Informazione* . . . 3
1.1.3 Come si *misura* l'informazione? . . . 3
1.1.4 BIT . . . 4
1.2 Sistemi di Comunicazione . . . 4
1.2.1 Modello Base . . . 4
1.2.2 Finalità della Codifica . . . 4
1.2.3 Definizione Probabilistica di Sorgente . . . 5
1.2.4 Richiamo sui Logaritmi . . . 5
1.2.5 Auto-informazione di un Simbolo . . . 5
1.3 Teoria dell'Entropia . . . 6
1.3.1 Entropia di una Sorgente Binaria . . . 6
1.3.2 Proprietà fondamentali e limiti dell'Entropia . . . 7
1.3.3 Estensione a Più Variabili: Esperimenti Congiunti e Condizionati . . . 7
1.3.4 Fondazione Assiomatica dell'Entropia . . . 10
1.3.5 Entropia Relativa . . . 12

II Teoria della Codifica di Sorgente **13**

2 Principi di Codifica di Sorgente **15**
2.1 Definizioni e Classificazione dei Codici . . . 15
2.1.1 Codifica di Sorgente . . . 15
2.1.2 Decifrabilità dei Codici . . . 16
2.1.3 Rappresentazione ad Albero dei Codici . . . 17
2.1.4 Algoritmo di Sardinas-Patterson . . . 17
2.2 Disuguaglianze di Kraft e Kraft-McMillan . . . 18
2.2.1 Diseguaglianza di Kraft (per codici *istantanei*) . . . 19
2.2.2 Disuguaglianza di McMillan (per codici UD) . . . 20
2.2.3 Teorema di Kraft-McMillan (per codici UD) . . . 21
2.3 Lunghezza Media e Teorema di Shannon . . . 21
2.3.1 Lunghezza media e Codice Ottimo . . . 21
2.3.2 Primo Teorema di Shannon . . . 22
2.4 Parametri di Trasmissione ed Efficienza . . . 22
2.4.1 Notazione e Rate di Trasmissione . . . 22
2.4.2 Efficienza dei codici e ridondanza . . . 23

3 Metodi e Algoritmi per la Compressione dell'Informazione **25**
3.1 Metriche di Valutazione e Vincolo Lossless . . . 25
3.2 Algoritmi statistici . . . 25
3.2.1 Codifica di Shannon . . . 26
3.2.2 Codifica di Shannon-Fano (nel caso binario) . . . 27
3.2.3 Condizioni per ottimalità . . . 30
3.2.4 Codifica di Huffman . . . 30

3.2.5 Codifica Aritmetica . . . 34
3.3 Metodi basati su Dizionario . . . 37
3.3.1 Dizionario . . . 37
3.3.2 Dizionari statici . . . 37
3.3.3 Dizionari dinamici . . . 38
3.4 Sorgenti con Memoria e Trasformazioni dei Dati . . . 48
3.4.1 Sorgenti con Memoria . . . 48
3.4.2 Codifica Predittiva . . . 48
3.4.3 Entropia Empirica . . . 49
3.4.4 Trasformata di Burrows-Wheeler . . . 49
3.4.5 Algoritmi di Trasformazione (Post-BWT) . . . 51
3.4.6 Combinazione di Tecniche . . . 53

III Codifica di Canale 54

4 Teoria della Codifica di Canale 55
4.1 Introduzione alla Codifica di Canale . . . 55
4.2 Modello Matematico del Canale Discreto . . . 56
4.2.1 Canale discreto senza memoria tempo-invariante . . . 56
4.3 Misure di Informazione e Capacità . . . 57
4.3.1 Entropie Condizionate: Rumore ed Equivocazione . . . 58
4.3.2 Informazione Mutua . . . 58
4.4 Capacità di Canale . . . 60
4.5 Analisi di Canali Notevoli . . . 60
4.5.1 Canali Speciali (Classificazione Strutturale) . . . 61
4.5.2 Canali Simmetrici e Uniformi . . . 62
4.5.3 Il Canale Binario Simmetrico (BSC) . . . 63
4.5.4 Canale simmetrico q-ario . . . 64
4.5.5 Canale Simmetrico con cancellazione . . . 65
4.5.6 Canale Z . . . 65
4.6 Tasso di Trasmissione e Criteri di Decodifica . . . 66
4.6.1 Codici di Canale e Tasso di Trasmissione . . . 66
4.6.2 Il Decodificatore di Canale . . . 66
4.6.3 Criteri di Decisione Ottimi . . . 67
4.7 Secondo Teorema di Shannon . . . 69
4.7.1 Enunciato del Teorema e Teorema Inverso . . . 69

IV Tecniche e Famiglie di Codici per il Controllo d'Errore 71

5 Strategie Generali di Controllo e Correzione d'Errore 72
5.1 Capacità di Rivelazione e Correzione . . . 73

6 Codici a Blocco 74
6.1 Principi Fondamentali dei Codici a Blocco . . . 74
6.2 Esempi Introduttivi di Codici a Blocco *Lineari* . . . 74
6.2.1 Codice a Ripetizione . . . 74
6.2.2 Codici a Controllo di Parità Singola (CCP) . . . 75
6.3 Struttura dei Codici a Blocco . . . 77
6.4 I Codici Algebrici . . . 78
6.4.1 Richiami di Strutture Algebriche . . . 78
6.5 Geometria dei Codici: Spazio e Distanza di Hamming . . . 78
6.6 Criterio di decodifica a distanza minima . . . 79
6.7 Capacità di correzione del codice . . . 80
6.8 Codici perfetti . . . 81
6.9 I Codici Lineari a Blocco . . . 82
6.9.1 Definizione e Proprietà Fondamentali . . . 82
6.9.2 La Matrice Generatrice e la Codifica . . . 83

6.9.3 Codifica di un codice Lineare . . . 83
6.9.4 La Matrice di Controllo della Parità (H) . . . 84
6.9.5 Decodifica Algebrica: Sindrome e Tabella di Slepian . . . 87
6.9.6 Correzione di errori tramite Sindrome e Tabella di Slepian . . . 88
6.9.7 Probabilità di Corretta Decodifica . . . 90
6.10 I Codici di Hamming (Binari) . . . 91
6.10.1 Costruzione . . . 91
6.10.2 Decodifica a Sindrome (Correzione di un singolo errore) . . . 92
6.11 Limiti dei Codici a Blocchi e Codici Perfetti . . . 94
6.11.1 Il Limite di Singleton . . . 94
6.11.2 I Limiti di Hamming (Sphere Packing Bound) . . . 95
6.11.3 Il Limite VGS (Varshamov-Gilbert-Sacks) . . . 97
6.11.4 Codici Perfetti . . . 98
6.12 Codici Ciclici . . . 99
6.12.1 Definizione e Rappresentazione Polinomiale . . . 99
6.12.2 Operazioni su codici ciclici . . . 99
6.12.3 Polinomio generatore . . . 100
6.12.4 Caratterizzazione con i polinomi . . . 100
6.12.5 Matrice Generatrice . . . 100
6.12.6 Scelta del polinomio generatore . . . 101
6.12.7 Polinomio di controllo . . . 101
6.12.8 Matrice di controllo e Codice Duale . . . 102
6.12.9 Codici ciclici Sistematici . . . 103
6.12.10 Codici Ciclici binari - Algoritmi di Codifica . . . 103
6.12.11 Codici Ciclici - Algoritmi di decodifica e rilevazione di errore . . . 105
6.12.12 Vantaggi dei codici ciclici . . . 107

7 Codici Convoluzionali 108
7.1 Principi e Struttura dei Codici Convoluzionali . . . 108
7.1.1 L'Operazione di Convoluzione e la Memoria . . . 108
7.1.2 Parametri e notazioni . . . 108
7.1.3 Codificatore Convoluzionale . . . 109
7.1.4 Sequenze Generatrici . . . 109
7.1.5 Rappresentazioni Grafiche . . . 109
7.1.6 Capacità Correttiva: Distanze del Codice . . . 110
7.1.7 Funzione Generatrice . . . 111
7.2 Decodifica dei Codici Convoluzionali . . . 111
7.2.1 Metriche per la Decodifica a Massima Verosimiglianza su Traliccio . . . 111
7.2.2 L'Algoritmo di Viterbi . . . 112
7.2.3 Limiti e Complessità Computazionale . . . 113

La Teoria dell'Informazione è una disciplina scientifica e matematica formalmente inaugurata da Claude Shannon nel 1948. Il suo contributo fondamentale fu quello di separare il significato semantico di un messaggio dalla sua forma sintattica, permettendo così di trattare l'informazione come una quantità misurabile. L'obiettivo primario della teoria è definire i limiti fondamentali della comunicazione: quanto si può comprimere un dato e quanto affidabilmente lo si può trasmettere su un canale rumoroso.

Parte I

Fondamenti della Teoria dell'Informazione

Capitolo 1

Informazione ed Entropia

Sviluppando il concetto di informazione come quantità misurabile e indipendente dalla semantica, in questo capitolo vengono analizzati gli strumenti matematici necessari per quantificare l'incertezza. Il legame tra la probabilità di accadimento degli eventi e la loro portata informativa trova la sua sintesi nell'**entropia**, la grandezza che definisce il contenuto informativo medio di una sorgente di informazione.

Tali fondamenta teoriche consentono di interpretare il processo comunicativo come un fenomeno universale, fornendo modelli applicabili non solo all'informatica, ma anche allo studio del linguaggio naturale, della crittografia e della biologia molecolare.

1.1 Informazione e contenuto informativo

1.1.1 Cenni storici: L'intuizione di Hartley

Prima della formalizzazione matematica definitiva ad opera di Claude Shannon nel 1948, i primi passi verso una misurazione quantitativa dell'informazione furono compiuti dall'ingegnere R.V.L. Hartley nel 1928.

Il contributo pionieristico di Hartley consistette nel legare per la prima volta il concetto di informazione all'eliminazione delle alternative. Secondo la sua visione, si genera informazione quando si escludono determinati eventi da un insieme di possibilità (come avviene, ad esempio, risolvendo l'incertezza basilare tra un "sì" e un "no", o nel lancio di una moneta). Questa associazione fondamentale tra informazione e risoluzione dell'incertezza ha fornito la base concettuale su cui è stata successivamente edificata l'intera teoria probabilistica della comunicazione.

1.1.2 Cosa è l'*Informazione*

La risposta più immediata che si può dare è che l'informazione è uno scambio di conoscenza tra due entità, che comporta per chi la riceve il superamento di uno stato di incertezza e permette di risolvere una alternativa. Quanta incertezza riesce a sciogliere misura la sua *utilità* o per meglio dire il suo contenuto informativo. È quindi di fondamentale importanza dare una misura puntuale del contenuto informativo di un'informazione.

1.1.3 Come si *misura* l'informazione?

Per quantificare l'informazione in modo rigoroso, è necessario slegarsi dal significato semantico del messaggio e osservare esclusivamente la sua probabilità di occorrenza. L'idea guida è che il contenuto informativo sia direttamente proporzionale al grado di *sorpresa* che un evento arreca al destinatario:

- Un evento molto probabile (prevedibile) porta con sé poca informazione, poiché conferma ciò che era già ampiamente atteso.
- Un evento molto raro (imprevedibile) porta molta informazione, in quanto risolve uno stato di incertezza elevato.

Oltre a questa proporzionalità inversa con la probabilità, una misura coerente dell'informazione deve soddisfare la proprietà di **additività**: se riceviamo due informazioni indipendenti, il contenuto informativo totale deve essere pari alla somma delle singole parti. Matematicamente, l'unica funzione in grado di

trasformare il prodotto delle probabilità ($p_1 \cdot p_2$) nella somma dei contenuti informativi ($I_1 + I_2$) è il logaritmo.

La scelta della base b del logaritmo determina l'unità di misura utilizzata:

- **Base 2**: l'unità di misura è il **BIT** (Binary Digit).
- **Base** e: l'unità di misura è il **NAT** (Natural Unit).
- **Base 10**: l'unità di misura è l'**HARTLEY**.

1.1.4 BIT

Se l'informazione riduce il grado di incertezza, è naturale domandarsi quale sia il minimo **quanto** di informazione possibile. Tale valore minimo si riscontra quando l'incertezza riguarda la scelta tra due soli esiti **equiprobabili** (ovvero con probabilità $p = 1/2$). Da questa configurazione elementare deriva il **BIT**, termine coniato da Shannon come contrazione di ***B**inary dig**IT***.

È fondamentale sottolineare che questa definizione è concettualmente distinta dalla *cifra binaria*, **Binit**, utilizzata in informatica: mentre quest'ultima rappresenta un supporto fisico o logico a due stati, il BIT di Shannon definisce l'unità fondamentale per misurare il contenuto informativo. Una cifra binaria può contenere *fino a* un bit di informazione, ma ne veicola effettivamente uno solo quando i due stati hanno la medesima probabilità di verificarsi.

1.2 Sistemi di Comunicazione

Nella sua Teoria Matematica della Comunicazione, Shannon modella il processo comunicativo in modo preciso ma al contempo generico.

Preciso, perché i vari componenti e le fasi della comunicazione sono definiti in modo rigoroso, permettendo un'analisi quantitativa del fenomeno;

generico, in quanto lo schema risulta applicabile in ambiti profondamente differenti: dalla comunicazione verbale alle telecomunicazioni, fino al DNA.

1.2.1 Modello Base

Secondo lo schema classico di Shannon, qualunque processo comunicativo può essere scomposto in cinque componenti funzionali fondamentali:

Sorgente: L'entità che genera il messaggio originale scegliendo tra i simboli di un alfabeto Σ. La sorgente è caratterizzata statisticamente dalle probabilità di emissione di ciascun simbolo.

Trasmettitore (Codificatore): Il blocco che trasforma il messaggio in un segnale adatto alla trasmissione sul canale. In questa fase avvengono le operazioni di compressione e protezione del dato.

Canale: Il mezzo fisico che trasporta il segnale. Nelle analisi reali, il canale è sempre caratterizzato dalla presenza di **rumore**, ovvero un disturbo aleatorio che può alterare il segnale durante il transito.

Ricevitore (Decodificatore): L'entità che riceve il segnale dal canale e tenta di ricostruire il messaggio originale effettuando le operazioni inverse rispetto al trasmettitore.

Destinatario: L'utilizzatore finale a cui è destinata l'informazione ricostruita.

1.2.2 Finalità della Codifica

All'interno del trasmettitore, l'informazione subisce due processi di trasformazione distinti che rispondono a esigenze opposte:

Codifica di Sorgente: Ha l'obiettivo di massimizzare l'efficienza eliminando la ridondanza superflua. Questo processo permette di rappresentare il messaggio con il minor numero possibile di bit senza perdita di contenuto informativo (compressione).

Codifica di Canale: Ha l'obiettivo di garantire l'affidabilità della trasmissione. A differenza della precedente, questa operazione aggiunge *strategicamente* una quota di ridondanza (bit di controllo) per permettere al ricevitore di rilevare e correggere eventuali errori introdotti dal rumore del canale.

1.2.3 Definizione Probabilistica di Sorgente

Per poter misurare e manipolare l'informazione, è necessario modellare la **sorgente** in termini puramente statistici e probabilistici, astraendo dal significato dei messaggi che essa genera.

Una sorgente discreta di informazione viene definita da:

- Un **alfabeto sorgente** $\Sigma = \{\sigma_1, \sigma_2, \ldots, \sigma_m\}$, ovvero un insieme finito di m simboli distinguibili.
- Una **distribuzione di probabilità** associata ai simboli. Ad ogni simbolo $\sigma_i \in \Sigma$ è associata una probabilità di emissione $p_i = P(X = \sigma_i)$, tale che:

$$0 \leq p_i \leq 1 \quad \forall i$$

$$\sum_{i=1}^{m} p_i = 1$$

- Symbol rate r: misurata in $\frac{simboli}{secondo}$, rappresenta la velocità **media** di emissione della sorgente.

Quando le successive emissioni di simboli da parte della sorgente sono eventi statisticamente *indipendenti* tra loro (cioè la probabilità di emettere un simbolo non dipende dai simboli emessi in precedenza), la sorgente prende il nome di **Sorgente Discreta Senza Memoria**.

Al contrario, se i simboli emessi dipendono da un insieme finito di t simboli emessi in istanti precedenti, parliamo di **sorgente con memoria**, o **sorgente di Markov**, di **ordine** t. Per sog

1.2.4 Richiamo sui Logaritmi

Essendo l'informazione legata a probabilità (che sono valori compresi tra 0 e 1) e dovendo rispettare proprietà di additività per eventi indipendenti, lo strumento matematico d'elezione per questa teoria è il **logaritmo**.

Quando si necessita di quantificare il contenuto informativo complessivo generato da più eventi, emerge una condizione fondamentale: poiché la probabilità congiunta di eventi indipendenti è data dal loro prodotto, mentre l'informazione totale deve corrispondere alla loro somma, l'unica funzione matematica in grado di soddisfare questo **vincolo di additività** è il logaritmo.

1.2.5 Auto-informazione di un Simbolo

Definita la probabilità di emissione p_i di un singolo evento (o simbolo) σ_i, possiamo ora quantificare l'informazione che esso trasporta.

Shannon definì **misura di informazione** (o auto-informazione) dell'evento σ_i la quantità:

$$I(\sigma_i) = \log_b \left(\frac{1}{p_i}\right) = -\log_b p_i$$

Le proprietà di $I(\sigma_i)$ sono:

1. $I(\sigma_i) \geq 0 \quad 0 \leq p_i \leq 1$
2. informazione **nulla** per eventi **certi** $I(\sigma_i) \to 0 \iff p_i \to 1$
3. Monotonia: $p_i < p_j \implies I(\sigma_i) > I(\sigma_j)$ (eventi *più rari* veicolano informazione maggiore)
4. Additività $I(\sigma_i, \sigma_j) = I(\sigma_i) + I(\sigma_j)$

La base b del logaritmo non è necessariamente specificata a priori, ma la sua scelta determina l'unità di misura assegnata al contenuto informativo:

- Se la base è 2, l'unità di misura è il **BIT** (la più usata in questo ambito).
- Se la base è e, l'unità di misura è il **NAT** (1 NAT = 1.44 bits).
- Se la base è 10, l'unità di misura è l'**HARTLEY** (1 HARTLEY = 3.32 bits).

Esempio: Misura di informazione

Data una sorgente S con il seguente alfabeto e probabilità associate:

$$S = \begin{pmatrix} a & b & c & r \\ 1/2 & 1/4 & 1/8 & 1/8 \end{pmatrix}$$

I contenuti informativi dei singoli simboli (in bit, usando base 2) sono:

$$I(a) = \log_2\left(\frac{1}{1/2}\right) = \log_2(2) = 1 \text{ bit}$$

$$I(b) = \log_2\left(\frac{1}{1/4}\right) = \log_2(4) = 2 \text{ bit}$$

$$I(c) = \log_2\left(\frac{1}{1/8}\right) = \log_2(8) = 3 \text{ bit}$$

$$I(r) = \log_2\left(\frac{1}{1/8}\right) = \log_2(8) = 3 \text{ bit}$$

1.3 Teoria dell'Entropia

Estendendo il concetto di auto-informazione dal singolo evento all'intera sorgente, si definisce l'**Entropia** di una sorgente discreta senza memoria S sull'alfabeto $\Sigma = \{\sigma_1, \sigma_2, \ldots, \sigma_m\}$ come il valore atteso (o media pesata) del contenuto informativo dei suoi simboli:

$$H(S) = \sum_{i=1}^{m} p_i I(\sigma_i) = \sum_{i=1}^{m} p_i \log_2\left(\frac{1}{p_i}\right) = -\sum_{i=1}^{m} p_i \log_2 p_i$$

Assumendo la base 2 per il logaritmo (convenzione standard se non diversamente specificato), l'entropia si misura in **bit/simbolo**. Essa rappresenta due concetti equivalenti:

- L'**incertezza media** che un osservatore ha *prima* di scoprire l'output della sorgente.
- La **quantità media di informazione** per simbolo che la sorgente fornisce *dopo* l'emissione.

Poiché la probabilità p_i è compresa tra 0 e 1, il logaritmo $\log_2 p_i$ sarà sempre una quantità minore o uguale a zero. Di conseguenza, l'entropia, definita con il segno negativo davanti alla sommatoria, è una grandezza **sempre non negativa** ($H(S) \geq 0$).

Nota storica: L'origine del nome. Il termine "Entropia" fu suggerito a Shannon dal matematico John Von Neumann. Shannon inizialmente voleva chiamare la sua formula "informazione" o "incertezza". Von Neumann gli consigliò di usare *entropia* per due motivi: in primo luogo, la funzione matematica era formalmente identica a quella utilizzata nella meccanica statistica; in secondo luogo, essendo l'entropia un concetto che "nessuno sa cosa sia con certezza", Shannon avrebbe avuto "sempre un vantaggio" in qualsiasi dibattito accademico.

1.3.1 Entropia di una Sorgente Binaria

Un caso di studio fondamentale è quello della sorgente binaria, in cui l'alfabeto è composto da due soli simboli con probabilità p e $1-p$. Applicando la definizione, la funzione di entropia binaria $H(p)$ diventa:

$$H(p) = p \log_2\left(\frac{1}{p}\right) + (1-p) \log_2\left(\frac{1}{1-p}\right)$$

Analizzando l'andamento di questa funzione, si deducono due comportamenti limite:

- **Incertezza Nulla:** Se $p = 0$ o $p = 1$, l'esito è certo (la sorgente emette sempre lo stesso simbolo). In questo caso $H(p) = 0$.
- **Incertezza Massima:** Il valore massimo dell'entropia si ottiene quando i due esiti sono equiprobabili, ovvero $p = 0.5$. In questa configurazione $H(0.5) = 1$ bit/simbolo.

1.3.2 Proprietà fondamentali e limiti dell'Entropia

Definita l'entropia, è necessario analizzarne il comportamento formale. La *funzione entropia* $H(S)$ gode di proprietà matematiche che ne definiscono limiti assoluti e garantiscono la sua validità come *misura dell'informazione*.

Il Lemma del Logaritmo

Serve un appiglio matematico per individuare la limitazione superiore dell'entropia. Il **Lemma del Logaritmo** applicato all'equazione dell'entropia, permette di ricavare la seguente disuguaglianza:

$$-\sum_{i=1}^{m} p_i \log_b p_i \leq -\sum_{i=1}^{m} p_i \log_b \left(\frac{1}{m}\right)$$

Poiché il termine $\log_b(1/m)$ è una costante, può essere estratto dalla sommatoria, trasformandosi in $\log_b m \cdot \sum p_i$. Essendo la somma delle probabilità $\sum_{i=1}^{m} p_i = 1$, si ottiene l'assunto finale:

$$H(S) \leq \log_b m$$

Upper bound per l'entropia

Data una sorgente S con un alfabeto Σ composto da m simboli, ovvero $m = |\Sigma|$, l'entropia della sorgente è sempre limitata superiormente:

$$H(S) \leq \log_b m$$

Il valore massimo dell'entropia viene raggiunto se e solo se tutti i simboli della sorgente sono **equiprobabili**. Formalmente:

$$H(S) = \log_b m \iff \forall i = 1, \ldots, m \text{ si ha } p_i = \frac{1}{m}$$

Esempio: Upper bound per sorgente binaria

Prendiamo ad esempio il lancio di una moneta

$$S = \begin{pmatrix} T & C \\ p & 1-p \end{pmatrix} \qquad H(S) = p \log \frac{1}{p} + (1-p) \log \frac{1}{1-p}$$

La massima *incertezza* sull'esito si ha quando $p = \frac{1}{2} = 0.5$

1.3.3 Estensione a Più Variabili: Esperimenti Congiunti e Condizionati

Per analizzare situazioni più articolate si deve considerare anche l'entropia per eventi congiunti ed eventi condizionati. Consideriamo due sorgenti (variabili casuali) X ed Y definite da

$$X = \begin{pmatrix} x_1 & x_2 & \ldots & x_k \\ p(x_1) & p(x_2) & \ldots & p(x_k) \end{pmatrix} \qquad Y = \begin{pmatrix} y_1 & x_2 & \ldots & y_h \\ p(_1) & p(y_2) & \ldots & p(y_h) \end{pmatrix}$$

Consideriamo gli eventi come *indipendenti*.

Probabilità Congiunta e Probabilità Condizionata

Con queste premesse, la probabilità che si verifichi la coppia di eventi (x_i, y_j), detta **probabilità congiunta**, è il prodotto delle singole probabilità marginali:

$$p(x_i, y_j) = p(x_i) \cdot p(y_j)$$

La probabilità che si verifichi un evento y_j, a seguito di un evento x_i, con $p(x_i) > 0$, è detta **probabilità condizionata** ed è definita come:

$$p(y_j \mid x_i) = \frac{p(x_i, y_j)}{p(x_i)}$$

Entropia Congiunta e Massimo Valore

Considerando un esperimento congiunto che coinvolge due sorgenti (variabili aleatorie) X e Y, rispettivamente con k e h esiti possibili, l'esperimento congiunto avrà hk possibili risultati. L'**entropia congiunta** relativa a queste due sorgenti quantifica l'*incertezza media globale* sulle coppie di esiti (x_i, y_j), e si calcola utilizzando la probabilità congiunta:

$$H(X,Y) = -\sum_{i=1}^{k}\sum_{j=1}^{h} p(x_i, y_j) \log p(x_i, y_j)$$

Possiamo pensare a $H(X,Y)$ come al **numero medio di bit necessario per rappresentare una coppia di valori provenienti da X e Y**

Come per l'entropia di una singola sorgente, anche per l'entropia congiunta è presente una limitazione superiore, Il **massimo valore** che l'entropia congiunta può assumere è pari alla somma delle entropie marginali delle singole variabili:

$$H(X,Y) \leq H(X) + H(Y)$$

L'uguaglianza stretta, ovvero $H(X,Y) = H(X) + H(Y)$, si verifica se e solo se gli esperimenti X e Y sono statisticamente indipendenti.

Dimostrazione del Massimo Valore: Consideriamo le definizioni di $H(X)$, $H(Y)$ e la rispettiva entropia congiunta $H(X,Y)$

$$H(X) = -\sum_{i=1}^{k} p(x_i) \log p(x_i) = -\sum_{i=1}^{k} \left(\sum_{j=1}^{h} p(x_i, y_j) \right) \log p(x_i)$$

Per espandere $p(x_i)$, sfruttiamo la definizione di probabilità marginale: la probabilità totale che si verifichi l'evento x_i è pari alla somma delle probabilità di tutte le possibili combinazioni congiunte in cui x_i si presenta insieme a uno qualsiasi degli eventi y_j. Lo stesso principio si applica per ricavare $p(y_j)$.

$$H(Y) = -\sum_{j=1}^{h} p(y_i) \log p(y_j) = -\sum_{j=1}^{h} \left(\sum_{j=1}^{h} p(x_i, y_j) \right) \log p(y_j)$$

Lo stesso vale per $H(Y)$, *espandendo* le $p(y_j)$

Sfruttando la definizione delle probabilità marginali, ovvero $p(x_i) = \sum_j p(x_i, y_j)$ e $p(y_j) = \sum_i p(x_i, y_j)$, possiamo raccogliere tutti i termini all'interno di un'unica doppia sommatoria:

$$\begin{aligned}
H(X,Y) &= -\sum_{i=1}^{k}\sum_{j=1}^{h} p(x_i, y_j) \log p(x_i, y_j) \\
H(X) &= -\sum_{i=1}^{k} p(y_j) \log p(x_i) = -\sum_{i=1}^{k} \left(\sum_{j=1}^{h} p(x_i, y_j) \right) \log p(x_i) \\
H(Y) &= -\sum_{j=1}^{h} p(y_j) \log p(x_i) = -\sum_{j=1}^{h} \left(\sum_{i=1}^{k} p(x_i, y_j) \right) \log p(y_j) \\
H(X) + H(Y) &= -\sum_{i=1}^{k} \left(\sum_{j=1}^{h} p(x_i, y_j) \right) [\log p(x_i) + \log p(y_j)] =
\end{aligned}$$

Poniamo $q(x_i, y_j) = p(x_i)p(y_j)$, la somma delle entropie si riscrive come:

$$H(X) + H(Y) = -\sum_{i=1}^{k} \left(\sum_{j=1}^{h} p(x_i, y_j) \right) \log q(x_i, y_j)$$

Considerando il Lemma del Logaritmo e la formula dell'entropia congiunta, si ottiene:

$$H(X,Y) = -\sum_{i=1}^{k}\sum_{j=1}^{h} p(x_i, y_j) \log p(x_i, y_j) \leq -\sum_{i=1}^{k}\sum_{j=1}^{h} p(x_i, y_j) \log q(x_i, y_j)$$

Nella formulazione originaria del Lemma, è presente una sola sommatoria e qui due, ma con una re-indicizzazione può essere portata ad una sola.

Poiché $\sum_{i=1}^{k}\sum_{j=1}^{h} q(x_i, y_j) = \sum_{i=1}^{k} p(x_i)\sum_{j=1}^{h} p(y_j) = 1 \cdot 1 = 1$, la distribuzione q è valida per l'applicazione del Lemma.

Si è dimostrato che $H(X,Y) \leq H(X) + H(Y)$, con l'uguaglianza valida se e soltanto se X ed Y sono indipendenti.

Esempio: Entropia Congiunta

Consideriamo due esperimenti X ed Y con la seguente distribuzione di probabilità congiunta $p(x_i, y_i)$:

Y \ X	1	2	3	4
1	$\frac{1}{8}$	$\frac{1}{16}$	$\frac{1}{32}$	$\frac{1}{32}$
2	$\frac{1}{16}$	$\frac{1}{8}$	$\frac{1}{32}$	$\frac{1}{32}$
3	$\frac{1}{16}$	$\frac{1}{16}$	$\frac{1}{16}$	$\frac{1}{16}$
4	$\frac{1}{4}$	0	0	0

La relativa entropia congiunta sarà:

$$\begin{aligned} H(X,Y) &= \sum_{i=1}^{4}\sum_{j=1}^{4} p(x_i, y_j)\log_2\left(\frac{1}{p(x_i, y_j)}\right) \\ &= \frac{1}{4}\log_2(4) + \frac{1}{4}\log_2(8) + \frac{3}{8}\log_2(16) + \frac{1}{8}\log_2(32) \\ &= \frac{27}{8} = 3.375 \text{ bit} \end{aligned}$$

Entropia Condizionata

Conoscendo le distribuzioni di probabilità (e i rispettivi valori di entropia) di due sorgenti aleatorie X ed Y, come si calcola l'entropia **condizionata**? Il valore di $H(X|Y)$ quantifica l'incertezza **media** residua sugli esiti della sorgente X, conoscendo la distribuzione di probabilità di Y.

Per arrivare alla formulazione finale ci serve considerare prima la formula per calcolare l'entropia condizionata di X calcolata sul verificarsi del particolare evento y_j di Y:

$$H(X|y_j) = \sum_{i=1}^{k} p(x_i|y_j)\log\frac{1}{p(x_i|y_j)}$$

La media di questa incertezza, relativamente agli esiti di Y è l'**Entropia condizionata di X dato Y**:

$$H(X|Y) = \sum_{j=1}^{h} p(y_j)H(X|y_j)$$

$$H(X|Y) = \sum_{i=1}^{k}\sum_{j=1}^{h} p(y_j)p(x_i|y_j)\log\frac{1}{p(x_i|y_j)}$$

Proprietà dell'Entropia Condizionata. Valgono tre proprietà:

1. $H(X,Y) = H(Y) + H(X|Y)$
2. $H(X,Y) = H(X) + H(Y|X)$
3. $H(X|Y) \leq H(X), \quad H(X|Y) = H(X) \iff X, Y$ sono indipendenti

1.3.4 Fondazione Assiomatica dell'Entropia

Finora abbiamo trattato l'entropia da un punto di vista *operativo* e probabilistico. Shannon però ha definito questa specifica espressione matematica per quantificare *incertezza* e *informazione* con un approccio assiomatico, stabilendo i requisiti che una funzione deve possedere per poter essere considerata una misura coerente dell'informazione **media** prodotta da una sorgente.

I Tre Assiomi dell'Entropia di Shannon

Per una sorgente discreta definita come $S = \left(\begin{smallmatrix} \sigma_1 & \sigma_2 & \cdots & \sigma_n \\ p_1 & p_2 & \cdots & p_n \end{smallmatrix}\right)$, Shannon ha introdotto, in modo assiomatico, il *concetto di entropia*, alla ricerca di una funzione $H(S) = H_n(p_1, p_2, \ldots, p_n)$ in grado di misurare l'informazione media prodotta da una sorgente. Affinché tale funzione sia valida, deve soddisfare tre assiomi fondamentali:

1. **Continuità:** $H_n(p_1, p_2, \ldots, p_n)$ deve essere una funzione continua delle probabilità p_i.
2. **Monotonia:** Dati due esperimenti con risultati equiprobabili e rispettivamente n ed $n+1$ esiti, l'esperimento con più esiti presenta un'incertezza maggiore:
$$H_n\left(\frac{1}{n}, \ldots, \frac{1}{n}\right) < H_{n+1}\left(\frac{1}{n+1}, \ldots, \frac{1}{n+1}\right)$$
3. **Legge del Raggruppamento:** Se un esperimento viene scomposto in scelte o fasi successive, l'incertezza totale deve essere uguale alla somma dell'incertezza della **prima scelta** e delle incertezze delle **scelte successive**, ciascuna pesata per la propria probabilità di accadimento.

Il Terzo Assioma: La Legge del Raggruppamento

Il terzo assioma è il vincolo strutturale più forte, in quanto definisce il comportamento dell'entropia a fronte di scomposizioni *arbitrarie*. Garantisce che l'informazione totale non dipende da *come* decidiamo di raggruppare o osservare gli esiti, ma esclusivamente dalla distribuzione di probabilità.

Questo assioma garantisce la consistenza della misura: l'informazione totale non dipende da *come* decidiamo di raggruppare o osservare gli esiti della sorgente, ma solo dalla struttura intrinseca delle probabilità.

Lemmi Preliminari

Per arrivare a dimostrare formalmente l'unicità della *funzione entropia*, è necessario introdurre alcuni risultati matematici intermedi.

1. **Lemma 1 (Lemma della funzione logaritmica):** una funzione f, positiva e monotòna crescente, per la quale per ogni n intero e per ogni $t > 1$ vale $f(t^n) = nf(t)$, è della forma
$$f(t) = c\log(t)$$
2. **Lemma 2 (Lemma dell'additività per prove indipendenti):** si deduce dal terzo assioma che l'incertezza relativa alle n ripetizioni, tra loro *indipendenti*, di un esperimento, è pari a n volte l'incertezza/entropia del singolo esperimento:
$$H(Y^n) = nH(Y)$$
3. **Lemma 3 (Lemma dell'incertezza per uscite equiprobabili):** dagli assiomi II e III, l'incertezza associata a un esperimento/sorgente con k esiti/simboli **equiprobabili**, vale $C\log(k)$.

   Dimostrazione: un esperimento configurato in questo modo avrà l'incertezza in funzione solamente del numero di esiti k. Quindi Y^n ha k^n possibili esiti equiprobabili; applicando il lemma precedente si ottiene:
$$h(k^n) = H(Y^n) = nH(Y) = nh(k)$$
   Dal secondo assioma, $h(k)$ è una funzione **crescente in** k; applicando il primo Lemma si ottiene la tesi iniziale.

Teorema di Unicità della funzione Entropia

Consideriamo una sorgente X con n possibili simboli $x_1, x_2, \ldots, x_n$, con relative probabilità $p_1, p_2, \ldots, p_n$.

Teorema 1.3.1. *L'unica funzione H che soddisfa contemporaneamente gli assiomi I, II e III è:*

$$H(p_1, p_2, \ldots, p_n) = -C \sum_{i=1}^{n} p_i \log_b p_i$$

La costante C dipende unicamente dalla unità di misura e dalla base del logaritmo, ma generalmente si assume uguale $C = 1$.

Dimostrazione. La dimostrazione segue due macro-passaggi logici principali: prima si verifica che la formula proposta rispetti i tre assiomi, e successivamente si dimostra che qualsiasi funzione che soddisfi tali assiomi deve necessariamente coincidere con la formula stessa.

Fase 1: La formula logaritmica soddisfa i tre assiomi.
Sostituendo la funzione $H(p_1, \ldots, p_n) = -C \sum p_i \log_b p_i$ nei tre assiomi originari, se ne verifica immediatamente la validità:

- **Assioma I (Continuità):** L'assioma è soddisfatto poiché la funzione logaritmica è continua rispetto alle probabilità p_i.
- **Assioma II (Monotonia):** Per un esperimento con uscite equiprobabili, la formula si riduce a $H = C \log n$, ed è sempre vero che $C \log n < C \log(n + 1)$.
- **Assioma III (Raggruppamento):** Grazie alle proprietà dei logaritmi, i prodotti tra probabilità condizionate si trasformano in somme, portando a verificare l'identità (*si omettono i passaggi algebrici per brevità*).

Fase 2: Unicità della funzione per probabilità razionali Dimostriamo ora che se una funzione H soddisfa gli assiomi, deve avere la forma del Teorema. Consideriamo un esperimento Y avente h risultati con probabilità razionali, che poniamo a comune denominatore m:

$$p(y_j) = \frac{m_j}{m} \quad \text{per } j = 1, 2, \ldots, h$$

Immaginiamo un esperimento X con m uscite equiprobabili. Per il Lemma 3, la sua incertezza vale $H(X) = C \log m$. L'esperimento X può essere scomposto in due fasi: il nostro esperimento Y, seguito da uno tra h esperimenti $Z^{(j)}$ indipendenti, ognuno con m_j esiti equiprobabili e incertezza $H(Z^{(j)}) = C \log m_j$.

Per l'Assioma III (Raggruppamento) vale l'uguaglianza:

$$H(X) = H(Y) + \sum_{j=1}^{h} p(y_j) H(Z^{(j)})$$

Sostituendo le incertezze dei casi equiprobabili fornite dal Lemma 3:

$$C \log m = H(Y) + \sum_{j=1}^{h} p(y_j) \, (C \log m_j)$$

Possiamo ora isolare l'incertezza $H(Y)$ della nostra sorgente:

$$H(Y) = C \log m - C \sum_{j=1}^{h} p(y_j) \log m_j$$

Sapendo che la somma delle probabilità $\sum p(y_j) = 1$, si moltiplica $C \log m$ per 1 e si raccoglie la sommatoria:

$$H(Y) = C \sum_{j=1}^{h} p(y_j) \log m - C \sum_{j=1}^{h} p(y_j) \log m_j$$

$$H(Y) = -C \sum_{j=1}^{h} p(y_j)(\log m_j - \log m) = -C \sum_{j=1}^{h} \frac{m_j}{m} \log \frac{m_j}{m}$$

Fase 3: Estensione alle probabilità reali

Essendo stata provata l'uguaglianza per tutti i valori razionali, per il requisito di Continuità imposto dall'Assioma I, la medesima espressione estende la sua validità in modo univoco anche ai numeri reali. □

1.3.5 Entropia Relativa

L'entropia di una singola variabile casuale misura l'incertezza intrinseca e la quantità media di informazione necessaria per rappresentarla. L'**Entropia Relativa** misura la *discrepanza* tra due diverse distribuzioni di probabilità p e q definite sul medesimo alfabeto, ed è definita come:

$$D(p \parallel q) = \sum_x p(x) \log \frac{p(x)}{q(x)}$$

Dal punto di vista della teoria dell'informazione, $D(p \parallel q)$ quantifica l'**inefficienza**, ovvero quanti bit aggiuntivi per simbolo si generano assumendo erroneamente che la distribuzione della sorgente sia q, quando in realtà la distribuzione vera è p. *Operativamente*, rappresenta il sovraccarico di lunghezza del codice se si progettasse un codice ottimale basato su q anziché su p.

Non è comunque una *vera distanza* tra distribuzioni perché non è simmetrica e non rispetta la disuguaglianza triangolare.

Parte II

Teoria della Codifica di Sorgente

La comunicazione, in ogni sua forma, contiene un certo grado di ridondanza. La codifica di sorgente ha lo scopo di ridurre questa ridondanza senza alterare il contenuto informativo, ottimizzando la rappresentazione dei dati. Fin dai tempi antichi, sono state adottate strategie naturali per rendere più efficiente la trasmissione dell'informazione. Un esempio è l'uso delle abbreviazioni nei manoscritti medievali, in cui simboli speciali sostituivano parole comuni per risparmiare spazio e tempo nella scrittura. Anche i segnali di fumo, impiegati da diverse culture, seguivano schemi codificati per trasmettere messaggi con il minimo numero di segnali possibile. Un altro caso significativo è il codice Morse (1838), che assegnava sequenze più brevi ai caratteri più frequenti, anticipando il principio della codifica ottimale. Tuttavia, è con lo sviluppo della Teoria dell'Informazione che il processo di codifica di sorgente è stato formalizzato, fornendo una base matematica rigorosa per la compressione dei dati.

Capitolo 2

Principi di Codifica di Sorgente

L'alfabeto della sorgente per quanto capace di veicolare informazione non è quasi mai ideale per la trasmissione efficiente dell'informazione stessa. La codifica di sorgente quindi trasforma i simboli in quelli di un altro alfabeto, e in ricezione si effettuerà il processo inverso ottenendo l'informazione nell'alfabeto originale. Le codifiche si possono raggruppare in due categorie fondamentali: quelle che codificano ogni singolo simbolo in una nuova *parola* e le codifiche che lavorano su parole ottenendone di nuove nell'alfabeto di codifica.

Un'ulteriore distinzione, essenziale per comprendere i meccanismi di compressione, riguarda la lunghezza delle stringhe prodotte:

- **Codici a lunghezza fissa:** Tutte le parole di codice generate hanno la medesima lunghezza (un classico esempio è la codifica ASCII a 8 bit per carattere). Sono semplici da gestire e decodificare, ma non ottimizzano lo spazio.
- **Codici a lunghezza variabile:** Le parole di codice hanno lunghezze differenti. L'efficienza (ovvero la compressione dei dati) si ottiene assegnando stringhe di bit più brevi ai simboli che si presentano con maggiore probabilità, e stringhe più lunghe ai simboli più rari.

2.1 Definizioni e Classificazione dei Codici

2.1.1 Codifica di Sorgente

Diamo una definizione di codice:

Definizione 2.1.1. Sia S una sorgente con alfabeto $\{s_1, s_2, \ldots, s_m\}$.
Un codice è una funzione che per ogni sequenza di simboli della sorgente S associa una sequenza di simboli di un alfabeto $X = x_1, x_2, \ldots, x_r$. La funzione deve essere **iniettiva**.

Consideriamo un alfabeto A e l'insieme delle stringhe generabili A^*. Supponiamo di codificare in alfabeto **binario** una sequenza di messaggi in un alfabeto qualsiasi $X = x_1, x_2, \ldots, x_k$

1. si identifica il **codice**: un insieme k *parole* di A^*;
2. ad ogni simbolo di X si assegna **univocamente** una parola del codice.

Come risultato quindi una stringa di X^* viene codificata in una sequenza di stringhe di A^*.

Esempi di codice

- La numerazione telefonica: due numeri telefonici sono tali per cui uno non è prefisso di un altro.
- Codice Morse: codifica binaria che assegna una codifica più concisa alle lettere più frequenti.
- Codice ASCII: ogni carattere è codificato con un numero di 8 bit.

Codifica a Blocchi non singolari

Un codice a blocchi si dice **non singolare** se ogni simbolo della sorgente è mappato in una parola di codice distinta. Formalmente, per ogni coppia di simboli diversi $x_i \neq x_j$, le rispettive parole di codice devono essere diverse:

$$C(x_i) \neq C(x_j)$$

Questa è ovviamente una condizione necessaria per avere un codice non ambiguo sui singoli caratteri, ma **non è sufficiente** per decodificare correttamente un messaggio composto da una *sequenza* di simboli (estensione del codice).

Si consideri ad esempio il seguente codice, che risulta chiaramente non singolare:

- $a \to 01$
- $b \to 10$
- $c \to 1$

Ricevendo in input 101 un decodificatore può alternativamente decodificare il messaggio con *ca* o *bc*.

2.1.2 Decifrabilità dei Codici

Appurato che la *non singolarità* di un codice non è sufficiente per renderlo non ambiguo, serve introdurre ulteriori condizioni. Nella pratica della trasmissione dati, i messaggi sono formati da flussi continui di simboli concatenati, definiti come *estensione* del codice. Per ricostruire il messaggio originale senza errori, l'estensione del codice non deve presentare alcuna ambiguità. Per risolvere questo problema, si definiscono classi di codici con requisiti via via più stringenti.

Codici Univocamente Decifrabili (UD)

Un codice che permette di separare i simboli senza alcuna ambiguità si dice **univocamente decifrabile**. Questa proprietà è una *garanzia* per la correttezza della decodifica. Codici **univocamente decifrabili** sono un sottoinsieme della famiglia dei codici non singolari.

Definizione 2.1.2. Un codice è detto univocamente decifrabile (**UD**) se ogni sequenza di parole del codice corrisponde ad una sola sequenza di simboli della sorgente. C è UD se: $\forall x_1, x_2, \ldots, x_n, y_1, y_2, \ldots, y_m \in C$ e se $x_1 x_2 \ldots x_n = y_1 y_2 \ldots y_m \implies n = m$ con $x_i = y_i \forall i$

Rimangono due problematiche da affrontare: come costruire in modo sistematico codici UD e come riconoscerli.

Il Ritardo di Decifrazione. Garantire che un codice sia UD assicura la corretta decodifica, ma solleva un ulteriore problema pratico: il **ritardo di decifrazione**. In alcuni codici UD, per poter determinare con certezza il primo simbolo del messaggio, il ricevitore potrebbe essere costretto a leggere l'intera sequenza fino alla fine (ritardo illimitato), oppure ad attendere l'arrivo di successivi k simboli (ritardo limitato a k). Per le comunicazioni efficienti e in tempo reale, è preferibile abbattere questo ritardo.

Codici Istantanei (o a Prefisso)

Per azzerare il ritardo di decifrazione, si definisce un sottoinsieme più restrittivo di codici. Un codice UD che si può decodificare senza la necessità di attendere i simboli della parola successiva si dice **istantaneo** o **codice a prefisso**.

Condizione **necessaria e sufficiente** perché un codice sia istantaneo è che **nessuna parola di codice sia prefisso di un'altra parola**.

Con un codice prefisso, individuare le interruzioni tra una parola di codice e la successiva è immediato: leggendo il flusso in input, non appena i bit accumulati formano una parola di codice legittima, essa viene decodificata e si riparte dal bit successivo. Non vi è mai la necessità di *guardare avanti*.

Nota: Una strategia banale per ottenere un codice univoco e istantaneo consiste nell'avere tutte le parole della stessa lunghezza, come ad esempio nel codice ASCII. La semplicità è controbilanciata dalla implicita rinuncia a qualsiasi forma di ottimizzazione del codice.

Teorema 2.1.3. *I codici istantanei sono univocamente decifrabili.*

Dimostrazione. Seguendo l'albero che definisce il codice, si sceglie il percorso in base ai simboli in input da decodificare. Arrivati a una foglia, una parola di codice è decodificata in modo univoco e si riparte quindi dalla radice. □

2.1.3 Rappresentazione ad Albero dei Codici

I codici binari possono essere intuitivamente modellati e analizzati tramite grafi ad albero, e più specificamente tramite alberi binari:

- La radice dell'albero rappresenta la stringa vuota λ.
- Da ogni nodo possono diramarsi al massimo due archi: convenzionalmente, l'arco sinistro è etichettato con il bit 0 e l'arco destro con il bit 1.
- Un percorso dalla radice a un nodo qualsiasi rappresenta una sequenza binaria, data dalla concatenazione delle etichette degli archi attraversati.

L'albero di un generico **codice UD non istantaneo** avrà alcune parole di codice su nodi intermedi. Quindi una parola di codice può essere il prefisso di una parola più lunga, posizionata più in profondità su un ramo discendente di quel cammino. Questo rende necessario attendere ulteriori bit prima di poter decodificare il simbolo senza ambiguità.

Al contrario, se il codice è **istantaneo (o a prefisso)**, nessuna parola può essere prefisso di un'altra. Sull'albero, questo si traduce in una regola geometrica assoluta: **le parole di codice si trovano esclusivamente sulle foglie**. Nessun nodo che rappresenta una parola di codice può avere figli.

Decodifica di un codice istantaneo

Avere le parole di codice confinate sulle foglie rende il processo di decodifica meccanico, immediato e privo di ritardo. Il decodificatore legge il flusso di bit in ingresso e, partendo dalla radice, scende lungo l'albero seguendo i rami (a sinistra se legge 0, a destra se legge 1). Non appena il cammino raggiunge una foglia, l'algoritmo emette il simbolo sorgente corrispondente e riparte dalla radice per decodificare il prossimo simbolo.

Codice Completo

Analizzando la struttura geometrica dell'albero, è possibile valutare l'efficienza della codifica. Se in un albero è presente un nodo interno dal quale si dirama un solo figlio, significa che si sta utilizzando un bit in più che non porta ad alcuna reale biforcazione logica o scelta, introducendo uno spreco nella lunghezza della parola.

Definizione 2.1.4. Un albero, e di conseguenza il codice istantaneo che esso rappresenta, si dice **completo** se ogni nodo interno ha esattamente due figli.

In un codice completo, lo spazio delle combinazioni binarie è sfruttato in modo ottimale, senza "rami morti" che allungherebbero inutilmente le sequenze trasmesse.

2.1.4 Algoritmo di Sardinas-Patterson

Questo algoritmo, proposto in origine nel 1953 da August Sardinas e George Patterson, fornisce un criterio per stabilire, in tempo polinomiale, se un codice è univocamente decifrabile. Per farlo, effettua una ricerca sistematica di una stringa che ammette la scomposizione in due codeword. Costruendo due possibili fattorizzazioni si possono presentare due casi:

- una fattorizzazione è contenuta in una parola dell'altra;
- una fattorizzazione comincia con un prefisso che a sua volta è suffisso di una delle altre parole dell'altra fattorizzazione.

Se un suffisso coincide con una parola di codice, allora il codice **non** è UD.

L'algoritmo si basa sul seguente teorema:

Teorema 2.1.5. *Dato un codice C su un alfabeto A, si considerano gli insiemi $S_0, S_1, \ldots, S_n$ tali che:*

- $S_0 = C$

- $S_i = \{w \in A^* | \exists a \in S_0, \exists b \in S_{i-1}\}$ *tale che* $a = bw \vee b = aw$

La condizione necessaria e sufficiente affinché C sia ***UD*** *è quindi:*

$$\forall n > 0, S_0 \cap S_n = \emptyset$$

Vale a dire che tra i suffissi generati nessuno dovrà corrispondere a una parola di codice. ■

Condizioni di arresto

L'algoritmo è implicitamente definito dal teorema; essendo l'insieme dei suffissi generabili *finito*, da un certo momento comincerà a generare insiemi già analizzati. Si hanno quindi tre condizioni di arresto:

1. $\exists i > 0, S_0 \cap S_i \neq \emptyset \implies$ il codice **NON è UD**
2. $\exists i > 0, S_i = \emptyset \implies$ il codice **è UD**
3. $\exists i, j > 0$ con $i \neq j, S_i = S_j \implies$ il codice **è UD**

La complessità dell'algoritmo è $O(rL)$, con r pari al numero di parole di codice ed L pari alla somma delle lunghezze di tali parole.

Esempio: Verifica di un codice con Sardinas-Patterson

Sia dato il codice $C = \{1, 011, 01110, 1110, 10011\}$. Vogliamo verificare se è univocamente decifrabile calcolando gli insiemi dei suffissi.
Passo 0: L'insieme iniziale corrisponde al codice stesso.

$$S_0 = \{1, 011, 01110, 1110, 10011\}$$

Passo 1 (S_1): Cerchiamo le parole in S_0 che sono prefisso di altre parole in S_0 ed estraiamo i suffissi:

- 1 è prefisso di 1110 $\implies$ il suffisso è 110
- 1 è prefisso di 10011 $\implies$ il suffisso è 0011
- 011 è prefisso di 01110 $\implies$ il suffisso è 10

Otteniamo quindi: $S_1 = \{110, 0011, 10\}$. Poiché $S_0 \cap S_1 = \emptyset$ (nessun suffisso è una parola del codice), l'algoritmo procede.
Passo 2 (S_2): Confrontiamo l'insieme S_0 con l'insieme S_1. Cerchiamo prefissi di S_0 nelle parole di S_1, e viceversa:

- 1 (da S_0) è prefisso di 110 (in S_1) $\implies$ il suffisso è 10
- 10 (da S_1) è prefisso di 10011 (in S_0) $\implies$ il suffisso è 011

Otteniamo quindi l'insieme: $S_2 = \{10, 011\}$.
Verifica e Arresto: Controlliamo l'intersezione tra l'ultimo insieme generato e quello di partenza:

$$S_0 \cap S_2 = \{011\} \neq \emptyset$$

Avendo trovato un suffisso (011) che corrisponde esattamente a una parola del codice originale, l'algoritmo si arresta per la **Condizione 1**. Possiamo affermare che il codice **NON** è univocamente decifrabile.

2.2 Disuguaglianze di Kraft e Kraft-McMillan

Avendo a disposizione un alfabeto sorgente con m simboli e un alfabeto di codice Γ di dimensione d, è sempre possibile realizzare un codice *istantaneo* o *UD* in cui le parole abbiano specifiche lunghezze $l_1, l_2, \ldots, l_m$? No, non è *sempre* possibile! Per garantire la decifrabilità, le lunghezze scelte non possono essere arbitrarie, ma devono rispettare una precisa condizione.

Definiamo prima la quantità $K(C)$ conosciuta come Somma di Kraft:

Definizione 2.2.1 (Somma di Kraft)**.** Data una codifica $C = \{c_1, c_2, \ldots c_m\}$ su un alfabeto A di cardinalità $|A| = d$, siano $l_i = |c_i|$ le lunghezze delle rispettive parole di codice.

Si definisce la Somma di Kraft la quantità:

$$K(C) = \sum_{i=1}^{m} d^{-l_i}$$

Ad esempio:

$$C = \{00, 001, 10, 11\}, A = 0, 1 \rightarrow K(C) = 2^{-2} + 2^{-3} + 2^{-2} + 2^{-2} = \frac{7}{8}$$

2.2.1 Diseguaglianza di Kraft (per codici *istantanei*)

Introdotta nel 1949 da Leon Kraft questa diseguaglianza è di fondamentale importanza per la codifica; definisce una condizione da rispettare per le lunghezze delle parole di un codice istantaneo.

Teorema 2.2.2. *Per qualsiasi codice **istantaneo (a prefisso)** su un alfabeto di dimensione d, le lunghezze delle parole di codice $l_1, l_2, \ldots, l_m$ devono necessariamente soddisfare la relazione:*

$$K(C) = \sum_{i=1}^{m} d^{-l_i} \leq 1$$

***Specularmente**, se la relazione è soddisfatta per un insieme di lunghezze di parole di codice, allora esiste un codice istantaneo con quelle lunghezze di parole di codice.*

Attenzione: La relazione è una condizione ***necessaria e sufficiente per l'esistenza di un codice istantaneo con quelle lunghezze***. Se un codice ha l'insieme delle lunghezze delle parole che rispetta la diseguaglianza di Kraft, **non è automatico** che il codice sia istantaneo; sarà comunque necessario verificare che sia istantaneo.

Diseguaglianza di Kraft - dimostrazione condizione sufficiente

Dimostriamo la **condizione sufficiente per l'esistenza di un codice istantaneo**.

Dimostrazione. Per ogni insieme di lunghezze $l_1, l_2, \ldots, l_m$, se la diseguaglianza di Kraft è soddisfatta, si può **sempre** costruire un albero, di *ordine d*.

Indichiamo con:

- n_1 il numero dei *valori* di lunghezza pari a 1;
- n_2 il numero dei *valori* di lunghezza pari a 2;
- ...
- n_L il numero dei *valori* di lunghezza pari a L (il valore massimo);

Passo 1. Sia $L = \max(l_i)$, con $1 \leq i \leq m$

Passo 2. $\sum_{i=1}^{m} n_i = m$, con m il numero di parole di codice

Passo 3. Per ipotesi vale la diseguaglianza di Kraft

$$\sum_{i=1}^{m} d^{-l_i} = \sum_{i=1}^{L} n_i d^{-i} \leq 1$$

Ovvero:

$$n_1 d^{-1} + n_2 d^{-2} + n_3 d^{-3} + \cdots + n_L d^{-L} \leq 1$$

Passo 4. Si moltiplica per d:

$$n_1 + n_2 d^{-1} + n_3 d^{-2} + \cdots + n_L d^{-L+1} \leq d$$

Passo 5. Vale per il primo elemento che $n_1 \leq d$, in quanto a profondità 1 dalla radice saranno possibili al massimo d diramazioni, quindi:

$$n_2 d^{-1} + n_3 d^{-2} + \cdots + n_L d^{-L+1} \leq d - n_1$$

Passo 6. Si moltiplica ancora per d:

$$n_2 + n_3 d^{-1} + \cdots + n_L d^{-L+2} \leq d(d - n_1)$$

Passo 7. Vale che $n_2 \leq d(d - n_1)$, si moltiplica ancora per d, procedendo come in precedenza

Passo 8. Si deduce che $n_1 \leq d, n_2 \leq d(d - n_1), n_3 \leq d(d^2 - n_1 d - n_2)$ *e così via*

Passo 9. Questo insieme di diseguaglianze assicura che **è possibile** costruire un albero *d-ario* che per ogni livello a profondità i dalla radice ha:

(a) n_1 foglie a profondità 1 e $d - n_1$ nodi interni

(b) n_2 foglie a profondità 2 e $d(d - n_1) - n_2$ nodi interni

(c) n_3 foglie a profondità 3 e $d(d^2 - n_1 d - n_2) - n_3$ nodi interni

(d) ...

(e) n_L foglie di profondità massima L

Passo 10. Si ottiene alla fine un albero con $n_1 + n_2 + \cdots + n_L$ foglie; ad ogni foglia corrisponde una parola di codice ottenuta concatenando i simboli presenti sul percorso per raggiungere la foglia.

A profondità crescenti, aumenta il numero di scelte possibili per i cammini e le rispettive foglie.

□

2.2.2 Disuguaglianza di McMillan (per codici UD)

McMillan pochi anni dopo Kraft propose una generalizzazione relativa a *tutti* i codici univocamente decifrabili della diseguaglianza di Kraft.

Teorema 2.2.3. *Per un qualsiasi codice univocamente decifrabile su un alfabeto di cardinalità d, le* $l_1, l_2, \ldots l_m$ *lunghezze delle parole* ***devono*** *soddisfare la diseguaglianza di Kraft:*

$$\sum_{i=1}^{m} d^{-l_i} \leq 1$$

Specularmente, se la relazione è soddisfatta per un insieme di lunghezze, allora *esiste* un codice istantaneo con quelle lunghezze di parole.

Condizione Necessaria (Dimostrazione di McMillan)

Mentre la condizione di Kraft garantisce che la diseguaglianza sia **sufficiente per costruire un codice istantaneo**, la **condizione necessaria** stabilisce che **ogni** codice univocamente decifrabile (UD) deve **obbligatoriamente soddisfare tale vincolo**.

Non è quindi possibile trovare un codice UD le cui lunghezze delle parole violino la diseguaglianza di Kraft. Per dimostrare questa necessità, si utilizza l'approccio algebrico di McMillan.

Dimostrazione. Sia C un codice univocamente decifrabile su un alfabeto di ordine d e con lunghezze $l_1, l_2, \ldots, l_m$, con L lunghezza massima.

Consideriamo la quantità elevata alla potenza intera arbitraria n:

$$\left(\sum_{i=1}^{m} d^{-l^i} \right)^n = \left(d^{-l^1} + d^{-l^2} + \cdots + d^{-l^m} \right)^n$$

Sviluppando la potenza si ottengono m^n termini, ognuno dei quali è nella forma $d^{-l_{i_1}} d^{-l_{i_2}} \ldots d^{-l_{i_n}} = d^{-k}$, dove k è la lunghezza totale di una sequenza di n parole di codice (con $n \leq k \leq nL$).

Raggruppando i termini con lo stesso esponente k, possiamo riscrivere lo sviluppo come:

$$\left(\sum_{i=1}^{m} d^{-l_i} \right)^n = \sum_{k=n}^{nL} N_k d^{-k}$$

dove N_k è il numero di sequenze di n parole di codice aventi lunghezza totale k.

Essendo il codice UD, ogni sequenza di parole è decifrabile in modo univoco. Pertanto, il numero di sequenze distinte di lunghezza k non può superare il totale delle combinazioni possibili con d simboli su lunghezza k, ovvero $N_k \leq d^k$.

Sostituendo questa diseguaglianza otteniamo:

$$\left(\sum_{i=1}^{m} d^{-l_i}\right)^n \leq \sum_{k=n}^{nL} d^k d^{-k} = \sum_{k=n}^{nL} 1 = nL - n + 1 \leq nL$$

Estraendo la radice n-esima si ha per ogni n:

$$\sum_{i=1}^{m} d^{-l_i} \leq (nL)^{\frac{1}{n}}$$

Dato che $\lim_{n\to\infty}(nL)^{\frac{1}{n}} = 1$, si conclude che:

$$\sum_{i=1}^{m} d^{-l_i} \leq 1$$

□

2.2.3 Teorema di Kraft-McMillan (per codici UD)

Nel 1956, McMillan estese la validità della diseguaglianza di Kraft all'intera classe dei codici univocamente decifrabili (UD), dimostrando che il vincolo sulle lunghezze non è limitato ai soli codici istantanei.

Teorema 2.2.4. *Condizione **necessaria e sufficiente** affinché esista un codice univocamente decifrabile (UD) con lunghezze delle parole $l_1, l_2, \ldots, l_m$ su un alfabeto di dimensione d è che sia soddisfatta la relazione:*

$$\sum_{i=1}^{m} d^{-l_i} \leq 1$$

Questa dimostrazione della condizione necessaria consente di concludere che se la diseguaglianza di Kraft-McMillan **non è soddisfatta** per un insieme di lunghezze, **non può esistere alcun codice UD** avente parole di codice con tali lunghezze.

Ciò implica che non vi è alcun vantaggio, in termini di compressione o brevità delle parole, nel rinunciare alla *proprietà di istantaneità*: i limiti matematici per i codici UD sono gli stessi dei codici a prefisso.

2.3 Lunghezza Media e Teorema di Shannon

Dopo aver stabilito le proprietà necessarie per avere un codice univocamente decifrabile, ci possiamo porre il problema di scegliere il *migliore* tra un insieme di codici UD. Per farlo ci serve una metrica, stabilita la quale troveremo quali sono i limiti *invalicabili* che devono rispettare i codici.

2.3.1 Lunghezza media e Codice Ottimo

Avendo stabilito le condizioni necessarie e sufficienti per garantire l'univocità della decodifica, l'obiettivo ora è quello di avere codici *UD* con la minore lunghezza **media** possibile; introduciamo quindi la L_S come metrica per valutarli. Tra tutti i codici **UD** per una data sorgente e un dato alfabeto di codice, quello con la L_S minore è detto **codice compatto**.

Dati:

- C codice con parole $c_1, c_2, \ldots, c_m$ di lunghezza $l_1, l_2, \ldots, l_m$
- S alfabeto di sorgente
- X alfabeto di codice
- $p_1, p_2, \ldots, p_m$ le probabilità dei simboli della sorgente

Si definisce **lunghezza media del codice**:

$$L_S(C) = \sum_{i=1}^{m} p_i l_i$$

2.3.2 Primo Teorema di Shannon

Abbiamo stabilito qual è il nostro obiettivo per i codici univocamente decifrabili, ora è di fondamentale importanza determinare come individuarlo. Il primo teorema di Shannon fornisce un limite inferiore per la compressione senza perdita di informazione.

Teorema 2.3.1 (Primo Teorema di Shannon)**.** *Sia C un codice univocamente decifrabile per una sorgente senza memoria S con probabilità $p_1, p_2, \ldots, p_m$, alfabeto X di cardinalità d, allora*

$$L_S(C) \geq \frac{H(S)}{\log d}$$

Quindi per codici binari la L_S deve essere almeno pari all'entropia della sorgente. Considerando la definizione di entropia per la quale la base del logaritmo è d, allora si ha che

$$L_S(C) \geq H_d(S)$$

Dimostrazione.

- Se C è un codice **UD**, allora $K(C) \leq 1$
- Dal Lemma del Logaritmo si prova che $\sum_{i=1}^{m} p_i \log \frac{1}{p_i} \leq \sum_{i=1}^{m} p_i \log \frac{1}{q_i}$, con l'uguaglianza valida solo se $q_i = p_i \forall i$
- Si considera in particolare $q_i = \frac{d^{-l_i}}{K(C)} = \frac{d^{-l_i}}{\sum_{i=1}^{m} d^{-l_i}}$ (*normalizzazione* per d^{-l_i})
- Se ne deduce che:

$$\begin{aligned} H(S) = -\sum_{i=1}^{m} p_i \log p_i &\leq \sum_{i=1}^{m} p_i \log \frac{1}{q_i} \\ &= -\sum_{i=1}^{m} p_i \log q_i = -\sum_{i=1}^{m} p_i \log \frac{d^{-l_i}}{K(C)} \\ &= -\sum_{i=1}^{m} p_i \left(\log d^{-l_i} - \log K(C)\right) \\ &= -\sum_{i=1}^{m} p_i \log d^{-l_i} + \sum_{i=1}^{m} p_i \log K(C) \\ &= \sum_{i=1}^{m} p_i l_i \log(d) + \underbrace{\log K(C)}_{\leq 0} \leq L_S(C) \log(d) \end{aligned}$$

Quindi $H(S) \leq L_S(C)\log(d) \rightarrow \frac{H(S)}{\log(d)} \leq L_S(C)$

In caso di codice **UD** per cui vale $\sum_{i=1}^{m} d^{-l_i} = 1$, l'uguaglianza si ha solo se $l_i = \log_d(\frac{1}{p_i})$ e quindi $L_S(C) = H(S)$. Ciò avviene se e solo se $\log_d(\frac{1}{p_i})$ è un valore **intero** per ogni i. □

2.4 Parametri di Trasmissione ed Efficienza

2.4.1 Notazione e Rate di Trasmissione

- r è la **symbol rate**, misurata in $\frac{\text{simboli}}{\text{secondo}}$
- R è la **information Rate**, misurata in $\frac{\text{bit}}{\text{secondo}}$
- r_b è il **signalling rate** (o **bit rate**), misurato in $\frac{\text{binit}}{\text{secondo}}$

Per una sorgente che emette n simboli (con $n \gg 1$), l'informazione totale sarà pari a *circa* R; la *velocità di informazione* della sorgente è:

$$R = \frac{n \cdot H(X)}{\frac{n}{r}} = r \cdot H(X) \left[\frac{\text{bit}}{\text{secondo}}\right]$$

Dal teorema di Shannon un'informazione proveniente da una sorgente discreta senza memoria può essere codificata con **cifre binarie** e trasmessa su di un canale senza rumore con un bit rate r_b vincolato dalla relazione

$$r_b \geq R \left[\frac{\text{binit}}{\text{secondo}} \right]$$

Data una sorgente discreta senza memoria con alfabeto di M simboli, il suo information rate R sarà:

- $R = r \log_2(M)$ se i simboli sono **equiprobabili** $\rightarrow$ l'informazione è trasmessa con efficienza con simboli *M-ari* e symbol rate r.
- $R = rH(X) < r \log_2(M)$ con simboli **NON equiprobabili** $\rightarrow$ la strategia ideale consiste nel tenere conto delle diverse probabilità (e contenuto informativo) dei simboli, per poter trasmettere ad una velocità prossima ad R.

Information rate e codifica binaria

Per le codifiche binarie dei simboli di una sorgente, l'output del codificatore sarà equivalente a quello di una sorgente binaria con **entropia** $\Omega(p)$ e information rate $R = r_b\Omega(p)$, per un opportuno valore di p. Un codificatore di per sé non modifica il valore informativo, quindi $R_{\text{ingresso}} = R_{\text{uscita}}$:

$$R_{\text{ingresso codificatore}} = rH(X) = r_b\Omega(p) \leq r_b$$

L'output del codificatore può essere visto come una nuova sorgente binaria che emette r_b simboli/s, la cui entropia per simbolo è $\Omega(p)$.

2.4.2 Efficienza dei codici e ridondanza

Di una sorgente abbiamo il valore dell'entropia che ci indica il numero di simboli che in **media** servono per rappresentarne i simboli e la lunghezza media del codice scelto per codificare la sorgente; da questi due valori deriviamo l'efficienza del codice η che :

$$\eta = \frac{H_d(S)}{L_S(C)}$$

L'efficienza η indica quanto il codice si avvicina al limite di massima compressione. Un valore pari a 1 (100%) significa che il codice è perfetto e non c'è alcuno spreco di simboli.

Dal primo teorema di Shannon segue la relazione

$$0 \leq \frac{H_d(S)}{L_S(C)} \leq 1$$

Di riflesso, dall'efficienza si ottiene il valore della **ridondanza** del codice $\rho(C)$

$$\rho(C) = 1 - \eta(C) = \frac{L_S(C) - H_d(S)}{L_S(C)}$$

La ridondanza rappresenta la percentuale di *bit sprecati* dal codice, ovvero i simboli usati in eccesso rispetto al minimo teorico indispensabile stabilito dall'entropia.

Identifichiamo tre classi di efficienza per i codici:

- Codice **assolutamente ottimo** $\rightarrow L_S(C) = H(S)$
- Codice **ottimo** $\rightarrow$ quando si ottiene il minimo valore possibile per $L_S(C)$, anche se maggiore dell'entropia della sorgente
- Codice **sub-ottimo** $\rightarrow$ quando la lunghezza media è maggiore di quella di un codice ottimo per la stessa sorgente

Nota: Un codice può essere ottimo, ma non essere assolutamente ottimo (ovvero senza raggiungere l'entropia). Questo accade quasi sempre quando le probabilità non sono potenze di $\frac{1}{d}$.

I rapporti $\frac{R}{r_b}$ e $\frac{H(S)}{L_S(C)} \leq 1$ sono *equivalenti* in quanto:

$$R = r \cdot H(S) \quad r_b = r \cdot L_S(C) \implies \frac{r \cdot H(S)}{r \cdot L_S(C)} = \frac{H(S)}{L_S(C)} = \eta$$

Tale rapporto rappresenta una misura dell'efficienza di un codice ottimo o sub-ottimo.

L'efficienza si definisce come

$$\text{Efficienza } \% = \frac{H(S)}{L_S(C)} \cdot 100$$

Condizioni necessarie per codici assolutamente ottimi

Per essere **assolutamente ottimo** si deve avere che $K(C) = 1$ e quindi le probabilità dei simboli del codice sono tali che:

$$p_i = \frac{1}{2^{l_i}}, i = 1, 2, \dots m$$

Pertanto un codice **assolutamente ottimo** avrà

$$L_S(C) = -\log(p_i) = I_i I(\sigma_i) = \log_b \left(\frac{1}{p_i}\right)$$

Dimostrazione. Con $K(C) = 1$ le distribuzioni di probabilità sono $\{2^{-l_i}\}$

- $$p_i = \frac{1}{2^{l_i}} \to \log \frac{1}{p_i} = l_i$$
- $$L_S(C) = \sum_{i=1}^{m} p_i l_i = H(S) = \sum_{i=1}^{m} p_i I_i = \sum_{i=1}^{m} p_i \log \frac{1}{p_i}$$

Ciò fa capire che la strategia da adottare per la codifica di sorgente consista nell'assegnare a simboli con probabilità più alta le parole di codice più corte rispetto ai simboli meno probabili. □

Quando $L_S(C) = H_d(S)$?

Per un codice **UD** con alfabeto *d-ario*, $L_S(C) = H(S) \iff \sum_{i=1}^{m} d^{-l_i} = 1$ e $l_i = \log_d \left(\frac{1}{p_i}\right)$. Questo può avvenire soltanto se gli l_i hanno tutti un valore intero, ovvero $p_i = \left(\frac{1}{d}\right)^{\alpha_i}$, con α_i intero. Scegliendo $l_i = \alpha_i$, si ottiene un **codice compatto** e vale

$$\begin{aligned} 1 = \sum_{i=1}^{m} p_i &= \sum_{i=1}^{m} \left(\frac{1}{d}\right)^{\alpha_i} \\ &= \sum_{i=1}^{m} d^{-\alpha_i} = \sum_{i=1}^{m} d^{-l_i} \end{aligned}$$

Capitolo 3

Metodi e Algoritmi per la Compressione dell'Informazione

Definito l'impianto teorico che governa la codifica di sorgente, passiamo ad analizzare i metodi con i quali si persegue l'obiettivo di rendere più efficiente la trasmissione e la memorizzazione dell'informazione.

Esistono due approcci distinti per affrontare il problema. Storicamente il primo è quello che si basa sulla **analisi statistica** delle informazioni per proporre una codifica più efficiente, il secondo invece evita l'approccio analitico a priori, appoggiandosi a un **dizionario** per abbassare la ridondanza dei dati. Non sono comunque approcci del tutto antitetici, in quanto spesso le codifiche di tipo *statistico* possono risultare efficaci in combinazione con algoritmi a dizionario.

Altra differenza fondamentale tra i due approcci è il fatto che i metodi statistici operano sui singoli simboli, riducendo la *ridondanza di codifica*, mentre quelli a dizionario manipolano sequenze ripetute di simboli, affrontando la *ridondanza tra sequenze di simboli*. Questo comporta notevoli differenze nei risultati ottenibili in certe condizioni.

Esiste poi un terzo approccio, quello della trasformata Burrows-Wheeler, che senza un'analisi statistica, o conoscenza a priori, sfrutta le proprietà statistiche *intrinseche* dei dati per realizzare un pre-processing dei dati in grado di favorire il lavoro degli algoritmi di compressione.

3.1 Metriche di Valutazione e Vincolo Lossless

È fondamentale premettere che in questo capitolo tratteremo esclusivamente algoritmi di compressione **lossless** (senza perdita di informazione), in cui la sequenza decodificata risulta bit-a-bit identica a quella originale. Per valutare e confrontare le performance di questi algoritmi (indipendentemente dal loro approccio), si utilizzano metriche universali:

- **Rapporto di Compressione (Compression Ratio - CR):** È il rapporto tra la dimensione dei dati originali e quella dei dati compressi ($CR = \frac{\text{Dim. Originale}}{\text{Dim. Compressa}}$). Un CR di 4 indica che i dati compressi occupano il 25% dello spazio originale.
- **Risparmio di Spazio (Space Savings):** Misura la percentuale di spazio effettivamente risparmiata rispetto all'originale ($1 - \frac{1}{CR}$).
- **Bit rate medio:** Misurato in bit/simbolo, quantifica il numero medio di bit spesi per codificare un singolo carattere. Nei codici statistici, coincide con la lunghezza media $L_S(C)$.

3.2 Algoritmi statistici

Gli algoritmi statistici sono diretta conseguenza dell'introduzione del concetto di entropia e i successivi studi a riguardo. Alla base del loro funzionamento c'è l'analisi delle frequenze di occorrenza dei singoli caratteri emessi dalla sorgente.

L'obiettivo matematico di questo approccio è minimizzare la lunghezza media del codice $L_S(C)$, abbattendo la *ridondanza di codifica* per avvicinarsi il più possibile al limite teorico inferiore imposto dal Primo Teorema di Shannon (ovvero l'entropia $H(S)$ della sorgente).

Per raggiungere questo obiettivo si sfrutta un principio cardine tanto semplice quanto potente:

assegnare parole di codice (codeword) più corte ai simboli che compaiono con maggiore frequenza, e codeword più lunghe a quelli più rari.

Fasi operative. A livello implementativo, l'applicazione di un compressore statistico *puro* richiede tipicamente due fasi distinte:

1. **Costruzione del modello (Statistica):** Una prima scansione dei dati (o l'assunzione di un modello a priori) per calcolare la probabilità p_i di ciascun simbolo dell'alfabeto.
2. **Codifica:** La costruzione vera e propria dell'assegnamento (spesso rappresentata tramite un albero binario) e la successiva traduzione del flusso di simboli in una sequenza continua di bit.

3.2.1 Codifica di Shannon

Per provare la validità del teorema della codifica di sorgente (*Primo Teorema di Shannon*), Shannon ha ideato questa tecnica di compressione lossless per costruire codici prefissi. Sfrutta la conoscenza, stimata o misurata, dei simboli dell'alfabeto di sorgente.

Come vedremo, ci sono codifiche che risulteranno più efficienti, ma questa è stata la prima di questo tipo.

Teorema 3.2.1. *Data S sorgente senza memoria, se $\forall i \quad 1 \leq i \leq m$ si sceglie l_i come l'**intero** t.c. $l_i = \lceil \log_d \frac{1}{p_i} \rceil$, allora si può trovare un codice prefisso C t.c.*

$$\log_d \frac{1}{p_i} \leq l_i < \log_d \frac{1}{p_i} + 1$$

$$H_d(S) \leq L_S(C) < H_d(S) + 1$$

Dimostrazione. La relazione $log_d \frac{1}{p_i} \leq l_i \leq \log_d \frac{1}{p_i} + 1$ vale solo se si possono trovare valori di l_i che soddisfano la diseguaglianza di Kraft-McMillan. Sommando su tutti i valori di i a destra e a sinistra, si ottiene la diseguaglianza di Kraft-McMillan. Notare che:

$$\begin{aligned} \log_d \frac{1}{p_i} \leq l_i &\rightarrow -l_i \leq \log_d p_i \\ &\rightarrow d^{-l_i} \leq p_i \\ &\rightarrow \sum_{i=1}^{m} d^{-l_i} \leq 1 \end{aligned}$$

Quindi tale codice può essere costruito. Moltiplicando per p_i la diseguaglianza $\log_d(\frac{1}{p_i}) \leq l_i < \log_d \frac{1}{p_i} + 1$ e sommando per ogni i, si ottiene la tesi:

$$H(S) \leq L_S(C) \leq H(S) + 1$$

□

Non ottimalità della codifica di Shannon. Non è garantito che vengano generati dei codici compatti. Ad esempio:

S	Prob.	$\log(1/p_i)$
S_0	$\frac{2}{3}$	$\log_2\left(\frac{3}{2}\right) \approx 0.58$
S_1	$\frac{2}{9}$	$\log_2\left(\frac{9}{2}\right) \approx 2.17$
S_2	$\frac{1}{9}$	$\log_2(9) \approx 3.17$

Tabella 3.1: Esempio non ottimale

La codifica di Shannon propone lunghezze per le parole di codice $\{1, 3, 4\}$, ottenendo $L_S(C) = 1.78$. Quindi C **non** è compatto, perché ad esempio un codice C' con lunghezze $\{1, 2, 2\}$ avrebbe $L_S(C') = 1.33$. Ma essendo l'entropia pari a $H(S) \cong 1.22$, anche C' non è ottimale.

Estensione della sorgente

Possiamo migliorare l'efficienza della codifica quando la semplice codifica simbolo per simbolo non dà buoni risultati? Sì, possiamo farlo intervenendo sui simboli della sorgente.

Shannon dimostra che ci si può avvicinare all'entropia codificando non più i singoli simboli di S, ma blocchi di più simboli, ottenendo una nuova sorgente S'.
Si definisce S' **estensione di ordine** n di una sorgente S, se i suoi simboli sono ottenuti raggruppando i simboli originari in sequenze di lunghezza n. Codificando S' non si avrà perdita di informazione. Codificando S'_n con un codice**UD** istantaneo, questo soddisferà la seguente condizione:

$$H(S) \leq L_S(C) \leq H(S) + \frac{1}{n}$$

Se ne conclude che il meccanismo di estensione della sorgente consente, agendo sul valore di n, di avvicinarsi al *limite di Shannon* quanto si desidera. In particolare,

$$\lim_{n\to\infty} L_n = H(S)$$

Osservazione. In pratica aumentare n per avvicinarsi all'entropia comporta un significativo aumento di complessità del decodificatore e inevitabilmente un ritardo di trasmissione.

Generalizzazione. Si dimostra che $H_d(S^n) = n \cdot H_d(S)$ Inoltre, $H_d(S^n) \leq L_{S^n} < H_d(S^n) + 1$. Considerando per ogni simbolo di S la lunghezza media, si ottiene

$$H_d(S) = \frac{H_d(S^n)}{n} \leq \frac{L_{S^n}(C)}{n} < \frac{H_d(S^n)}{n} + \frac{1}{n} = H_d(S) + \frac{1}{n}$$

Con L_n si indica la lunghezza media per i simboli di S, ovvero:

$$L_n = \frac{L_{S^n}(C)}{n}$$

Quindi si realizza che per blocchi sufficientemente grandi di simboli, si può raggiungere l'entropia

$$\lim_{n\to\infty} L_n = H_d(S)$$

Problema: il numero dei simboli della sorgente può crescere a dismisura. Scegliendo un valore per n anche sufficientemente grande, non è comunque possibile sapere se e quanto si raggiunge il valore ottimale.

Serve quindi un modo per trovare codici compatti (**ottimali**) per la codifica delle sorgenti senza memoria.

3.2.2 Codifica di Shannon-Fano (nel caso binario)

Sviluppata indipendentemente da Claude Shannon e Robert Fano, questa tecnica rappresenta il primo approccio algoritmico pratico per la costruzione di un codice prefisso *efficiente* basato sulle probabilità. La codifica di Shannon-Fano opera in modo costruttivo, adottando una strategia **top-down** (dall'alto verso il basso).

L'idea centrale è quella di dividere iterativamente l'alfabeto della sorgente, precedentemente ordinato in ordine di probabilità non crescente, in due sottoinsiemi con probabilità cumulativa il più possibile simile; ad ogni sottoinsieme/ramo si assegna un bit distinto (0 o 1). Questa procedura *euristica* cerca di bilanciare l'albero di codifica in modo naturale secondo le probabilità dei simboli. Tuttavia, pur garantendo prestazioni notevoli, l'algoritmo si rivela **sub-ottimale**: non assicura matematicamente la minima lunghezza media possibile per la sorgente, difetto che verrà risolto in seguito dall'algoritmo di Huffman.

Algoritmo:

1. Si elencano i simboli di S in ordine di probabilità **non** crescente.
2. Si divide la lista formando due gruppi di simboli tali che le somme di probabilità siano *quasi* uguali.
3. Si aggiunge una cifra 0 a destra del primo gruppo, si aggiunge 1 a destra di quelle del secondo gruppo.
4. Ciascuno dei due gruppi è suddiviso con lo stesso criterio, appendendo i nuovi simboli alla codifica.
5. Si continua fino a che ogni gruppo è costituito da un solo elemento.

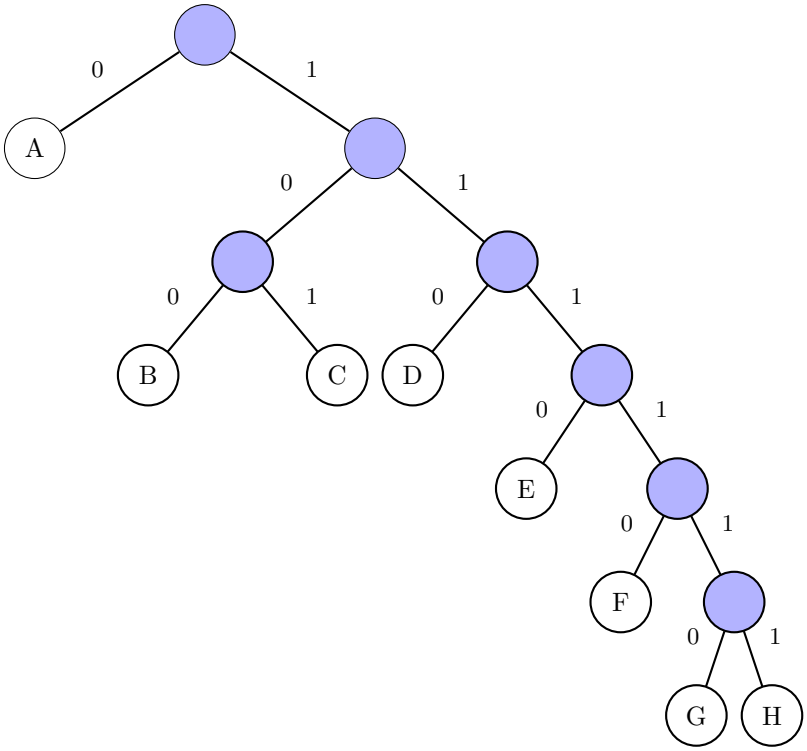
Algorithm 1 Costruzione Codice Shannon-Fano

Require: Una lista di simboli S con le loro probabilità P
Ensure: Un codice binario per ogni simbolo in S

function ShannonFano(Lista L)
  if $|L| > 1$ **then** ▷ Passo 5: Continua solo se il gruppo ha più di 1 elemento
    Ordina L in base alle probabilità in ordine **non** crescente ▷ Passo 1
    Dividi L in due sottogruppi L_1 e L_2
    tali che $\sum_{s \in L_1} P(s) \approx \sum_{s \in L_2} P(s)$ ▷ Passo 2
    for ogni simbolo $s \in L_1$ **do**
      s.codice $\leftarrow s$.codice + '0' ▷ Passo 3 (gruppo 0)
    end for
    for ogni simbolo $s \in L_2$ **do**
      s.codice $\leftarrow s$.codice + '1' ▷ Passo 3 (gruppo 1)
    end for
    ShannonFano(L_1) ▷ Passo 4: Ricorsione sul primo gruppo
    ShannonFano(L_2) ▷ Passo 4: Ricorsione sul secondo gruppo
  end if
  return ▷ Se $|L| = 1$, la ricorsione per questo ramo termina
end function

Inizializza i codici di tutti i simboli $s \in S$ a stringa vuota
ShannonFano(S) ▷ Avvia la procedura sull'intera lista

x_i	P_i	Passi						Codeword
		1	2	3	4	5	6	
A	0.50	0						0
B	0.15	1	0	0				100
C	0.15	1	0	1				101
D	0.08	1	1	0				110
E	0.08	1	1	1	0			1110
F	0.02	1	1	1	1	0		11110
G	0.01	1	1	1	1	1	0	111110
H	0.01	1	1	1	1	1	1	111111
$H(X) = 2.15$								$L = 2.18$



	p_i	$\frac{1}{p_i}$	$\log_2\left(\frac{1}{p_i}\right)$	$l_i \cdot p_i$
A	0.50	2	1	0.50
B	0.15	6.66	2.736964	0.45
C	0.15	6.66	2.736964	0.45
D	0.08	12.5	3.643856	0.24
E	0.08	12.5	3.643856	0.32
F	0.02	50	5.643856	0.10
G	0.01	100	6.643856	0.06
H	0.01	100	6.643856	0.06

Il valore dell'entropia è inferiore, ma la lunghezza media del codice è molto vicina al suo valore.

Non ottimalità

La codifica di Shannon-Fano non è ottimale; non è garantito che produca sempre la minima lunghezza media possibile per una data sorgente. In questo si differenzia dalla codifica di Huffman.

Esempio:

- Con symbol rate $r = 1000[\frac{simboli}{secondo}]$, la information rate sarebbe $R = r \cdot H(S) = 2150[\frac{bit}{secondo}]$.
- Un codice Shannon-Fano con $L_S = 2.18$ richiede un bit rate $r_b = 2180[\frac{bit}{sec}]$ (notare che $r_b > R$, come previsto per codici sub-ottimi).
- L'efficienza sarà:

$$\eta = \frac{H(S)}{L_S} \cdot 100 = \frac{2.15}{2.18} \cdot 100 = 98.6\%$$

Un risultato comunque notevole, basti pensare che se si fosse utilizzata una codifica a lunghezza di parole fissa si sarebbe ottenuto un risultato decisamente peggiore:

$$r_b = L_S \cdot r = 3 \cdot 1000 = 3000 \left[\frac{binit}{sec}\right]$$

$$\eta = \frac{H(S)}{L_S} \cdot 100 = \frac{2.15}{3} = 71.6\%$$

3.2.3 Condizioni per ottimalità

Si può dimostrare che, affinché un codice ***UD* istantaneo** sia ottimo, si devono verificare queste condizioni:

1. A simboli di S meno probabili sono assegnate parole di codice più lunghe
$$p_i > p_j \Rightarrow l_i \leq l_j$$
2. Le due parole di codice meno probabili, quindi più lunghe, devono avere la stessa lunghezza
$$l_{M-1} = l_M$$
3. Delle parole di codice della stessa lunghezza, almeno due devono coincidere per tutti i bit tranne l'ultimo

Dimostrazione. 1. Dato C un codice **UD istantaneo ottimo** per cui non vale la condizione 1, si costruisce C' che differisce per la codifica di due parole che invalidano la condizione 1. Di conseguenza $L_{C'} < L_C$; si ripete il procedimento fino a che la prima condizione è rispettata.

2. Se C è**UD**, nessuna parola è prefisso di un'altra. Se la 2 non è rispettata, allora $l_M > l_{M-1}$; invertendo l'ultimo bit di X_{M-1} il risultato è una parola non appartenente al codice e di lunghezza l_{M-1} che può sostituire l_M.

3. Si può accorciare di un bit la codifica di una parola se non vale la condizione 3.

□

3.2.4 Codifica di Huffman

La strategia di codifica proposta da David Huffman produce un codice ottimale dal punto di vista della lunghezza media, ovvero produce il minimo valore possibile per $L_S(C)$, approssimando il più possibile il limite imposto dal primo teorema di Shannon. Non produce necessariamente un codice assolutamente ottimo.

L'idea è nata grazie a un problema assegnato dal suo professore, Robert Fano, che chiedeva di trovare il codice binario più efficiente possibile. All'idea già usata da Shannon e Fano di assegnare ai simboli della sorgente con minore frequenza parole di codice più lunghe, Huffman propose di usare un albero binario ordinato secondo le frequenze dei simboli.

L'algoritmo, con un approccio ***bottom-up***, produce un **albero binario completo** (*albero di Huffman*) che identifica il codice. I codici di Huffman vengono usati in molte tecniche di compressione e, a seconda del tipo di input, consentono un risparmio tra il 20% e il 90%.

Codici prefissi ottimali

Sia C un codice **prefisso compatto** (o **ottimale**) per S t.c. c_i è la parola di codice associata al simbolo con probabilità p_i, e $p_1 \geq p_2 \geq \cdots \geq p_m$ Allora:

1. $p_j \geq p_k \rightarrow |c_k| \geq |c_j|$
2. $|c_m| = |c_{m-1}|$ ovvero i simboli meno probabili hanno codeword della stessa lunghezza.

   Se $|c_m| > |c_{m-1}|$ basta rimuovere a c_m l'ultimo simbolo, ottenendo un altro codice prefisso ma di lunghezza inferiore.
3. **Almeno** due codewords tra quelle di lunghezza massima, differiscono **solo** per l'ultimo simbolo (bit).

Se tutte le parole di lunghezza massima M differissero per un simbolo che non è l'ultimo, queste potrebbero essere sostituite dal prefisso di lunghezza $M-1$, e si avrebbe ancora un codice prefisso con una L_S minore.

Algoritmo di Huffman

Sia data la sorgente $S = \begin{pmatrix} s_1 & s_2 & \dots & s_m \\ p_1 & p_2 & \dots & p_m \end{pmatrix}$ con le probabilità in ordine $p_1 \geq p_2 \geq \dots \geq p_m$

- Si indica con $R(S)$ la **sorgente ridotta**, che si ottiene sostituendo i simboli meno probabili con un simbolo la cui proprietà è la somma della probabilità dei simboli che ha sostituito:

$$R(S) = \begin{pmatrix} s_1 & s_2 & \dots & (s_{m-1}, s_m) \\ p_1 & p_2 & \dots & p_{m+1} + p_m \end{pmatrix}$$

- Sia C_R un codice prefisso per $R(S)$ e z la parola di codice associata al simbolo (s_{m-1}, s_m). Il codice C è ottenuto assegnando ad ogni parola della sorgente S la stessa parola corrispondente per il simbolo di $R(S)$ e usando $z0$ per c_m e $z1$ per c_{m-1}, è un codice prefisso.

  Inoltre, $L_S(C) = L_{R(S)}(C_R) + p_{m-1} + p_m$

- Se C_R è ottimale, anche C è ottimale.

  Se non fosse così dovrebbe esistere un codice prefisso ottimale (compatto) C' per S t.c.

$$L_S(C') < L_S(C)$$

  .

  Per la terza proprietà almeno due parole, z_{m-1} e z_m, di codice di lunghezza massima devono differire solamente per l'ultimo simbolo. Consideriamo il codice C'_R ricavato da C' che associa al simbolo (s_{m-1}, s_m) la parola di codice ottenuta rimuovendo da z_m l'ultimo simbolo.

  Si ottiene così un codice **prefisso** t.c. $L_S(C') = L_{R(S)}(C'_R) + p_{m-1} + p_m$.

  Quindi contro l'ipotesi di ottimalità di C_R, si avrebbe che $L_{R(S)}(C'_R) < L_{R(S)}(C_R)$.

Data una sorgente S, si produce una successione di sorgenti S_i (con $i = 0, \dots, m-2$) tale che:

$$S_0 = S \text{ e } S_{i+1} = R(S_i)$$

... fino ad ottenere una sorgente con due soli simboli.

L'albero è così costruito *bottom-up*, e ha i simboli della sorgente nelle foglie.

Le due foglie con probabilità più piccole diventano figli di uno stesso nodo avente come probabilità la somma di queste. Arrivati alla sorgente con due simboli si ottiene un codice compatto; proseguendo a ritroso si ricava un codice compatto per tutte le sorgenti, fino ad S.

Il codice di Huffman costruito in questo modo non è unico, ma qualunque codice di Huffman per una data sorgente è ottimale.

Esempio.

Costruzione del codice come albero.

Algoritmo di compressione di Huffman con codici binari

La coda di priorità può essere implementata tramite uno heap di alberi binari in cui la priorità è data dall'opposto della probabilità dei simboli.

Limiti della codifica di Huffman

La codifica richiede la conoscenza delle probabilità dei simboli della sorgente, e questo comporta un passaggio di preprocessing, e l'invio di questa informazione al codificatore. Quindi si ha un impatto sia sul tempo di codifica sia sulla dimensione dell'output.

Decodifica di Huffman

Il decodificatore deve basarsi sulle stesse informazioni sull'input usate per la codifica. Quindi l'intestazione del file compresso dovrà contenere o l'albero usato dal compressore, oppure le probabilità dei simboli della sorgente per poterlo ricostruire.

Ciò aumenta le dimensioni dell'output compresso e nel caso si debba ricostruire l'albero anche il tempo di decodifica.

Algorithm 2 Costruzione Albero di Huffman

Require: Testo in input T
Ensure: Radice dell'albero di Huffman

Calcola le occorrenze $\text{occ}(a)$ per ogni simbolo $a \in T$
Inizializza una Coda a Priorità (min-heap) Q

for ogni simbolo a con $\text{occ}(a) > 0$ **do**
    $n \leftarrow \text{NuovoNodo}(\text{simbolo} = a, \text{prob} = \text{occ}(a))$
    Q.Insert(n)
end for

while Q.size > 1 **do**
    $nodoSinistro \leftarrow Q$.ExtractMin()
    $nodoDestro \leftarrow Q$.ExtractMin()

    $probSomma \leftarrow nodoSinistro$.prob + $nodoDestro$.prob
    $nodoPadre \leftarrow \text{NuovoNodo}(\text{prob} = probSomma)$
    $nodoPadre$.left $\leftarrow nodoSinistro$
    $nodoPadre$.right $\leftarrow nodoDestro$

    Q.Insert($nodoPadre$)
end while

return Q.ExtractMin() ▷ L'ultimo nodo rimasto è la radice

Algorithm 3 *

Huffman(A: array; n: intero)

```
Q: Priority_Queue;

for i = 1 to n do
    Insert(Q,A[i])                    ▷ Con una coda a priorità basata su lista:
end for
                                      ▷ Primo ciclo:
                                      ▷ Esegue una operazione di
                                      ▷ accodamento per ognuna delle n
                                      ▷ iterazioni - n O(1)

for i = 1 to n - 1 do
    x = Extract-Min(Q)                ▷ Secondo ciclo:
    y = Extract-Min(Q)                ▷ Esegue due operazioni di estrazione
    "crea un nuovo albero con radice z e valore P_x   ▷ dalla coda e una di accodamento per
    + P_y e con figli destro x e sinistro y"          ▷ ognuna delle n-1 iterazioni - (n-1) O(n)
    Insert(Q,z)                       ▷ Totale: O(n^2)
end for

return Extract-Min(Q)                 ▷ Con una coda a priorità basata su heap:
                                      ▷ Totale: O(n log n).

                                      ▷ Nota: l'algoritmo è un greedy
```

Algorithm 4 Decodifica Huffman

```
Input: Albero di Huffman, file compresso
Output: File di output decodificato

while ci sono bit da leggere do
    Leggi un bit dal file compresso
    Usa il bit per attraversare l'albero di Huffman (0: sinistra, 1: destra)
    if nodo corrente è una foglia then
        Stampa il carattere ASCII corrispondente nel file di output
        Torna al nodo radice dell'albero di Huffman
    end if
end while
Il programma termina quando non ci sono altri bit da decodificare
            (o quando si raggiunge il numero di caratteri memorizzati nell'intestazione)
```

Osservazioni sulla codifica Huffman

L'algoritmo di Huffman rappresenta una soluzione ottima al problema della costruzione di codici prefissi a lunghezza variabile, minimizzando la lunghezza media del codice per una sorgente nota. Si può dimostrare che $L_S(C)$ rispetta il vincolo dettato dal teorema di Shannon, ovvero $L_S(C) \geq H(S)$. In questo contesto si avvicina al limite teorico minimo, pur non raggiungendolo necessariamente, a causa del vincolo di impiegare lunghezze *intere* per le codeword.

Da notare che la costruzione dell'albero di Huffman **non** è univoca: in presenza di più coppie di simboli con probabilità minime uguali, le scelte sono tutte legittime. Si avranno codici differenti, tutti però ottimali.

In applicazioni pratiche, soprattutto in sistemi di comunicazioni con buffer a dimensione fissa, risulta vantaggioso impiegare codici con **varianza minima** nella lunghezza delle codeword, anche a scapito di un lieve aumento di $L_S(C)$. Lunghezze più omogenee facilitano la gestione del buffering e del padding dei dati.

Per ottenere codici a varianza minima, in fase di costruzione dell'albero a parità di probabilità, è sempre preferibile aggregare i nodi che rappresentano i sotto-alberi con altezza minore.

Ottimalità di Huffman. La codifica *per simbolo*, quando ad ogni simbolo corrisponde una codeword, risulta **ottimale** garantendo una **ridondanza minima**.

Per la ridondanza di codice definita da

$$\rho(C) = 1 - \eta(C) = \frac{L_S(C) - H_d(S)}{L_S(C)}$$

si trova un limite superiore alla ridondanza:

$$\rho_{Huff}(C) \leq p_1 + 1 - \log_2(e) + \log_2(\log_2(e)) \approx p_1 + 0.086$$

Con p_1 che indica la probabilità del simbolo più probabile.

La codifica sarà invece **assolutamente ottima**, eguagliando l'entropia della sorgente ($L_S(C) = H(S)$ con ridondanza nulla), se e solo se le probabilità sono potenze negative di due:

$$L_S(C) = H(S) \iff \forall p_i \quad p_i = \frac{1}{2^{l_i}}$$

Si ottengono invece prestazioni *sub ottimali* nel caso di sorgenti caratterizzate da *molti* simboli ad *altissima probabilità*. La necessità di assegnare ai simboli codifiche con un **numero intero** di caratteri limita la possibilità di rappresentare fedelmente le distribuzioni di probabilità rendendo più vantaggiosi, in certi casi, altre codifiche come la codifica aritmetica.

Estensione della sorgente per Huffman. Da non confondere con il processo di aggregazione dei nodi dell'albero, l'estensione della sorgente interviene *a priori* sui dati. Per superare il limite prestazionale imposto dai singoli simboli ad altissima probabilità, si raggruppano i simboli originali in blocchi di lunghezza n (creando il super-alfabeto S^n) e solo successivamente si applica l'algoritmo di Huffman su di essi. In questo modo l'inefficienza dovuta al vincolo del bit intero viene frazionata su n simboli, permettendo alla lunghezza media per simbolo originale di avvicinarsi all'entropia $H(S)$.

3.2.5 Codifica Aritmetica

La codifica aritmetica è una tecnica di compressione entropica, alternativa alla codifica di Huffman, ma basata su una strategia profondamente diversa. L'idea chiave, introdotta da Peter Elias, e formalizzata da Abramson nel 1963, consiste nel rappresentare un intero messaggio come un singolo numero reale compreso nell'intervallo $[0, 1)$.

Contrariamente a Huffman, dove ogni simbolo viene sostituito con una parola di codice, nella codifica aritmetica si lavora sul flusso completo dei simboli: l'intero messaggio determina progressivamente un intervallo sempre più ristretto in $[0, 1)$. Il numero finale appartenente a tale intervallo è la rappresentazione codificata del messaggio.

In termini teorici, la lunghezza del codice prodotto tende a essere molto vicina all'entropia della sorgente, rendendo la codifica aritmetica quasi ottimale.

Codifica aritmetica Vs Huffman

In caso di distribuzioni di probabilità dei simboli asimmetriche, si avvicina maggiormente all'entropia rispetto alla codifica di Huffman.

Consideriamo una sorgente $S = \{a, b\}$ con $p(a) = \frac{99}{100}$ e $p(b) = \frac{1}{100}$; l'entropia sarà

$$H(S) = p(a) \log_2 \left(\frac{100}{99} \right) + p(b) \log_2 \left(\frac{100}{1} \right) \approx 0.08056$$

Qualunque codice prefisso, anche Huffman, impiega almeno un bit per simbolo, di conseguenza L_S deve essere almeno 1, che in questo caso è molto maggiore del valore dell'entropia!

La codifica aritmetica aggira questo problema usando la codifica binaria di un numero frazionale

Esempio di codifica in binario. Codifichiamo in binario 0.825

$$\begin{aligned} 0.825 \cdot 2 = 1.65 \quad & U = 1 \\ 0.65 \cdot 2 = 1.3 \quad & U = 1 \\ 0.3 \cdot 2 = 0.6 \quad & U = 0 \\ 0.6 \cdot 2 = 1.2 \quad & U = 1 \\ 0.2 \cdot 2 = 0.4 \quad & U = 0 \\ 0.4 \cdot 2 = 0.8 \quad & U = 0 \\ 0.8 \cdot 2 = 1.6 \quad & U = 1 \\ 0.6 \cdot 2 = 1.2 \quad & U = 1 \end{aligned}$$

Se durante la conversione binaria di un numero frazionario si verifica nuovamente un valore già incontrato nel processo iterativo, significa che la sequenza binaria sta entrando in un ciclo ripetitivo. In tal caso, si può interrompere l'algoritmo e rappresentare il numero come una frazione binaria periodica.

Ad esempio, la conversione di 0.825 in binario genera la sequenza:

$$\begin{aligned} 0.825 \times 2 = 1.65 \quad &\Rightarrow \quad 1 \\ 0.65 \times 2 = 1.3 \quad &\Rightarrow \quad 1 \\ 0.3 \times 2 = 0.6 \quad &\Rightarrow \quad 0 \\ 0.6 \times 2 = 1.2 \quad &\Rightarrow \quad 1 \\ \text{e così via...} & \end{aligned}$$

A un certo punto, il valore 0.6 si ripresenta, e quindi anche la sequenza di bit in uscita inizierà a ripetersi ciclicamente. Si ottiene dunque una rappresentazione periodica:

$$0.825 = 0.\overline{1101001}$$

Algoritmo di Codifica Aritmetica

Per messaggi di lunghezza fissata n, gli intervalli generati non si sovrappongono mai. Conseguentemente, qualsiasi numero appartenente all'intervallo finale identifica in modo univoco il messaggio originale.

Algorithm 5 Codifica aritmetica

Require: Stringa da codificare S su alfabeto A con probabilità P
Ensure: Lower bound dell'intervallo finale convertito in binario
1: Inizializza l'intervallo a $[0, 1)$
2: **for** ogni simbolo $s \in S$ **do**
3: Suddividi l'intervallo corrente in n sotto intervalli, dove $n = |A|$
4: Ogni sotto intervallo ha ampiezza proporzionale alla probabilità
5: Seleziona il sotto intervallo corrispondente al simbolo s
6: Aggiorna l'intervallo corrente a quello selezionato
7: **end for**
8: **return** L'output è il lower bound dell'intervallo corrente...

Esempio. Si codifica la stringa $BILLGATES$:
Per prima cosa servono le frequenze dei caratteri:

Simbolo	Frequenza
`(spazio)`	0.1
A	0.1
B	0.1
E	0.1
G	0.1
I	0.1
L	0.2
S	0.1
T	0.1

Tabella 3.2: Distribuzione di frequenze dei simboli

Nota: per spiegare l'algoritmo con l'esempio, i numeri che mostreremo saranno in base 10, ma ovviamente in realtà sono sempre in base 2.

Simbolo	Probabilità	Range
spazio	0.1	[0.00, 0.10)
A	0.1	[0.10, 0.20)
B	0.1	[0.20, 0.30)
E	0.1	[0.30, 0.40)
G	0.1	[0.40, 0.50)
I	0.1	[0.50, 0.60)
L	0.2	**[0.60, 0.80)**
S	0.1	[0.80, 0.90)
T	0.1	[0.90, 1.00)

Tabella 3.3: Distribuzione di probabilità e intervalli di codifica

Ricapitolando:

- Dato un carattere X con un *intervallo iniziale*

$$[\mathit{LowRange}(X), \mathit{HighRange}(X))$$

- I nuovi limiti diventano

$$\mathit{NewHigh}(X) \coloneqq \mathit{OldLow} + (\mathit{OldHigh} - \mathit{OldLow}) \cdot \mathit{HighRange}(X)$$

$$\mathit{NewLow}(X) \coloneqq \mathit{OldLow} + (\mathit{OldHigh} - \mathit{OldLow}) \cdot \mathit{LowRange}(X)$$

- Per codificare il simbolo s (usando i numeri da 1 a n) e il simbolo i con probabilità p_i:

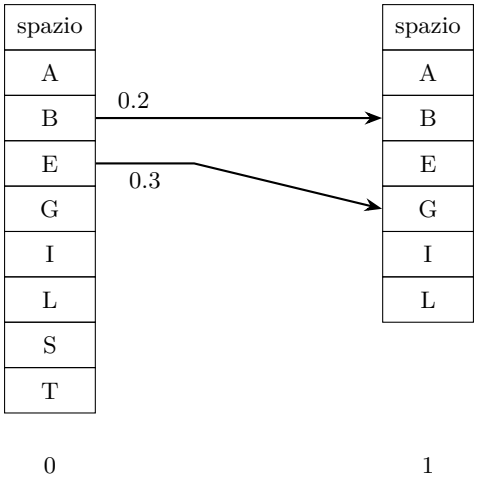


Figura 3.1: Passaggio tra intervalli durante la codifica del simbolo B

- $low_bound = \sum_{i=1}^{s-1} p_i$
- $high_bound = \sum_{i=1}^{s} p_i$
- $range = high - low$
- $low = low + range \cdot low_bound$
- $high = low + range \cdot high_bound$

Decodifica. La decodifica esegue lo stesso passaggio di restringimento dell'intervallo che avviene nella codifica:

$$\sum_{i=1}^{s-1} p_i \leq \frac{\text{value} - \text{low}}{\text{high} - \text{low}} \leq \sum_{i=1}^{s} p_i$$

Algoritmo di decodifica. INPUT: Stringa da decodificare e alfabeto con distribuzione di probabilità

1. Inizializza l'intervallo a $[0, 1)$.
2. Per ogni simbolo della stringa in input:
   (a) Suddividi l'intervallo corrente in sotto intervalli (n = cardinalità dell'alfabeto). Ogni sotto intervallo avrà un'ampiezza proporzionale alla probabilità associata a ciascun elemento nell'alfabeto iniziale.
   (b) Seleziona il simbolo corrispondente al sotto intervallo nel quale ricade il valore dell'input in analisi e seleziona tale intervallo come corrente.

Si tratta di un algoritmo *lossless* (senza perdita di informazione).

Gestione della terminazione. Poiché la decodifica è un processo iterativo basato sul restringimento degli intervalli, l'algoritmo necessita di una condizione di arresto. Esistono due soluzioni pratiche:

1. Trasmettere al decodificatore, come informazione accessoria (ad esempio nell'intestazione del file), la lunghezza esatta del messaggio originale.
2. Introdurre nell'alfabeto di sorgente un simbolo speciale di fine file (`eof`) con una probabilità molto piccola (es. $p = 0.000001$). Quando il decodificatore incontra l'intervallo associato a `eof`, il processo termina.

Perché funziona e perché è quasi ottimale. La codifica aritmetica rappresenta un intero messaggio come un numero reale compreso in un intervallo, che viene progressivamente ristretto a ogni simbolo elaborato. A ciascun simbolo è associato un sotto intervallo proporzionale alla sua probabilità: i simboli più probabili riducono meno l'intervallo, mentre quelli meno probabili lo restringono di più.

L'ampiezza dell'intervallo finale dopo l'elaborazione dell'intera sequenza è pari al prodotto delle probabilità dei simboli che compongono il messaggio. Di conseguenza, il numero di bit necessari per rappresentare un numero all'interno di quell'intervallo è proporzionale a:

$$-\log_2\left(\prod_{i=1}^{n} p_i\right) = -\sum_{i=1}^{n} \log_2 p_i,$$

cioè alla somma delle auto-informazioni dei simboli. Questo equivale all'informazione totale contenuta nel messaggio. In media, la lunghezza del codice per simbolo tende all'entropia della sorgente, rendendo la codifica aritmetica estremamente efficiente.

La codifica aritmetica è dunque *quasi ottimale*: sebbene nella pratica siano necessari aggiustamenti dovuti alla rappresentazione finita (overflow, underflow, troncamento a interi), la lunghezza del codice si avvicina molto al limite teorico stabilito dall'entropia. Inoltre, può codificare simboli molto probabili in meno di un bit, cosa impossibile con codici prefisso come Huffman.

3.3 Metodi basati su Dizionario

Questa classe di metodi di compressione non sfrutta la conoscenza, o la stima, delle probabilità dei simboli della sorgente. Questo consente di affrontare situazioni per le quali la conoscenza statistica non è possibile o realizzabile, rendendo gli approcci visti finora inefficaci o improponibili.

I metodi *statistici* operano sui singoli caratteri, e questo limita la compressione per il limite dato dall'entropia; i metodi a dizionario sostituiscono sequenze di simboli (*frasi* o *sottostringhe*) con riferimenti a un dizionario che ne memorizza i valori.

Questa tecnica, introdotta da Lempel e Ziv, costruisce in modo adattivo un dizionario, sopperendo alla conoscenza statistica a priori della sorgente. L'obiettivo è codificare pattern ricorrenti in modo efficiente. In questo modo risulteranno spesso più efficaci per dati in ingresso con molte ridondanze.

3.3.1 Dizionario

Formalizziamo un **dizionario** per un testo T come un insieme

$$D = \{(f, c) \mid f \in F, c \in C\}$$

dove

- F è un sottoinsieme di fattori (sottostringhe) del testo T
- C è l'insieme delle corrispondenti parole di codice

3.3.2 Dizionari statici

Il modo più semplice di costruire un dizionario è una costruzione *statica*, vale a dire si realizza il dizionario prima dell'effettiva compressione del dato. Una delle forme più comuni di dizionario statico è la **codifica a digrammi**. Il dizionario è costituito da tutte le lettere (simboli) della sorgente, e dall'aggiunta di **alcune** coppie di lettere, i digrammi.

Un esempio può essere un dizionario con i caratteri ASCII a 8 bit e 256 digrammi costituiti dalle coppie di caratteri più comuni.

Esempio. Supponiamo che l'alfabeto sia $A = \{a, b, c, d, r\}$ e il dizionario sia il seguente Il codificatore analizza **due simboli per volta** e cerca corrispondenze nel dizionario.

- Digramma identificato

  $\Rightarrow$ digramma codificato

- Digramma non identificato

Codice	Frase	Codice	Frase
000	a	100	r
001	b	101	ab
010	c	110	ac
011	d	111	ad

Tabella 3.4: Dizionario per A

$\Rightarrow$ codifica del primo carattere della coppia

Nel nostro esempio, *abracadabra* viene codificato con `101 100 110 111 101 100 000`.

Alternativamente ai digrammi, si potrebbero inserire nel dizionario frasi *più lunghe*, sfruttando la conoscenza di parole comuni, come articoli o preposizioni, o parole ricorrenti. Si può sviluppare ancora di più questa tecnica sfruttando la conoscenza del tipo di messaggio, ad esempio se si tratta di un testo di argomento sportivo, o economico, etc. Quello che si può guadagnare in efficienza si perde in mancanza di flessibilità.

In conclusione i dizionari statici presentano notevoli limitazioni.

3.3.3 Dizionari dinamici

I metodi a dizionario rappresentano una evoluzione concettuale rispetto agli approcci statistici, abbandonando la necessità di una conoscenza a priori delle caratteristiche (probabilistiche) dell'input. I compressori ideati da Lempel e Ziv costruiscono il dizionario in modo **adattivo**, analizzando i dati durante la compressione stessa. Il principio fondante è che i dati già processati costituiscono il miglior dizionario possibile per codificare la parte successiva, sfruttando la ridondanza locale e le ripetizioni presenti nel flusso in ingresso.

L'approccio **adattivo** elimina due limiti dei dizionari statici:

1. codebook generato implicitamente dall'input senza overhead di trasmissione del dizionario stesso.
2. non si deve scegliere tra un dizionario *troppo generico*, e quindi meno efficiente, e un dizionario *troppo specifico* che sarebbe ottimale per un determinato tipo di dati, ma deficitario per altri tipi di dato.

Il meccanismo chiave si basa sulla **codifica per sostituzione**, le sottostringhe ricorrenti vengono sostituite con riferimenti a occorrenze precedenti, unite a porzioni di *testo* non facenti parte di codeword. Questo dualismo permette al compressore di adattarsi a pattern non stazionari, superando i limiti dei codici fissi.

Gli algoritmi introdotti nel 1977 e nel 1978 da Lempel e Ziv sono ***lossless*** e usati per la compressione di dati eterogenei.

LZ77

L'idea chiave dei codificatori basati su dizionario, come LZ77, risiede nello sfruttare una caratteristica del dato da comprimere: la porzione precedentemente elaborata costituisce un ottimo dizionario *intrinseco*, poiché, prendendo a riferimento un testo in linguaggio naturale, tipicamente condivide il medesimo linguaggio e la medesima struttura statistica del testo ancora da comprimere.

In questo modo il dizionario viene creato *in divenire* dall'algoritmo e viene quindi meno la necessità di trasmettere il dizionario insieme ai dati compressi. Il decodificatore sarà in grado di ricostruirlo in modo adattivo e autonomo compiendo le stesse operazioni del codificatore.

Questo principio di ***località*** si rivela valido non solo per testi, ma *quasi* per ogni tipo di dato che vogliamo comprimere.

Codifica. Il processo di codifica risulta semplice da implementare; i passi di codifica producono delle *triple*:

1. Offset: puntatore che indica quanto *guardare indietro* nel testo precedentemente decodificato.
2. Lunghezza della frase
3. Carattere attualmente elaborato

Ad esempio con un alfabeto $A = \{a, b\}$, il testo *abaaabaabb* darebbe come risultato

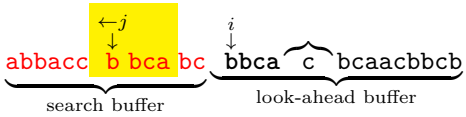
$$\langle 0,0,a\rangle \quad \langle 0,0,b\rangle \quad \langle 2,1,a\rangle \quad \langle 3,2,b\rangle \quad \langle 5,3,b\rangle$$

$$\underbrace{\langle 0,0,a\rangle}_{\text{a}} \quad \underbrace{\langle 0,0,b\rangle}_{\text{b}} \quad \underbrace{\langle 2,1,a\rangle}_{\text{aa}} \quad \underbrace{\langle 3,2,b\rangle}_{\text{bab}} \quad \underbrace{\langle 5,3,b\rangle}_{\text{aabb}}$$

Il codificatore utilizza una **finestra scorrevole** sui dati in ingresso, divisa in due parti

- **search buffer**: contiene la sezione già codificata
- **look-ahead buffer**: contiene i dati ancora da codificare

Ciò che separa i due buffer è il puntatore i, che indica l'inizio del look-ahead buffer. Un ulteriore puntatore j scorre il search-buffer a ritroso, partendo da i, cercando la più lunga sequenza possibile che sia un prefisso per il look-ahead buffer.

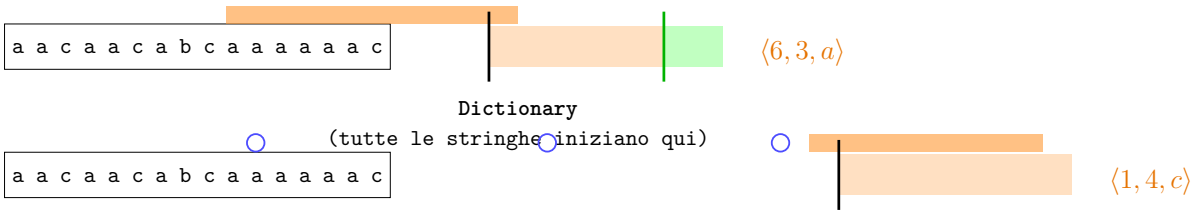


L'output compresso consisterà in una sequenza di triple $\langle o, l, s\rangle$:

- o, intero che rappresenta la distanza tra i e j (**offset**)
- l, intero che rappresenta la lunghezza del prefisso individuato (**frase**)
- s carattere, il primo simbolo presente nel look-ahead buffer immediatamente successivo al prefisso, cioè codeword già presente nel dizionario

Nella codifica ad ogni passo si possono incontrare tre casistiche:

1. Non si trova **alcun match** considerando il prossimo carattere da codificare
2. **Match**
3. **Match** che si estende *dentro* il look-ahead buffer



In prima sintesi i passi dell'algoritmo sono:

1. Cerca la copia più lunga che inizia prima
2. Output della tripla $\langle \textit{offset}, \textit{length}, \textit{symbol}\rangle$
3. Aggiorna il cursore di $\textit{length} + 1$

Copia simbolo per simbolo (Match estesi). Un dettaglio fondamentale di LZ77 è che la ricostruzione del prefisso non avviene *in blocco*, ma rigorosamente **simbolo per simbolo**. Questa particolarità implementativa permette di gestire il caso in cui la lunghezza del match sia superiore all'offset ($l > o$), ovvero quando la ripetizione si estende oltre il search buffer e *invade* il look-ahead buffer.

$$\overbrace{aab??}^{} \rightarrow \overbrace{a\ aba?}^{} \rightarrow \overbrace{aabaa}^{} \rightarrow \overbrace{aabaab}^{} \rightarrow \overbrace{aabaaba}^{} \rightarrow \overbrace{aabaa}^{}baa$$

In questo scenario, quindi, un singolo offset (inferiore alla lunghezza del prefisso stesso) è sufficiente per istruire il decodificatore a generare un'intera sequenza ciclica continua.

Parametri *strutturali* dell'algoritmo. Come è necessario dimensionare il search buffer? Può essere *infinito*? No, non si permette all'algoritmo di cercare all'indietro indefinitamente fino all'inizio del flusso di dati. Tipicamente il puntatore della finestra è di 12 bit. Valori maggiori non offrono vantaggi, per il principio di località dei dati.

Per quanto riguarda la dimensione delle parole riferite, tipicamente non si va oltre i 4 bit.

Usare indici e lunghezze maggiori permetterebbe di ottenere un numero minore di triple ma queste occuperebbero più spazio, e con una finestra di ricerca più grande inevitabilmente si impiegherebbe più tempo nella ricerca dei prefissi, è necessario un compromesso.

Capacità di compressione. Nel caso in cui si comprima un file di testo (con caratteri a 8 bit), ogni tripla generata richiederà uno spazio di memoria pari a:

$$n + m + 8 \text{ bit}$$

dove n sono i bit necessari per l'indice (offset) e m i bit per la lunghezza della frase. Il numero di bit totali del file compresso sarà quindi la somma dei bit di tutte le triple: $(n + m + 8) \times$ numero_triple. Il parametro di valutazione delle prestazioni è il **fattore di compressione**, definito come:

$$\text{Fattore di compressione} = \frac{\text{Numero bit iniziali}}{\text{Numero bit finali}}$$

Codifica.

```
Codifica LZ77
p = 1
WHILE (p < |S|) {
    Cerca il match massimo per S[p...] in S[p - n, ..., p-1]
// Supponi che il match sia in posizione p-i, con lunghezza l
    output della tripla <p-i, l, S[p+l]>
}
```

Decodifica.

```
Decodifica LZ77
p = 1
FOREACH triple <f, l, c> {
    S[p ... p + l - 1] = S[p - f ... p - f + l - 1]
    // Supponi che il match occorre alla posizione p - f con lunghezza l
    S[p + l] = c
    p = p + l + 1
}
```

Prestazioni, limiti e vantaggi. La decodifica è molto veloce, più della codifica che deve accollarsi la ricerca dei prefissi. Come già detto, accade che più il buffer è esteso, più le codewords possono essere lunghe e quindi essere in numero minore nell'output compresso. Parimenti le triple generate occuperanno più spazio e si spenderà più tempo cercando frasi in una finestra di dimensioni maggiori.

Varianti. L'algoritmo LZ77 è stato oggetto di numerosi studi che hanno portato a diverse varianti, pur mantenendo l'idea generale di Lempel e Ziv. La più celebre è la variante **LZss** (Storer-Szymanski), che mira a eliminare lo spreco di bit dovuto all'obbligo di emettere sempre una tripla fissa.

Alcune modifiche all'implementazione delle triple:

Indice: codifica variabile (Huffman) per rappresentarne il valore. Utile perché saranno molto più comuni offset *piccoli*, per via del principio di località, e quindi si utilizzano meno bit per memorizzarli.

Lunghezza: lo stesso vale per le lunghezze delle codewords, le lunghezze brevi sono più probabili e quindi si codificano con Huffman.

Simbolo (LZss): il carattere in chiaro viene aggiunto solo quando serve (quando non c'è match). Si utilizza un **flag** di 1 bit per segnalare se i dati successivi rappresentano una coppia $\langle \text{offset}, \text{lunghezza} \rangle$ (riferimento al dizionario) oppure un singolo carattere non compresso.

Esempio di codifica e decodifica.

LZ78

L'algoritmo LZ78 si differenzia dal predecessore per alcune scelte implementative, ma sfrutta le stesse idee di base. La prima differenza consiste nel limitare il numero di frasi, tra tutte quelle possibili, che possono andare a comporre il dizionario.

La prima differenza consiste nel limitare il numero di frasi, tra tutte quelle possibili, che possono andare a comporre il dizionario. Questo limite è teoricamente e praticamente necessario: sia per contenere l'occupazione di memoria, sia per mantenere fissa e ridotta la dimensione in bit dell'indice i trasmesso in output (se il dizionario crescesse all'infinito, gli indici richiederebbero troppi bit, vanificando la compressione). Nello specifico viene utilizzato un *trie* per memorizzare il dizionario e si vuole impedire che le dimensioni di quest'ultimo diventino eccessive.

Altra differenza sostanziale, legata a doppio filo con la precedente, è il non avere una finestra (o quantomeno un limite meno stringente) che limiti quanto a ritroso si possono cercare e riferire sottostringhe. In questo modo LZ78 non si limita a sfruttare il principio di località, ma trae beneficio anche dalla regolarità nei dati. Queste differenze influenzano le velocità di codifica e decodifica come vedremo.

Codifica. La scansione del *testo* produce un elenco di sottostringhe, che comporranno il dizionario, e solo queste potranno essere riferite.

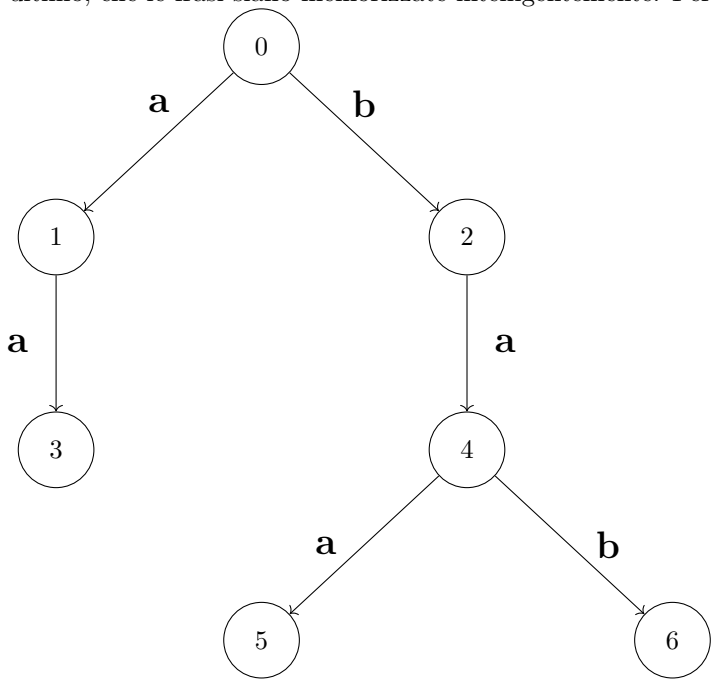
Con queste differenze l'output non sarà più una sequenza di *triple*, ma una sequenza di coppie $\langle i, c \rangle$, dove:

i indice della frase precedente

c prossimo carattere

La lunghezza, a differenza di LZ77, non è necessaria perché l'indice referenzia una frase *esatta*. Quando una combinazione $\langle$frase nel dizionario + carattere$\rangle$ non è ancora stata incontrata, la si emette in output e la si memorizza nel dizionario come nuova frase.

Implementazione. Per come l'algoritmo scrive ed accede al dizionario, è cruciale per l'efficienza di quest'ultimo, che le frasi siano memorizzate intelligentemente. Per questo si utilizza un **trie**:



Ogni codeword è specificata dal percorso dalla radice alla foglia che identifica il *numero* della frase da emettere.

Esempio di Codifica. Per comprendere il meccanismo, simuliamo passo passo la codifica della stringa `CCACCBCABCABA`. Il dizionario viene inizializzato logicamente con la stringa vuota all'indice 0.

Step	Sottostringa analizzata	Output $\langle I, C\rangle$	Nuova voce Dizionario
1	`C` (non presente)	$\langle 0, C\rangle$	1: `C`
2	`C` (pres.) → `CA` (non pres.)	$\langle 1, A\rangle$	2: `CA`
3	`C` (pres.) → `CC` (non pres.)	$\langle 1, C\rangle$	3: `CC`
4	`B` (non presente)	$\langle 0, B\rangle$	4: `B`
5	`CA` (pres.) → `CAB` (non pres.)	$\langle 2, B\rangle$	5: `CAB`
6	`CAB` (pres.) → `CABA` (non pres.)	$\langle 5, A\rangle$	6: `CABA`

Tabella 3.5: Processo di codifica LZ78 passo-passo

Decodifica. La decodifica riceve in input la sequenza di coppie e per ogni coppia analizzata ricompone il dato compresso e *costruisce* il dizionario/trie.

La decodifica riceve in input la sequenza di coppie $\langle i, c\rangle$. Per ogni coppia analizzata, si ricompone il dato originale e congiuntamente costruisce il dizionario.

Si emette in output la stringa presente nel dizionario all'indice i concatenata al carattere c, e contestualmente si salva questa nuova stringa intera come nuova voce nel dizionario.

Esempio di Decodifica. Riprendiamo in input la sequenza generata dall'esempio precedente: $\langle 0, C\rangle$, $\langle 1, A\rangle$, $\langle 1, C\rangle$, $\langle 0, B\rangle$, $\langle 2, B\rangle$, $\langle 5, A\rangle$.

Anche qui, il dizionario parte logicamente con la stringa vuota all'indice 0.

Step	Input $\langle I, C\rangle$	Stringa Emessa (Dict$[I]$ + C)	Nuova voce Dizionario
1	$\langle 0, C\rangle$	`""` + `C` = **`C`**	1: `C`
2	$\langle 1, A\rangle$	`C` + `A` = **`CA`**	2: `CA`
3	$\langle 1, C\rangle$	`C` + `C` = **`CC`**	3: `CC`
4	$\langle 0, B\rangle$	`""` + `B` = **`B`**	4: `B`
5	$\langle 2, B\rangle$	`CA` + `B` = **`CAB`**	5: `CAB`
6	$\langle 5, A\rangle$	`CAB` + `A` = **`CABA`**	6: `CABA`
Output finale decodificato: `CCACCBCABCABA`			

Tabella 3.6: Processo di decodifica LZ78 passo-passo

Prestazioni, limiti e vantaggi. Rispetto a LZ77, grazie all'uso del *trie* e l'assenza del search buffer (che comporterebbe una ricerca ad ogni passo), la fase di codifica risulta generalmente più veloce, mentre la decodifica può risultare comparativamente più lenta, per la necessità di ricostruire il trie, pur rimanendo rapida. Come per le varianti di LZ77 si usano codifiche come Huffman o la codifica aritmetica per memorizzare le coppie in output. È stato dimostrato che, sotto determinate ipotesi sulla sorgente, entrambe le versioni sono ottimali; vale a dire che ottengono un rapporto di compressione che si avvicina all'entropia della sorgente. Per arrivare a questa conclusione si assume che la stringa sia infinita e prodotta da una sorgente il cui comportamento non varia nel tempo. Per LZ78 la ridondanza è stimata nell'ordine di $\mathcal{O}\left(\frac{1}{\log_2 n}\right)$ con n lunghezza della stringa. Con t numero totale delle *frasi*, Hansel, Perrin e Simon hanno provato che la lunghezza del codice è limitata da:

$$LZ78(s) \leq |s| H_0(s) + t \log\left(\frac{|s|}{t}\right) + \theta(t)$$

Più recentemente Manzini e Kosaraju hanno altresì dimostrato che, con stringhe a bassa entropia, LZ78 non è efficiente perché non si adatta a run lunghi di simboli identici.

LZW

L'algoritmo proposto da Terry Welch nel 1984, come miglioramento di LZ78, è anche questo un metodo di compressione *lossless* basato su dizionari, di uso molto comune (ad esempio nel formato grafico GIF).

Invece di riempire il dizionario *da zero* come LZ78, il dizionario in questo caso è inizializzato con tutti i simboli dell'alfabeto sorgente. Più precisamente con i 256 possibili valori per simboli a 8 bit. A partire dall'indice 256 verranno via via aggiunti i prefissi individuati.

Nelle implementazioni reali (come nel formato GIF), viene imposto un limite rigido non sulla lunghezza della singola stringa, ma sul numero totale di voci del dizionario. Questo limite è determinato dal numero

di bit allocati per la trasmissione degli indici (ad esempio, usando codici a 12 bit, il dizionario non può superare le 4096 voci).

Algoritmo di codifica. Nel primo passo la variabile *Prefisso* è inizializzata a NULL e si passa a leggere il primo carattere nello stream di input, nella variabile C. Quando si incontra una sequenza ($Prefisso + C$) non ancora presente nel dizionario, la si aggiunge a quest'ultimo, mentre in output viene emesso il codice corrispondente alla sola parte già nota (*Prefisso*).

Algorithm 6 Algoritmo di codifica LZW

```
1: Inizializza il dizionario con i singoli caratteri dell'alfabeto
2: Prefisso ← NULL
3: while ci sono caratteri da leggere do
4:     Leggi il carattere C dall'input
5:     if Prefisso + C è presente nel dizionario then
6:         Prefisso ← Prefisso + C
7:     else
8:         output ← output + code(Prefisso)
9:         Inserisci Prefisso + C nel dizionario con il primo codice non usato
10:        Prefisso ← C
11:    end if
12: end while
13: output ← output + code(Prefisso)
```

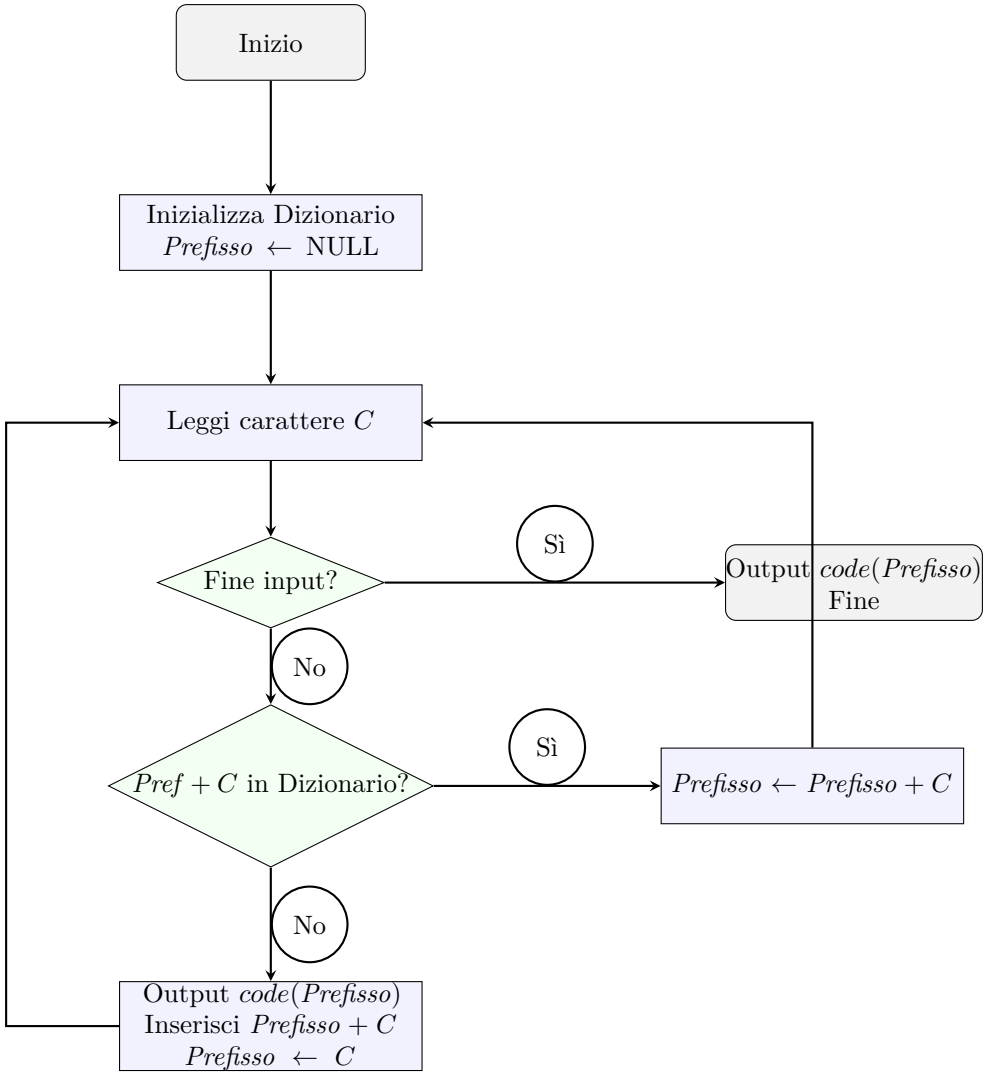


Figura 3.2: Diagramma di flusso logico della codifica LZW

Algoritmo di decodifica. Il decodificatore inizializza il dizionario con l'alfabeto di base, come il codificatore. Ricevendo i codici compressi, ricostruisce dinamicamente le frasi successive, aggiungendole al proprio dizionario passo dopo passo.

Algorithm 7 Algoritmo di decodifica LZW

```
Leggi il primo CODE dall'input
OldCode ← CODE
Character ← traduzione di CODE (dal dizionario base)
output ← Character
while ci sono codici da leggere do
    Leggi il prossimo CODE dall'input
    if CODE non è nel dizionario then
        String ← voce del dizionario per OldCode + Character
    else
        String ← voce del dizionario per CODE
    end if
    output ← String
    Character ← primo carattere di String
    Inserisci voce del dizionario per OldCode + Character nel dizionario
    OldCode ← CODE
end while
```

Caso particolare della decodifica. Esiste un caso speciale durante la decompressione: si verifica quando il decodificatore riceve in input un codice che **non è ancora presente** nel suo dizionario.

Questo accade per un'anomalia temporale causata da pattern fortemente ripetitivi del tipo $cWcWc$ (dove c è un singolo carattere e W una stringa, anche vuota). In queste specifiche sequenze, il codificatore inserisce una nuova voce nel dizionario e *immediatamente* al passo successivo emette in output proprio quel nuovo codice, prima che il decodificatore abbia i dati sufficienti per ricostruirlo.

Per risolvere l'impasse, il decodificatore applica una regola deduttiva fissa: **la voce mancante si ottiene prendendo la stringa decodificata all'iterazione precedente e concatenandole il suo stesso primo carattere.**

Esempio pratico (sorgente `aaa`):

- **Codifica:** L'algoritmo elabora le prime due `a`. Emette il codice di `a` (es. 97) e salva il nuovo prefisso `aa` nel dizionario all'indice 256. Subito dopo elabora la terza `a`, riconosce la stringa `aa` appena salvata, ed emette il nuovo codice: **256**. L'output trasmesso è quindi `<97, 256>`.
- **Decodifica:** Il decodificatore riceve `97` e stampa `a`. Poi riceve **256**, ma il suo dizionario al momento si ferma ai caratteri base (fino a 255)!
- **Risoluzione:** Prende la stringa appena decodificata (`a`), ne estrae il primo carattere (`a`) e li somma (`a` + `a` = `aa`). Ricostruisce e stampa correttamente la stringa mancante.

phrase	next	output	dictionary
t	h	⟨116⟩	256: th
h	i	⟨104⟩	257: hi
i	n	⟨105⟩	258: in
n	k	⟨110⟩	259: nk
k	i	⟨107⟩	260: ki
in	g	⟨258⟩	261: ing
g	_	⟨103⟩	262: g_
_	t	⟨32⟩	263: _t
th	i	⟨256⟩	264: thi
ing	s	⟨261⟩	265: ings
s	_	⟨115⟩	266: s_
_t	h	⟨263⟩	267: _th
h	r	⟨104⟩	268: hr
r	o	⟨114⟩	269: ro
o	u	⟨111⟩	270: ou
u	g	⟨117⟩	271: ug
g	h	⟨103⟩	272: gh
h		⟨104⟩	273: h

Esempio di codifica.

Compressione LZW

Output della compressione passo-passo della stringa *thinking things through*. Il carattere spazio è indicato convenzionalmente con il simbolo _.

Esempio di decodifica.

LZ e GZip (Argomento Extra)

Tra le innumerevoli varianti di LZ77, GZip (creato da J. Gailly e M. Adler nel 1992) è il software di compressione dati per uso generico più comune in ambiente Unix/Linux, ed è distribuito sotto licenza Gnu Free Software Foundation. Questo argomento è considerato un approfondimento "Extra" rispetto all'algoritmo base.

L'algoritmo presenta diverse differenze e ottimizzazioni chiave:

Ricerca veloce tramite Tabelle Hash. Per velocizzare la ricerca delle occorrenze nel search buffer, GZip utilizza una tabella hash che memorizza le posizioni di tutte le sequenze di 3 caratteri. L'algoritmo calcola l'hash dei successivi 3 caratteri da codificare:

- **Se il lookup ha successo:** restituisce una lista di indici delle occorrenze passate. L'algoritmo scorre la lista per trovare il match più lungo e genera l'output.
- **Altrimenti:** i caratteri vengono codificati in modo "raw" (con codice ASCII) e la tabella hash viene aggiornata aggiungendo le nuove sequenze.

Parametri Strutturali e Liste. La finestra di ricerca ha una dimensione massima di 32 KB, mentre la lunghezza massima della sequenza puntata all'indietro è di 256 caratteri. Per evitare di consumare troppo tempo scorrendo le liste ritornate dalla tabella hash, la loro lunghezza è limitata: le liste sono ordinate per inserimento e i match più vecchi vengono eliminati. L'utente può controllare il compromesso tra velocità e compressione scegliendo un livello da 1 a 9.

Output a Coppie. GZip non emette le classiche triple, ma utilizza delle coppie nel formato:

$$\langle \textit{lunghezza}, \textit{distanza}/\textit{carattere} \rangle$$

Se il primo valore è 0, non si tratta di un indice di lunghezza, ma la coppia rappresenta direttamente il carattere in chiaro nel formato $\langle 0, \textit{carattere} \rangle$.

Ottimizzazione Greedy vs Globale. Di default, la ricerca del match è *greedy* (avida), ovvero l'algoritmo si accontenta del match più lungo trovato al passo corrente. Tuttavia, selezionando il livello massimo di compressione, GZip esegue un'ottimizzazione globale: valuta se conviene rinunciare a un match lungo iniziale in favore di una scomposizione che codifichi più simboli complessivi minimizzando le coppie finali nei passi successivi.

Confronto tra Metodi a Dizionario e Statistici

La sostanziale differenza tra i compressori statistici (come Huffman o la codifica aritmetica) e quelli a dizionario consiste nel fatto che i primi lavorano sui singoli simboli, mentre i secondi operano per gruppi di simboli (*frasi*). Nella codifica di Huffman, ad esempio, ogni simbolo ha una lunghezza variabile, mentre le codifiche LZ producono sequenze o *tuple* per rappresentare le frasi. Inoltre, a differenza di Huffman, i metodi LZ non richiedono la conoscenza e trasmissione di alcuna tabella di probabilità, poiché il dizionario viene ricostruito dinamicamente dall'algoritmo.

I compressori statistici sono valutabili nei termini di **entropia di ordine** 0, ovvero:

$$|out_{stat}(X)| \approx |X| H_0(X)$$

Essi non sono sensibili alla ripetitività delle sequenze, ma solo alla distribuzione di probabilità dei singoli simboli. Considerando un testo altamente ripetitivo, ad esempio una stringa X ripetuta k volte (X^k), si verifica che:

$$|out_{stat}(X^k)| \approx |X^k| H_0(X^k) = k|X| H_0(X) \approx k|out_{stat}(X)|$$

Ciò significa che la dimensione dell'output cresce in modo lineare all'aumentare delle ripetizioni.

Al contrario, i metodi a dizionario catturano perfettamente questa ridondanza. Con LZ77, ad esempio, una sottostringa nella forma X^{k-1} ha un'occorrenza precedente (come prefisso). Essa verrà quindi codificata con un'unica tripla $\langle |X|, (k-1)|X|, \textit{symbol} \rangle$. Di conseguenza:

$$|out_{lz}(X^k)| \leq |out_{lz}(X)| + c \quad \text{con } c \text{ costante.}$$

L'aggiunta di infinite ripetizioni comporta un incremento di spazio finale solamente costante (c), rendendo i metodi LZ enormemente superiori per dati con alta ripetitività o ridondanza locale.

Esercizio: Compressione della stringa $(ab)^n$. Consideriamo la stringa $S = (ab)^n$. Valutiamo come viene affrontata dalle tecniche analizzate.

Possiamo subito notare che la stringa presenta una ripetitività *perfetta* e una entropia molto bassa, abbiamo solo due simboli equiprobabili.

1. Qual è l'entropia di ordine 0?
   Poiché la stringa è formata da un'alternanza *perfetta e bilanciata* di due caratteri, le probabilità di comparsa per i singoli simboli sono $P(a) = \frac{1}{2}$ e $P(b) = \frac{1}{2}$. Applicando la formula dell'entropia di ordine 0:
   $$H_0(S) = -\left(\frac{1}{2}\log_2\left(\frac{1}{2}\right) + \frac{1}{2}\log_2\left(\frac{1}{2}\right)\right) = 1 \text{ bit/simbolo}$$

2. **Come viene compressa mediante Huffman?**
   Poiché i due simboli a e b sono equiprobabili, l'albero di Huffman assegnerà a ciascuno un codice di lunghezza fissa pari esattamente a un bit. La lunghezza totale dell'output compresso sarà quindi sempre di $2n$ bit. L'effettivo "guadagno" dipende dalla codifica della stringa originale:

   - Se consideriamo la stringa sorgente codificata in un formato reale come ASCII (8 bit per simbolo), la dimensione iniziale è di $16n$ bit. Huffman ottiene quindi un eccellente **fattore di compressione** pari a 8 ($\frac{16n}{2n}$), portando la dimensione al limite teorico dell'entropia di ordine 0.
   - Se invece confrontiamo Huffman con un alfabeto minimo già a 1 bit, non vi è alcuna compressione aggiuntiva.

   In entrambi i casi, il limite di Huffman è che, lavorando sui singoli simboli, non è in grado di "accorgersi" della struttura ciclica $(ab)^n$ per comprimere ulteriormente.

3. **Come viene codificata con i compressori LZ-based?**
   Casi come questo dimostrano la superiorità dei metodi a dizionario rispetto al limite dell'entropia di ordine 0, poiché sfruttano appieno la *regolarità* e la ripetitività. I due algoritmi si comportano in modo diverso ma altamente efficiente:

   - **LZ77:** Dopo aver acquisito la prima occorrenza di ab, il puntatore nel search buffer individuerà un prefisso utile scorrendo all'indietro di un offset $o = 2$. Grazie alla particolarità del **match esteso** ($l > o$), il codificatore genera una singola tripla che agirà come istruzione: indicherà al decodificatore di replicare materialmente il ciclo ab per l'intera capienza del look-ahead buffer. A causa dei limiti implementativi (la dimensione della finestra e i bit massimi per memorizzare la lunghezza l), una stringa $(ab)^n$ molto lunga non verrà compressa in una singola tripla, ma in una breve sequenza di triple che sfruttano la lunghezza massima consentita (l_{max}). L'intera stringa collassa quindi in una frazione minima di spazio, sfruttando appieno la ripetitività.
   - **LZ78:** Non avendo il match esteso continuo, l'algoritmo popolerà il *trie* costruendo referenze a concatenazioni via via più lunghe (es. `a`, `b`, `ab`, `ba`, `aba`...). Sebbene generi più coppie di output rispetto a LZ77, il numero totale di frasi emesse cresce *più lentamente* rispetto alla dimensione $2n$ dell'input. Poiché il *trie* continua a crescere aggiungendo un nuovo nodo ad ogni passo di codifica, il limite al numero totale delle frasi del dizionario è dettato, all'atto pratico, dalla dimensione massima prefissata. Raggiunto tale limite, l'algoritmo tipicamente smette di aggiungere nuove frasi ed emette ciclicamente le stesse coppie di valori in output.

3.4 Sorgenti con Memoria e Trasformazioni dei Dati

Nelle sorgenti discrete **con memoria**, la probabilità di emissione di un simbolo in un dato istante non è indipendente, ma è condizionata dalla sequenza degli n simboli emessi in precedenza. Questa dipendenza *statistica* costituisce una forma di ridondanza intrinseca che i compressori possono sfruttare per massimizzare l'efficienza della codifica. Mentre la famiglia dei compressori a dizionario sfrutta questa memoria in modo indiretto tramite la ricerca di pattern ricorrenti, in questo capitolo analizzeremo tecniche e trasformazioni che manipolano attivamente i dati per isolare e abbattere esplicitamente l'entropia condizionata.

3.4.1 Sorgenti con Memoria

Si definisce sorgente discreta con memoria una sorgente caratterizzata dalle seguenti proprietà:

- $\exists m \in \mathbb{N}$ tale che la probabilità del simbolo in uscita dipende **al più** dalla sequenza degli m **simboli emessi in precedenza**.
- m è il minimo numero intero per cui tale proprietà è verificata ed è detto **memoria di** S.
- Emette simboli da un insieme di M simboli $X = \{x_1, x_2, \ldots, x_M\}$, ciascuno caratterizzato dalla propria probabilità p_i e auto-informazione I_i.
- Le probabilità p_i non dipendono dal tempo (sorgente **stazionaria**).
- I simboli emessi dalla sorgente in istanti differenti sono correlati. La probabilità che venga emesso il simbolo x_i dipende dalla sequenza di simboli x emessa precedentemente:

$$P(x_i \mid x) \neq P(x_i)$$

Ordine della memoria. Definiamo **ordine della memoria** della sorgente il numero di simboli precedentemente emessi che influenzano la probabilità di emissione del simbolo corrente. Una sorgente con memoria di **primo ordine** ha la probabilità del simbolo emesso dipendente solo dal simbolo precedente (x_j):

$$P(x_i \mid x) = P(x_i \mid x_j)$$

Calcolo dell'Entropia

Data una sorgente X con memoria di primo ordine, si può definire la sua entropia **condizionata** al simbolo x_j come:

$$H(X \mid x_j) = \sum_{i=1}^{M} P(x_i \mid x_j) \log_2 \left(\frac{1}{P(x_i \mid x_j)} \right)$$

Facendo la media su tutti i possibili simboli x_j si ottiene:

$$H(X) = \sum_{j=1}^{M} P(x_j) H(X \mid x_j)$$

Questa formula è valida solamente per il caso di memoria di primo ordine.

3.4.2 Codifica Predittiva

Consapevoli di avere una sorgente con memoria, si può trarre vantaggio di questo fatto per implementare una codifica.

Un metodo diretto per sfruttare la memoria di una sorgente consiste nell'utilizzare un meccanismo di **codifica predittiva**. L'obiettivo è realizzare un sistema in grado di *anticipare* l'emissione dei simboli basandosi sulla storia passata.

Il sistema si basa sul confronto tra il bit reale $x(i)$ e il bit predetto $\tilde{x}(i)$. Il loro sommatore modulo-2 genera una sequenza errore $\epsilon(i)$ che viene fornita in ingresso al codificatore a valle. Se si utilizza un ***buon predittore***, le predizioni esatte saranno molto frequenti, generando nella sequenza $\epsilon(i)$ una netta prevalenza di bit `0` rispetto ai bit `1`.

Di conseguenza, essendo la probabilità dei simboli fortemente sbilanciata, l'entropia di questa *sequenza errore* risulta estremamente bassa. Il codificatore potrà quindi rappresentarla producendo parole di codice molto corte, abbassando il *bit rate*.

3.4.3 Entropia Empirica

Spesso l'entropia di ordine zero non è abbastanza accurata per misurare le performance di un compressore. Per questo motivo, si definiscono entropie empiriche di ordine superiore che permettono di sfruttare la conoscenza del contesto, valutando la dipendenza delle frequenze dai simboli adiacenti.

Entropia empirica di ordine zero

L'entropia empirica di ordine zero è basata sulla distribuzione di probabilità implicitamente definita dall'input. Dato un input s di lunghezza n basato su un alfabeto di h simboli, indichiamo con n_i il numero di occorrenze del simbolo i-esimo. Assumendo convenzionalmente che $0 \log_2 0 = 0$, si definisce:

$$H_0(s) = -\sum_{i=1}^{h} \frac{n_i}{n} \log_2 \left(\frac{n_i}{n}\right)$$

Il termine $\frac{n_i}{n}$ rappresenta la probabilità empirica del simbolo. Il valore $|s|H_0(s)$ rappresenta la taglia dell'output di un compressore ideale che impiega $-\log_2 \left(\frac{n_i}{n}\right)$ bit per codificare l'i-esimo simbolo. Compressori basati su un modello di ordine zero (come la codifica di Huffman o la codifica Aritmetica) producono un output la cui dimensione si avvicina molto a questo limite teorico.

Entropia empirica di ordine k

Se si considera un contesto di lunghezza k, l'entropia empirica viene calcolata sulle sottostringhe che seguono (o precedono) specifici pattern. Si definisce come:

$$H_k(s) = \frac{1}{|s|} \sum_{w \in \Sigma^k} |w_s| H_0(w_s)$$

dove w_s è una stringa costituita dalla concatenazione di tutti i simboli che seguono l'occorrenza della parola w all'interno di s. La grandezza $|s|H_k(s)$ rappresenta un *lower bound* per le prestazioni di un compressore con memoria k. Poiché all'aumentare del contesto si hanno statistiche sempre più precise, $H_k(s)$ è una funzione non crescente rispetto a k, ovvero $H_{k+1}(s) \leq H_k(s)$.

3.4.4 Trasformata di Burrows-Wheeler

La trasformata di Burrows-Wheeler è un algoritmo di compressione lossless che, nelle parole dei due autori:

> "...opera applicando una trasformazione reversibile a un blocco di testo in input. La trasformazione in se non comprime il testo, ma lo riordina in modo da renderlo più facile da comprimere con semplici algoritmi come la codifica move-to-front. L'algoritmo raggiunge velocità comparabili alle tecniche di Lempel e Ziv, ma ottiene una compressione comparabile ai migliori modelli statistici. La dimensione del blocco in input deve essere grande (qualche kilobyte) per ottenere una buona compressione."

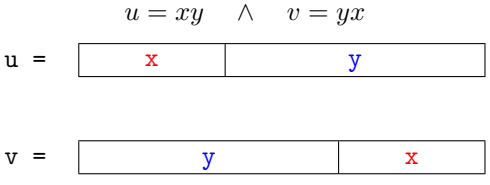
L'aspetto da sottolineare è che la trasformata rappresenta solo il primo passo di un processo di compressione. Più nel dettaglio, la BWT applica una permutazione reversibile alla stringa originale costruendo, ordinando lessicograficamente e manipolando tutte le sue **rotazioni cicliche**.

Rotazioni cicliche

Una rotazione ciclica (o shift circolare) è una specifica permutazione degli elementi di una sequenza in cui i caratteri vengono spostati mantenendo inalterato il loro ordine relativo. A differenza delle permutazioni generiche, che per una stringa di n caratteri generano $n!$ disposizioni, le rotazioni cicliche possibili per una parola di lunghezza n sono esattamente n.

In termini più formali, si consideri una parola v. Una parola u viene definita sua **coniugata** (ovvero sua **rotazione ciclica**) se esiste una scomposizione tale che:

$$u = xy \quad \wedge \quad v = yx$$



Codifica BWT

Dato un testo s di lunghezza n, l'algoritmo di codifica si articola nei seguenti passaggi:

1. **Generazione delle rotazioni:** Si costruisce una matrice M di dimensione $n \times n$, le cui righe corrispondono a tutte le n possibili rotazioni cicliche della stringa s.
2. **Ordinamento lessicografico:** Le righe della matrice M vengono ordinate alfabeticamente, generando una nuova matrice ordinata M'.
3. **Estrazione del testo:** Si estrae l'ultima colonna della matrice M' (denotata convenzionalmente con L, da *Last*). Questa colonna costituisce il testo permutato $bw(s)$.
4. **Salvataggio dell'indice:** Si memorizza la posizione della riga (l'indice l) all'interno di M' che contiene esattamente la stringa di partenza s. Questo valore è strettamente necessario per la successiva fase di decodifica.

Al termine dell'esecuzione, la codifica restituisce due elementi:

$bw(s)$**:** il testo permutato (corrispondente alla colonna L).

l**:** un valore intero che rappresenta l'indice di riga.

Decodifica BWT

L'algoritmo di decodifica ha il compito di ricostruire il testo originale s a partire esclusivamente dall'output della codifica, ovvero il testo permutato $L = bw(s)$ e l'indice l. La reversibilità della trasformata si basa sulla stretta relazione strutturale che lega l'ultima colonna (L) alla prima colonna (F) della matrice concettuale ordinata M'.

Ricostruzione della prima colonna (F). Poiché le righe della matrice M' sono le rotazioni cicliche della stringa originale, ogni colonna contiene esattamente lo stesso insieme di caratteri (una permutazione di s). Inoltre, poiché la matrice M' è costruita ordinando lessicograficamente le righe, la sua prima colonna F (*First*) sarà intrinsecamente ordinata in senso alfabetico. Di conseguenza, il decodificatore è in grado di ricostruire integralmente la colonna F senza dati aggiuntivi, semplicemente effettuando un ordinamento alfabetico crescente dei caratteri ricevuti nella stringa L.

Vettore di permutazione τ (o T). Per ripristinare la sequenza originale dei caratteri, la decodifica si affida a una proprietà fondamentale delle matrici di rotazioni ordinate: la conservazione dell'ordine relativo (nota anche come *LF-mapping*). Questa proprietà garantisce che la k-esima occorrenza di uno specifico carattere in L corrisponda esattamente alla k-esima occorrenza di quello stesso carattere in F.

Grazie a questa biunivocità, il decodificatore costruisce un vettore di permutazione T (o τ) di dimensione n. Il vettore è definito in modo tale che $T[i] = j$, dove j è la posizione nella colonna F del carattere che si trova in posizione i nella colonna L. Sfruttando il vettore T e iniziando dall'indice di riga l, la stringa originale s viene ricostruita iterativamente navigando la sequenza:

$$s[n-1-i] = L[T^{(i)}[l]] \quad \text{per } i = 0, \ldots, n-1$$

dove $T^{(i)}[l]$ indica l'applicazione iterata del vettore di permutazione per i volte a partire dalla posizione l.

Proprietà della BWT

L'efficacia della Trasformata di Burrows-Wheeler risiede nella sua capacità di raggruppare caratteri identici o simili, creando *run* (sequenze) dello stesso simbolo nell'output.

Omogeneità locale. Questa caratteristica è nota come **omogeneità locale**. Poiché la matrice delle rotazioni M' è ordinata lessicograficamente, righe adiacenti tenderanno a condividere lo stesso prefisso (che corrisponde al contesto destro nel testo originale). Di conseguenza, i caratteri che precedono contesti simili, e che finiscono nell'ultima colonna L, hanno un'alta probabilità di essere identici. Questo abbassa drasticamente l'entropia locale del testo permutato.

Parole bilanciate. La BWT non migliora la compressione in egual misura per qualsiasi testo. L'idea di fondo è che la trasformata produca una stringa con elevata similarità locale (e quindi altamente comprimibile a valle) se la stringa di input risulta molto *bilanciata*.

Una parola v è definita **bilanciata** se, per ogni singola lettera a dell'alfabeto e per ogni coppia di fattori u e u' di v aventi la medesima lunghezza ($|u| = |u'|$), vale la relazione:

$$||u|_a - |u'|_a| \leq 1$$

In aggiunta, una parola è detta **circolarmente bilanciata** se tutte le sue possibili rotazioni cicliche risultano essere bilanciate.

Utilizzi della BWT

Essendo una permutazione e non un compressore di per sé, la BWT viene utilizzata come primo stadio in pipeline di compressione lossless avanzate. L'implementazione pratica più celebre è il compressore **bzip2**, la cui architettura standard prevede la seguente catena di elaborazione:

1. **BWT**: Raggruppa i caratteri sfruttando il contesto (omogeneità locale).
2. **Move-to-Front (MTF)**: Converte l'omogeneità locale in una sequenza di interi, con una forte prevalenza di valori prossimi allo zero.
3. **Run-Length Encoding (RLE)**: Comprime le lunghe sequenze di zeri generate dal MTF.
4. **Codificatore Statistico**: Applica una codifica entropica finale (come Huffman o codifica aritmetica) per emettere i bit, sfruttando la bassissima entropia della sequenza risultante.

3.4.5 Algoritmi di Trasformazione (Post-BWT)

La Trasformata di Burrows-Wheeler non riduce la dimensione dei dati, ma si limita a riordinarli massimizzando l'omogeneità locale. Tuttavia, i classici codificatori entropici di ordine zero (come Huffman o la codifica aritmetica) sono insensibili all'ordine dei simboli e si basano esclusivamente sulle frequenze globali dell'intero blocco.

Per colmare questa lacuna e sfruttare il lavoro della BWT, l'output permutato viene elaborato da algoritmi di trasformazione intermedi. Tecniche come il Move-to-Front (MTF), il Run-Length Encoding (RLE) e il Distance Coding hanno lo scopo di convertire la località spaziale dei caratteri in una distribuzione statistica globalmente sbilanciata. Si produce un flusso di interi ad entropia estremamente bassa, rendendolo ideale per la fase finale di compressione entropica.

Move-to-Front (MTF)

Pubblicata da Ryabko nel 1980 con il nome di **book stack** e successivamente *riscoperta* da Bentley nel 1986, si tratta di una trasformata reversibile sui dati. Partendo da un flusso di caratteri/byte in ingresso, restituisce un array di interi. Quale è l'idea alla base di MTF? Ogni simbolo del flusso in ingresso è sostituito con l'indice corrispondente nella *lista dei simboli recenti*. Ricevendo dati con molti simboli ripetuti, questi simboli avranno come indice 0, e grazie al principio di località, quando saranno diversi, sarà molto probabile che siano poco *distanti* nell'elenco dei *simboli recenti*. Con queste premesse, si otterrà un array di interi in cui i valori più piccoli saranno molto frequenti, mentre numeri grandi si avranno solo quando un simbolo non compariva da molto tempo. Questa caratteristica consentirà di applicare con successo una codifica a lunghezza variabile sui numeri ottenuti come la codifica Huffman, o la codifica aritmetica. I benefici offerti e la rapidità di esecuzione mitigano l'aumento di complessità derivante dall'avere un'elaborazione ulteriore sui dati da comprimere.

Codifica MTF. I processi di codifica e decodifica non sfruttano conoscenze statistiche sull'input, devono solo coordinarsi sull'ordine iniziale dei simboli. Dati

X lista ordinata dei simboli in ordine lessicografico

L flusso di dati in ingresso

R vettore di interi in uscita

Algorithm 8 Codifica Move-To-Front

for all s in input **do**
  individua l'indice p del simbolo s in X
  aggiungi p a R
  sposta s in testa a X
end for

Esempio di codifica. *Input*: caraaaaaaabbbr
Alfabeto $X = \{$"a", "b", "c", "r"$\}$

caraaaaaabbbr	abcr	2
caraaaaaabbbr	cabr	1
caraaaaaabbbr	acbr	3
caraaaaaabbbr	racb	1
caraaaaaabbbr	arcb	0
caraaaaaabbbr	arcb	0000
caraaaaaabbbr	arcb	3
caraaaaaabbbr	barc	00
caraaaaaabbbr	rbac	2

$R = [2\ 1\ 3\ 1\ 0\ 0\ 0\ 0\ 3\ 0\ 0\ 2]$

Algorithm 9 Decodifica Move-To-Front

for $i = 0$ to $n - 1$ **do**
  leggi il valore $R[i]$
  assegna a $L[i]$ il carattere in posizione $R[i]$ di X
  sposta il carattere in testa a X
end for
return L

Decodifica MTF.

Proprietà di MTF. Move-to-Front aggiorna **dinamicamente** la struttura dati su cui si appoggia per codificare e decodificare, questo perché la codifica di ogni simbolo successivo dipende *anche* dallo stato corrente, dettato dalla sequenza dei simboli analizzati fino a quel momento. È un algoritmo **dinamico e adattivo**: lo stato della lista X evolve nel tempo in base ai simboli processati, consentendo di sfruttare il principio di *località temporale*) della sequenza in ingresso.

È ideale da combinare a valle della trasformata di Burrows-Wheeler, sfruttando l'omogeneità locale da essa generata.

Run Length Encoding

Questa tecnica di compressione **lossless** è ottimizzata per sorgenti caratterizzate da lunghe sequenze di simboli identici, definiti **run**. Il processo di codifica sostituisce ogni run con una coppia (simbolo, lunghezza) che ne identifica univocamente la struttura.

La lunghezza dei run non è arbitraria: se il parametro k definisce il numero di bit dedicati alla codifica della lunghezza, ogni run può avere una lunghezza massima pari a $2^k - 1$. Sequenze di simboli più lunghe devono essere necessariamente suddivise in più blocchi di codifica.

Efficienza della codifica RLE. Le parole di codice *in media* necessiteranno di meno bit per la loro codifica rispetto ai run che rappresentano.

Indichiamo con

$\overline{E}$: la media dei bit presenti nei run

$\overline{N}$: il numero medio dei bit nelle codeword che codificano i run

La *bit rate* r_b ottenuta sarà:

$$r_b = \left(\frac{\overline{N}}{\overline{\overline{E}}} \right)$$

Affinché il processo risulti effettivamente efficiente in termini di compressione, il rapporto tra la lunghezza della codeword e il numero di simboli rappresentati deve soddisfare la condizione $r_b < 1$.

Distance Coding

Il *Distance Coding* è una tecnica di trasformazione complementare, talvolta impiegata per rappresentare i simboli emessi dalla BWT o in alternativa al Move-to-Front. L'idea consiste nel codificare ogni occorrenza di un simbolo non tramite il suo valore assoluto, ma indicando la **distanza** (il numero di simboli distinti) che intercorre rispetto alla sua precedente apparizione nel flusso.

In presenza di alta omogeneità locale, i simboli tendono a ripresentarsi con frequenze ravvicinate, rendendo i valori di *distanza* molto piccoli e pertanto altamente comprimibili tramite codificatori entropici a lunghezza variabile.

3.4.6 Combinazione di Tecniche

La trasformata BWT non opera come un compressore autonomo, ma costituisce il primo stadio di una pipeline di compressione più complessa. L'architettura standard, resa celebre dal compressore **bzip2**, implementa la seguente catena di elaborazione sequenziale:

1. **BWT**: Trasforma l'input in una sequenza con alta omogeneità locale.
2. **Move-to-Front (MTF)**: Converte i simboli in un flusso di interi dominato da valori vicini allo zero, sfruttando la *località temporale*.
3. **Run-Length Encoding (RLE)**: Compatta ulteriormente le lunghe sequenze di zeri generate dal passaggio precedente.
4. **Codificatore Statistico**: Applica una codifica entropica finale (come Huffman o codifica aritmetica) per emettere il flusso di bit compresso.

Questa combinazione di tecniche costituisce l'architettura distintiva del compressore **bzip2**, che sfrutta la sinergia tra le trasformazioni reversibili (BWT, MTF, RLE) per preparare i dati a una codifica statistica finale altamente efficiente.

Teoremi sui limiti prestazionali

Parte III

Codifica di Canale

Capitolo 4

Teoria della Codifica di Canale

Quanto analizzato per la codifica di sorgente ha senso esclusivamente nell'ipotesi in cui i dati non subiscano alterazioni durante la trasmissione. Tuttavia, i canali *reali* sono **rumorosi** e possono introdurre errori in grado di degradare o distruggere il messaggio. La rimozione della ridondanza ottenuta tramite la codifica di sorgente, infatti, priva i dati di qualsiasi intrinseca robustezza: un errore nei metadati critici (come un albero di Huffman) o una *desincronizzazione* nel flusso di bit può rendere l'intera sequenza incomprensibile a destinazione.

Per affrontare il problema di trasmettere in maniera affidabile su un canale non-affidabile, si ricorre alla **codifica di canale**, il cui limite teorico è governato dal **Secondo Teorema di Shannon**.

4.1 Introduzione alla Codifica di Canale

Mentre l'obiettivo della codifica di sorgente è ridurre la ridondanza per incrementare l'efficienza del sistema, l'approccio adottato nella codifica di canale è l'esatto opposto: introdurre ridondanza in modo strutturato per incrementare l'affidabilità.

L'algoritmo di codifica *trasforma* la sequenza di dati in ingresso in una nuova sequenza di lunghezza maggiore, intrinsecamente più robusta agli effetti del rumore. Sfruttando questa ridondanza, in uscita dal canale, la decodifica di canale effettua l'operazione inversa: rileva l'eventuale presenza di errori e tenta di correggerli per ricostruire la sequenza di informazione originale prodotta dalla sorgente.

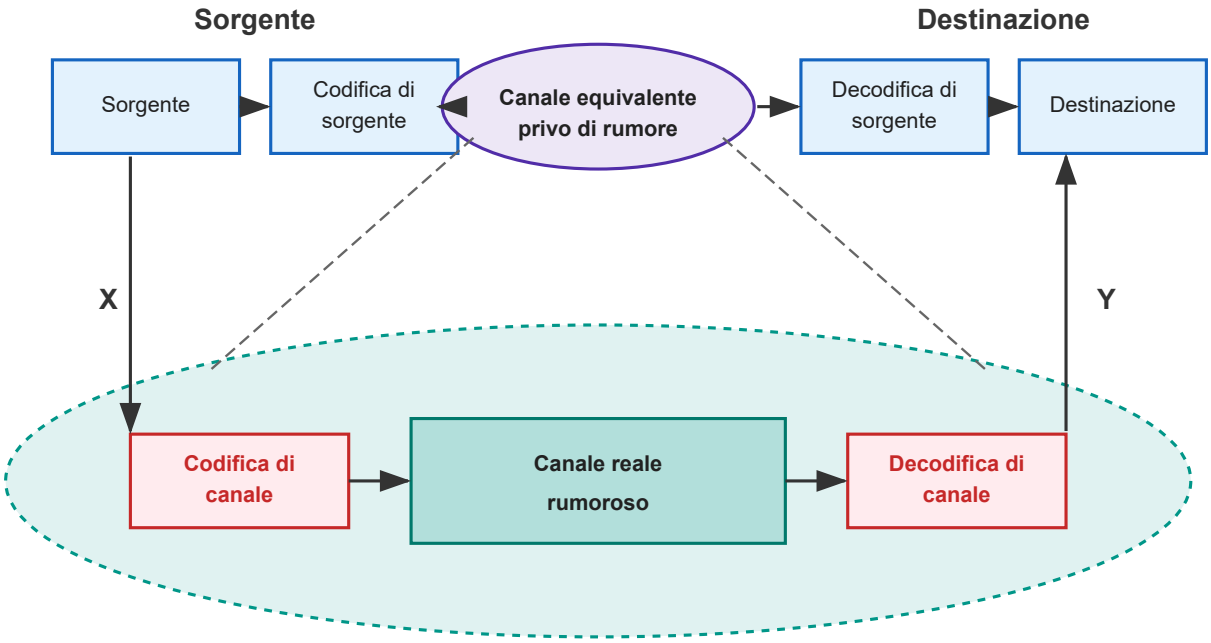


Figura 4.1: Modello teorico del sistema di telecomunicazione

Per semplicità, nel prosieguo della trattazione assumeremo che sia la sorgente, sia il canale di trasmissione siano discreti, tempo-invarianti e senza memoria. Su tali basi definiremo la quantità di informazione trasferita e la capacità del canale stesso.

4.2 Modello Matematico del Canale Discreto

Un canale di comunicazione **discreto** è caratterizzato da un alfabeto finito in ingresso (o sorgente) $X = \{x_1, x_2, \ldots, x_K\}$ di cardinalità K, e un alfabeto finito in uscita (o destinazione) $Y = \{y_1, y_2, \ldots, y_M\}$ di cardinalità M. Fenomeni *rumorosi* o scelte progettuali possono far sì che l'alfabeto di destinazione differisca da quello in ingresso (ad esempio, possono verificarsi fenomeni di *cancellazione*).

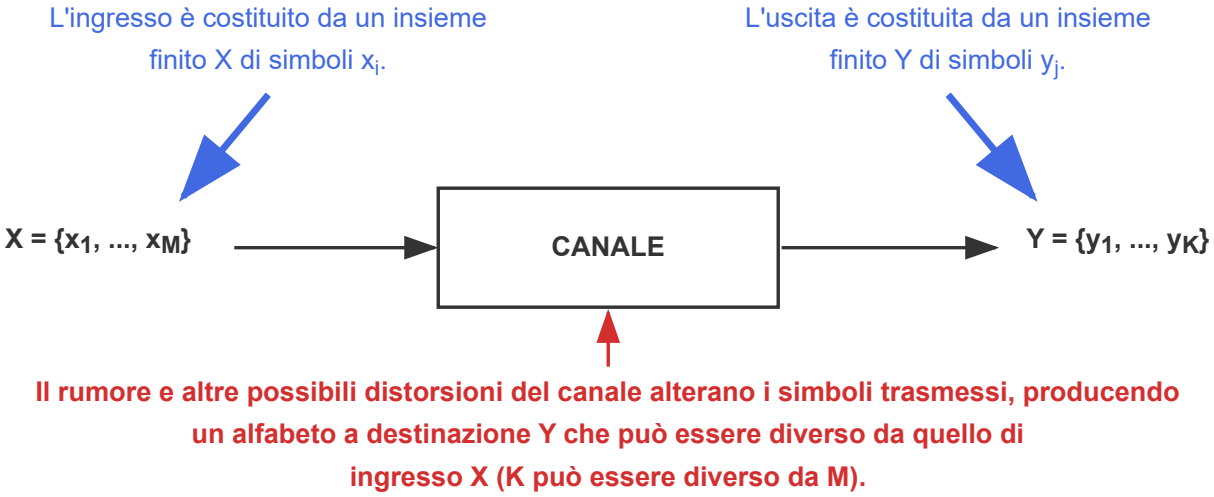


Figura 4.2: Modello del canale discreto

Notazione probabilistica

Per descrivere statisticamente il comportamento del canale si definiscono le seguenti probabilità:

- $P(x_i)$: probabilità *a priori* che x_i venga emesso dalla sorgente.
- $P(y_j)$: probabilità che y_j venga ricevuto a destinazione.
- $P(x_i, y_j)$: probabilità **congiunta** che sia trasmesso x_i e contemporaneamente ricevuto y_j.
- $P(x_i \mid y_j)$: probabilità **condizionata** (a posteriori) che sia stato trasmesso x_i sapendo di aver ricevuto y_j.
- $P(y_j \mid x_i)$: probabilità **condizionata** (di transizione in avanti) che venga ricevuto y_j data l'emissione di x_i.

4.2.1 Canale discreto senza memoria tempo-invariante

Nella nostra trattazione faremo riferimento a un modello di canale con due ipotesi semplificative fondamentali:

- **Senza memoria:** l'uscita y_j in un determinato istante t dipende statisticamente **solo** dal simbolo in ingresso x_i nello stesso istante t, ed è del tutto indipendente dai simboli trasmessi o ricevuti negli istanti precedenti.
- **Tempo-invariante (o stazionario):** le probabilità condizionate di transizione $P(y_j \mid x_i)$ non dipendono dall'istante di tempo in cui avviene la trasmissione, ma sono costanti nel tempo.

Un canale di questo tipo è **univocamente** definito dagli alfabeti X e Y, e dall'insieme delle probabilità di transizione in avanti.

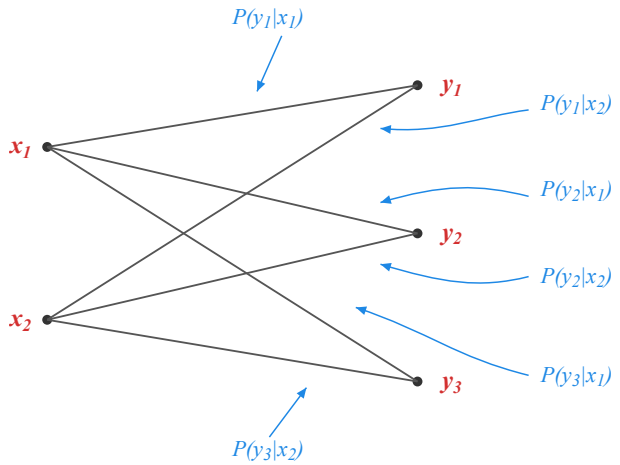


Figura 4.3: Probabilità di transizione in avanti di un canale discreto con $K = 2$ e $M = 3$.

Matrice di transizione

Essendo le probabilità di transizione stabilite e invarianti, possono essere organizzate in una matrice di dimensione $K \times M$, detta **matrice di transizione del canale** (o matrice canale):

$$P = [P(y|x)] = \begin{bmatrix} P(y_1|x_1) & P(y_2|x_1) & \dots & P(y_M|x_1) \\ P(y_1|x_2) & P(y_2|x_2) & \dots & P(y_M|x_2) \\ \vdots & \vdots & \ddots & \vdots \\ P(y_1|x_K) & P(y_2|x_K) & \dots & P(y_M|x_K) \end{bmatrix}$$

Trattandosi di probabilità condizionate, per la legge della probabilità totale, la somma degli elementi di ciascuna riga deve essere strettamente pari a 1:

$$\sum_{j=1}^{M} P(y_j|x_i) = 1 \quad \forall i = 1, \dots, K$$

$$P = \begin{bmatrix} P(y_1 \mid x_1) & P(y_2 \mid x_2) & \dots & P(y_K \mid x_1) \\ P(y_1 \mid x_2) & \ddots & & \\ \vdots & & \ddots & \\ P(y_1 \mid x_M) & & & P(y_K \mid x_M) \end{bmatrix}$$

Figura 4.4: Matrice delle probabilità di transizione.

4.3 Misure di Informazione e Capacità

Per caratterizzare rigorosamente le prestazioni di un canale discreto e determinare i suoi limiti *teorici* di trasmissione, è necessario estendere gli strumenti probabilistici introdotti per la sorgente al sistema congiunto ingresso-uscita. Mentre l'entropia di sorgente $H(X)$ quantifica l'incertezza *a priori* sul messaggio generato, ora l'attenzione è spostata sulla misura di come tale incertezza si modifichi a seguito dell'attraversamento del mezzo trasmissivo. Vogliamo cioè quantificare l'incertezza introdotta dal rumore e, per differenza, l'informazione utile che sopravvive alla trasmissione. A tal fine, introduciamo nuove misure per l'entropia, che descrivono la relazione statistica tra la variabile aleatoria in ingresso a un canale, X, e quella di uscita Y.

4.3.1 Entropie Condizionate: Rumore ed Equivocazione

Alla ricezione di un particolare simbolo, essendo il canale *rumoroso*, non abbiamo assoluta certezza su quale simbolo di X in ingresso abbia portato a quel particolare y. L'evento *ricezione di y* è un evento condizionato dall'evento **emissione di** x. Le probabilità sono descritte nella matrice di uscita - entrata e, in base a quei valori, possiamo calcolare l'**entropia condizionata per un particolare simbolo**:

$$H(X|y_j) = \sum_{x_i \in X} P(x_i|y_j) \log \left(\frac{1}{P(x_i|y_j)} \right)$$

$H(X|y_j)$ quantifica l'**incertezza sul simbolo trasmesso** x_i **alla ricezione del simbolo** y_j. Il valore medio di $H(X|y_j)$ calcolato su tutti i possibili valori di y_j identifica l'**equivocazione** (o entropia condizionata):

$$H(X|Y) = \sum_{j=1}^{M} H(X|y_j)P(y_j) = \sum_{j=1}^{M} \sum_{i=1}^{K} P(x_i, y_j) \log_2 \left(\frac{1}{P(x_i|y_j)} \right)$$

Questa grandezza rappresenta l'incertezza residua **media** che permane sul simbolo trasmesso X, pur avendo osservato l'uscita Y. Nella Teoria dell'Informazione questo valore prende il nome di **Equivocazione**: misura la quantità di informazione che è andata perduta nel canale a causa del rumore e che, di conseguenza, non è giunta al destinatario. Si tratta, in sostanza, di un'incertezza *a posteriori* rispetto alla ricezione dei simboli.

Ambiguità (o Entropia del Rumore). Esiste la quantità duale rispetto all'equivocazione, definita come $H(Y|X)$, che misura l'incertezza sull'uscita noto l'ingresso.

$$H(Y|X) = \sum_{x_i \in X} H(Y|x_i)P(x_i) = \sum_{x_i \in X} \sum_{y_j \in Y} P(x_i, y_j) \log \left(\frac{1}{P(y_j|x_i)} \right)$$

Questa grandezza è definita come la media delle entropie di riga della matrice di transizione, pesata con le probabilità della sorgente. Essa rappresenta quantitativamente il *disordine* o *rumore* introdotto dal canale.

Entropia Congiunta. Infine, l'incertezza complessiva del sistema ingresso-uscita è descritta dall'Entropia Congiunta $H(X, Y)$. Essa misura l'informazione necessaria a descrivere contemporaneamente sia il simbolo trasmesso che quello ricevuto:

$$H(X, Y) = \sum_{i=1}^{M} \sum_{j=1}^{K} P(x_i, y_j) \log \left(\frac{1}{P(x_i, y_j)} \right)$$

L'entropia del sistema globale corrisponde alla somma dell'entropia della sorgente ($H(X)$) e dell'ambiguità introdotta dal canale ($H(Y|X)$), o dualmente:

$$H(X, Y) = H(X) + H(Y|X) = H(Y) + H(X|Y)$$

4.3.2 Informazione Mutua

Combinando le misure appena introdotte, arriviamo alla definizione di **Informazione Mutua** $I(X; Y)$. Questa grandezza quantifica la riduzione di incertezza sulla sorgente X ottenuta grazie all'osservazione dell'uscita Y. Sfruttando le relazioni tra le entropie, l'informazione mutua può essere espressa in tre modi equivalenti:

1. **Differenza tra entropia di sorgente ed equivocazione:**

   $$I(X; Y) = H(X) - H(X|Y)$$

   È la definizione base: l'informazione trasmessa è quella generata meno quella persa.

2. **Differenza tra entropia di uscita e ambiguità:**

   $$I(X; Y) = H(Y) - H(Y|X)$$

   Utile per il calcolo della capacità, isola il rumore del canale.

3. **Relazione con l'entropia congiunta:**

$$I(X;Y) = H(X) + H(Y) - H(X,Y)$$

Questa formula evidenzia che l'informazione mutua è la parte di informazione condivisa tra ingresso e uscita (la somma delle singole entropie sarebbe maggiore se fossero indipendenti).

La quantità di informazione scambiata dal canale tra le variabili aleatorie X e Y è pari alle mutue informazioni delle stesse:

$$I(X;Y) = \sum_{i=1}^{K}\sum_{j=1}^{M} P(x_i, y_j) \log_2 \left(\frac{P(x_i, y_j)}{P(x_i)P(y_j)} \right)$$

Utilizzando il teorema di Bayes $P(x_i, y_j) = P(x_i)P(y_j|x_i)$ a numeratore, e la probabilità totale a denominatore, si ottiene la forma esplicita in funzione della matrice di transizione e della sorgente:

$$I(X;Y) = \sum_{i=1}^{K}\sum_{j=1}^{M} P(x_i)P(y_j|x_i) \log_2 \left(\frac{P(y_j|x_i)}{\sum_{i=1}^{K} P(x_i)P(y_j|x_i)} \right)$$

Proprietà dell'Informazione Mutua

- **Simmetria:**

$$I(X;Y) = I(Y;X)$$

Anche se il canale fisico è direzionale (da X a Y), la dipendenza statistica è reciproca: l'informazione che l'uscita fornisce sull'ingresso è quantitativamente identica a quella che l'ingresso fornisce sull'uscita.

- **Non negatività:**

$$I(X;Y) \geq 0$$

L'informazione mutua è sempre una quantità non negativa: osservare Y nel peggiore dei casi lascia invariata l'incertezza su X, non può aumentarla.

- **Condizione di Indipendenza:** La mutua informazione è nulla solo quando le due variabili risultano *statisticamente indipendenti*; ciò avviene quando il rumore del canale impedisce totalmente il trasferimento di informazione.

Caso Ideale: Assenza di Rumore. Consideriamo il caso limite, e ideale, di un canale privo di rumore. In assenza di disturbi, l'uscita del canale è **univocamente determinata** dall'ingresso. Ogni simbolo y_j identifica uno ed un solo x_i:

$$P(x_i \mid y_j) = 1 \Rightarrow I(x_i; y_j) = \log_2 \left(\frac{1}{P(x_i)} \right) = I_i$$

Conseguentemente, l'informazione veicolata dal canale coincide con l'autoinformazione del simbolo in ingresso. In media si ottiene il limite superiore:

$$I(X;Y) = H(X)$$

Caso Peggiore: Canale inutile (rumore infinito). Il caso limite opposto si ha quando il rumore è così elevato da rendere ingresso e uscita statisticamente indipendenti. Ricevere un simbolo y_j non fornisce alcun contributo alla risoluzione dell'incertezza su x_i:

$$P(x_i \mid y_j) = P(x_i) \Rightarrow I(x_i; y_j) = \log_2 \left(\frac{P(x_i \mid y_j)}{P(x_i)} \right) = \log_2 1 = 0$$

In questo caso, l'informazione mediamente trasferita è **nulla** ($I(X;Y) = 0$).

4.4 Capacità di Canale

Definiamo la **Capacità di Canale** C_S come la misura delle prestazioni massime teoriche di un canale (*discreto senza memoria*). Matematicamente, si definisce come il **massimo** dell'informazione mutua, calcolabile variando le distribuzioni di probabilità in ingresso:

$$C_S = \max_{\{P(x_i)\}} \{I(X;Y)\} \quad \left[\frac{\text{bit}}{\text{simbolo}}\right]$$

Espandendo l'informazione mutua utilizzando le misure di **equivocazione** e **ambiguità**, possiamo scrivere:

$$C_S = \max_{\{P(x_i)\}} [H(X) - H(X|Y)] = \max_{\{P(x_i)\}} [H(Y) - H(Y|X)]$$

Il valore di C_S è *indipendente dalla specifica sorgente*, poiché viene calcolato ipotizzando la distribuzione di probabilità in ingresso ottima, ovvero quella in grado di massimizzare il trasferimento di informazione. Di conseguenza, C_S dipende unicamente dalla matrice di transizione (caratteristica intrinseca del canale), mentre l'Informazione Mutua $I(X;Y)$ è una misura che varia in funzione della specifica statistica della sorgente X effettivamente utilizzata.

Capacità per unità di tempo. Può essere utile esprimere la capacità del canale in termini di velocità di trasmissione. Se il sistema trasmissivo emette r simboli al secondo (**symbol rate**), la capacità del canale per unità di tempo C si ottiene moltiplicando la capacità per simbolo C_S per la frequenza di simbolo:

$$C = r \cdot C_S \quad \left[\frac{\text{simboli}}{\text{sec}} \cdot \frac{\text{bit}}{\text{simbolo}} = \frac{\text{bit}}{\text{sec}}\right]$$

C rappresenta la massima velocità teorica per il trasferimento di informazione consentita dal canale. In generale, il calcolo analitico della capacità non è banale; tuttavia, in caso di canale simmetrico, esso risulta notevolmente semplificato.

Esempio: calcolo capacità di canale simmetrico. Consideriamo un canale con alfabeti in ingresso e uscita $X = Y = \{0, 1, 2, 3, 4\}$ e la seguente matrice di transizione:

$$P = \begin{bmatrix} 0 & \frac{1}{2} & 0 & 0 & \frac{1}{2} \\ \frac{1}{2} & 0 & \frac{1}{2} & 0 & 0 \\ 0 & \frac{1}{2} & 0 & \frac{1}{2} & 0 \\ 0 & 0 & \frac{1}{2} & 0 & \frac{1}{2} \\ \frac{1}{2} & 0 & 0 & \frac{1}{2} & 0 \end{bmatrix}$$

Poiché le righe della matrice di transizione sono tutte permutazioni degli stessi valori, il canale è **simmetrico**. L'ambiguità introdotta dal canale (il termine $H(Y|X)$) dipende unicamente dai valori della riga ed è quindi una costante rispetto alla distribuzione in ingresso:

$$H(Y|X) = H(Y|x_i) = -\left(\frac{1}{2}\log_2\frac{1}{2} + \frac{1}{2}\log_2\frac{1}{2}\right) = 1 \text{ bit}$$

La capacità di canale si ottiene massimizzando l'entropia dell'uscita $H(Y)$. Sappiamo che l'entropia massima per un alfabeto di $M = 5$ simboli è $\log_2(5)$, e si ottiene quando l'uscita è equiprobabile. In un canale simmetrico, utilizzare un ingresso uniforme garantisce un'uscita altrettanto uniforme. Pertanto:

$$C_S = \max_{\{P(x_i)\}} [H(Y)] - H(Y|X) = \log_2(5) - 1 \quad \left[\frac{\text{bit}}{\text{simbolo}}\right]$$

4.5 Analisi di Canali Notevoli

Analizziamo il comportamento dell'informazione mutua e della capacità per alcune tipologie specifiche di canali, classificati in base alla struttura della loro matrice di transizione.

4.5.1 Canali Speciali (Classificazione Strutturale)

Canale Senza Rumore (Ideale). Questi canali sono caratterizzati dalla corrispondenza biunivoca tra ingresso e uscita.

- La matrice di transizione è una **matrice identità**.
- Equivocazione e Ambiguità nulle: $H(X|Y) = H(Y|X) = 0 \Rightarrow I(X;Y) = H(X) = H(Y)$.
- $C_S = \max_{P(x)}\{I(X;Y)\} = \log_2(K)$ e si ottiene quando la distribuzione della sorgente $P(X)$ è **uniforme**.

Canale senza perdite. In questo canale l'ingresso è totalmente determinato dall'uscita: l'osservazione del simbolo ricevuto elimina ogni incertezza su quale simbolo sia stato trasmesso. Matematicamente, questo si traduce in un'equivocazione nulla ($H(X|Y) = 0$). Nella matrice di transizione, ogni colonna contiene al massimo un solo elemento non nullo. L'informazione mutua risulta $I(X;Y) = H(X)$, e la capacità è $C_S = \log_2(K)$.

Canale Deterministico. Un canale si definisce deterministico quando ogni simbolo in ingresso determina **univocamente** il simbolo in uscita. La matrice di transizione avrà un solo elemento pari a 1 per riga, mentre tutti gli altri saranno pari a 0. Matematicamente questo implica un'ambiguità nulla:

$$H(Y|X) = 0$$

L'informazione mutua si riduce quindi alla sola entropia dell'uscita:

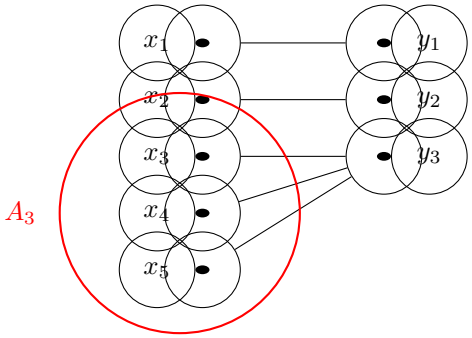
$$I(X;Y) = H(Y) - H(Y|X) = H(Y)$$

La capacità del canale è data dal massimo dell'entropia dell'uscita, ovvero il logaritmo della cardinalità dell'alfabeto di uscita M:

$$C_S = \max_{P(x)}\{H(Y)\} = \log_2 M \quad [\text{bit/simbolo}]$$

Si tratta del caso *speculare* al canale senza perdite: in questo caso non si crea incertezza *nella trasmissione*.

$$p(y_j \mid x_i) = \begin{cases} 1 & \text{se } i \in A_j \\ 0 & \text{se } i \notin A_j \end{cases}$$



Esempio: $K = 5, M = 3; A_3 = \{x_3, x_4, x_5\}$

Nota bene: Il fatto che il canale sia deterministico non garantisce che sia senza perdite. Come si evince dalla definizione, se più ingressi "collassano" sullo stesso simbolo di uscita (funzione non iniettiva), si genera incertezza nel processo inverso. In tal caso l'equivocazione è **strettamente positiva** ($H(X|Y) > 0$), rendendo impossibile la perfetta ricostruzione dell'ingresso.

Canale Totalmente Rumoroso (Inutile). Quando il rumore in un canale è massimo, l'uscita sarà statisticamente indipendente dall'ingresso. La matrice di transizione sarà quindi costituita da righe identiche a segnalare che la probabilità di ricevere un certo simbolo y è la stessa, indipendentemente da quale x sia stato trasmesso:

$$P(y_j|x_i) = P(y_j|x_k) \quad \forall i, k$$

In queste condizione un ricevitore non avrà alcuna possibilità di distinguere gli ingressi. Matematicamente, l'indipendenza statistica ($P(y|x) = P(y)$) implica che l'osservazione dell'uscita non riduce l'incertezza sull'ingresso:

$$H(X|Y) = H(X)$$

Di conseguenza, l'informazione mutua crolla a zero:

$$I(X;Y) = H(X) - H(X|Y) = 0$$

La capacità del canale è nulla ($C_S = 0$): il canale è di fatto "interrotto" e non permette alcun trasferimento di informazione.

4.5.2 Canali Simmetrici e Uniformi

Abbandoniamo i casi limite (ideali o degeneri) per analizzare una classe di canali che modella molte situazioni reali: i **canali simmetrici**. Intuitivamente, quando il rumore incide negativamente su tutti i possibili simboli in ingresso in modo *bilanciato* ed *equo*, parliamo di canale **simmetrico**. Nessun simbolo di ingresso è "più protetto" o "più esposto" al rumore rispetto agli altri, e nessun simbolo di uscita tende ad apparire più spesso di altri a causa di distorsioni sistematiche del canale. In altre parole, la struttura dell'errore è invariante rispetto al simbolo trasmesso. Nella matrice di transizione, questo si manifesterà con il fatto che ogni riga sarà una permutazione di ogni altra riga e conseguentemente, lo stesso accade per tutte le colonne.

L'entropia condizionata $H(Y|x_i)$ è **indipendente** dal simbolo x_i, perché non dipende dalla riga della matrice di transizione usata per calcolarla.

$$H(Y|X) = \sum_{i=1}^{M} p(x_i) H(Y|x_i) = H(Y|x_i) \quad \textit{Possiamo scegliere un qualsiasi } x_i$$

Per l'esempio precedente si ha che la capacità è:

$$C_S = \log_2(3) - H\left(\frac{1}{2}, \frac{1}{3}, \frac{1}{6}\right) = \frac{1}{2}\log_2(3) - \frac{2}{3}$$

$$\begin{aligned} H\left(\frac{1}{2}, \frac{1}{3}, \frac{1}{6}\right) &= -\left[\frac{1}{2}\log_2\left(\frac{1}{2}\right) + \frac{1}{3}\log_2\left(\frac{1}{3}\right) + \frac{1}{6}\log_2\left(\frac{1}{6}\right)\right] \\ &= \frac{1}{2}\log_2 2 + \frac{1}{3}\log_2 3 + \frac{1}{6}\log_2 6 \\ &= \frac{1}{2}(1) + \frac{1}{3}\log_2 3 + \frac{1}{6}(\log_2 2 + \log_2 3) \\ &= \frac{1}{2} + \frac{1}{3}\log_2 3 + \frac{1}{6}(1 + \log_2 3) \\ &= \left(\frac{1}{2} + \frac{1}{6}\right) + \left(\frac{1}{3} + \frac{1}{6}\right)\log_2 3 \\ &= \frac{2}{3} + \frac{1}{2}\log_2 3 \end{aligned}$$

Sostituendo questo risultato nella formula della capacità:

$$\begin{aligned} C &= \log_2 3 - \left(\frac{2}{3} + \frac{1}{2}\log_2 3\right) \\ &= \log_2 3 - \frac{1}{2}\log_2 3 - \frac{2}{3} \\ &= \frac{1}{2}\log_2 3 - \frac{2}{3} \quad \text{[bit/simbolo]} \end{aligned}$$

Il termine $H(Y|x_i)$ rappresenta l'incertezza introdotta dal canale (rumore). Poiché in un canale simmetrico questo valore è costante per ogni riga, esso dipende esclusivamente dalle caratteristiche fisiche del canale e non dalla statistica della sorgente.

Per calcolare la capacità, dobbiamo massimizzare l'informazione mutua:

$$C_S = \max_{P(X)}\{H(Y) - H(Y|X)\} = \max_{P(X)}\{H(Y)\} - H(Y|x_1)$$

Poiché il secondo termine è fisso, massimizzare la capacità equivale a massimizzare l'entropia di uscita $H(Y)$. Sappiamo che l'entropia $H(Y)$ è massima quando la distribuzione di uscita è **uniforme** (tutti i

simboli di uscita sono equiprobabili). In un canale simmetrico, una distribuzione uniforme in ingresso genera una distribuzione uniforme in uscita.

Pertanto, la capacità si ottiene semplicemente sottraendo l'entropia di una riga al massimo valore teorico dell'entropia di uscita:

$$C_S = \log_2 K - H(Y|x_1)$$

dove K è il numero di simboli di uscita (cardinalità dell'alfabeto Y).

4.5.3 Il Canale Binario Simmetrico (BSC)

Il modello più importante di canale simmetrico è il **Canale Binario Simmetrico BSC** che modella la trasmissione di singoli bit su un canale rumoroso. È il caso più semplice di canale simmetrico. Per definizione presenta queste caratteristiche:

- alfabeto binario (simboli x_1 e x_2), caratterizzati da probabilità $P(x_1) = p$ e $P(x_2) = 1 - p$
- anche l'alfabeto a destinazione è binario, con simboli y_1 e y_2
- le probabilità di transizione valgono:

$$P(y_1 \mid x_2) = P(y_2 \mid x_1) = \alpha$$

$$P(y_1 \mid x_1) = P(y_2 \mid x_2) = 1 - \alpha$$

Canale BSC - esempio 1. La matrice di transizione risulta:

$$P = \begin{bmatrix} P(y_1/x_1) & P(y_2/x_1) \\ P(y_1/x_2) & P(y_2/x_2) \end{bmatrix} = \begin{bmatrix} 1-\alpha & \alpha \\ \alpha & 1-\alpha \end{bmatrix}$$

Canale BSC - esempio 2. Vogliamo calcolare l'informazione mutua $I(X;Y) = H(Y) - H(Y|X)$. Definiamo la funzione di entropia di sorgente binaria $\Omega(q)$ come

$$\Omega(q) = q \log_2 \left(\frac{1}{q}\right) + (1-q) \log_2 \left(\frac{1}{1-q}\right)$$

Da cui deriva che valgono le seguenti condizioni:

- $$H(Y) = \Omega\left[P(y_1)\right] = \Omega(\alpha + p - 2\alpha p)$$
- $$H(Y|X) = \Omega(\alpha)$$

  Questo perché:

$$P(y_1) = p(1-\alpha) + (1-p)\alpha = p - p\alpha + \alpha - p\alpha = \alpha + p - 2\alpha p$$

E calcolando l'ambiguità come $H(Y|X) = \sum_{i=1}^{2} P(x_i) H(Y|x_i)$, sapendo che le righe della matrice contengono le probabilità α e $1 - \alpha$, si ottiene l'incertezza introdotta dal canale:

$$H(Y|X) = \Omega(\alpha) = \alpha \log_2 \left(\frac{1}{\alpha}\right) + (1-\alpha) \log_2 \left(\frac{1}{1-\alpha}\right)$$

Mutua informazione del canale BSC

In un canale BSC l'informazione mutua sarà:

$$I(X;Y) = H(Y) - H(Y|X) = \Omega(\alpha + p - 2\alpha p) - \Omega(\alpha)$$

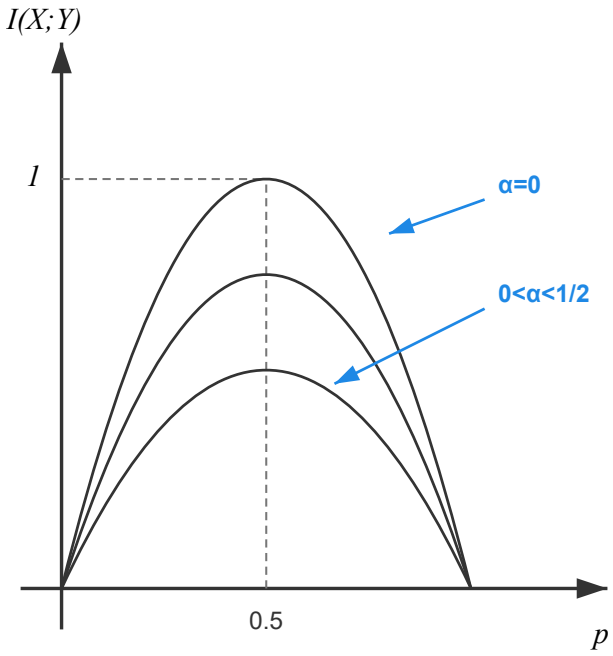


Figura 4.5: Per ogni α, la curva di $I(X;Y)$ ha il massimo per simboli equiprobabili ($p = 0.5$)

Capacità del Canale BSC

La capacità del canale è data dal massimo dell'informazione mutua. Come illustrato nel grafico, la massimizzazione si ottiene per simboli di ingresso equiprobabili ($p = 0.5$). In questo caso, l'entropia di uscita $H(Y)$ raggiunge il suo massimo teorico (essendo l'alfabeto binario, il massimo è $\log_2(2) = 1$).

$$C_S = \max_{P(X)}\{I(X;Y)\} = \log_2(2) - H(Y|x_i) = 1 - \Omega(\alpha)$$

Ovvero:

$$C_S = 1 - \left[\alpha \log_2\left(\frac{1}{\alpha}\right) + (1-\alpha)\log_2\left(\frac{1}{1-\alpha}\right)\right] \quad \text{[bit/simbolo]}$$

4.5.4 Canale simmetrico q-ario

Possiamo considerare come estensione del canale binario simmetrico un canale con q simboli.

- $M = K = q$ (simmetria)
- La matrice di transizione:

$$\begin{pmatrix} 1-\epsilon & \frac{\epsilon}{q-1} & \frac{\epsilon}{q-1} & \cdots & \frac{\epsilon}{q-1} \\ \frac{\epsilon}{q-1} & 1-\epsilon & \frac{\epsilon}{q-1} & & \vdots \\ \vdots & & & & \vdots \\ \frac{\epsilon}{q-1} & \cdots & & \cdots & 1-\epsilon \end{pmatrix}$$

- La capacità:

$$C_S = \log_2(q) - H(Y|x_1) = \log_2(q) + (1-\epsilon)\log_2(1-\epsilon) + \epsilon \log_2\left(\frac{\epsilon}{q-1}\right)$$

Essendo un canale *simmetrico*, in caso di errore, il ricevitore può ricevere **uno qualsiasi** degli altri simboli con la stessa identica probabilità.

4.5.5 Canale Simmetrico con cancellazione

In un canale con *cancellazione*, il rumore non causa l'inversione di un simbolo nell'altro, ma ne impedisce la corretta lettura. Il ricevitore è consapevole della perdita del dato, che viene quindi mappato in un *terzo* simbolo dedicato alla cancellazione. Conseguentemente, l'alfabeto di uscita avrà cardinalità maggiore di quello in ingresso ($K = M + 1$). Nel caso binario, avremo in ingresso due simboli e in uscita tre.

$$H(Y|X) = \sum_{i=1}^{M} p(x_i)H(Y|x_i) = H(Y|x_1) = H(1-\epsilon, 0, \epsilon) = H(1-\epsilon, \epsilon)$$

La capacità è data dalla formula:

$$C_S = \max_P \{H(Y) - H(Y|x_1)\} \quad \textbf{non dipende da } p$$

Essendo $H(Y|X)$ **costante** (pari a $H(1-\epsilon, \epsilon)$), bisogna massimizzare $H(Y)$.

La massimizzazione di $H(Y)$ si ottiene ponendo la probabilità di ingresso $p = 0.5$. Inserendo questo valore nello sviluppo dell'entropia di uscita, si ricava la capacità del canale con cancellazione:

$$C_S = (1-\epsilon) \quad [\text{bit/simbolo}]$$

Questo risultato è intuitivo: una frazione ϵ dei bit viene cancellata (e non fornisce informazione), mentre la restante frazione $(1-\epsilon)$ arriva intatta (fornendo esattamente 1 bit di informazione ciascuno).

4.5.6 Canale *Z*

In certe situazioni può accadere che in un canale binario i simboli 0 vengano trasmessi senza errore, mentre i simboli 1 siano affetti da una probabilità di errore che li porta ad essere erroneamente interpretati come 0. Questo schema di errori asimmetrico è rilevante per modellare alcune situazioni, soprattutto in ambito di tecnologie di memorizzazione in cui i bit di valore 1 hanno una probabilità di degradarsi col tempo, a differenza dei bit 0.

Un canale binario Z (con probabilità di errore $\frac{1}{2}$ come da esempio nelle slide) è definito nel seguente modo:

Analisi e Capacità.

- Alfabeti (simmetrici) $X = \{0, 1\}$ e $Y = \{0, 1\}$.
- **Uscita:** Se $p(x_1) = 1 - p$ e $p(x_2) = p$, le probabilità di uscita sono:

  $$p(y_2) = p(y_2|x_2)p(x_2) = \frac{1}{2}p$$

  $$p(y_1) = 1 - p(y_2) = 1 - \frac{p}{2}$$

  L'entropia di uscita risulta:

  $$H(Y) = -\frac{p}{2}\log_2\left(\frac{p}{2}\right) - \left(1 - \frac{p}{2}\right)\log_2\left(1 - \frac{p}{2}\right)$$

- **Equivocazione:** Calcoliamo $H(Y|X)$. La prima riga (input 0) è deterministica ($H = 0$), la seconda (input 1) ha entropia massima ($H = 1$ bit).

  $$H(Y|X) = p(x_1) \cdot 0 + p(x_2) \cdot 1 = p$$

- **Capacità:** Sostituendo nella formula dell'informazione mutua:

  $$C_S = \max_P \{H(Y) - H(Y|X)\} = \max_{0 \le p \le 1} \{H(Y) - p\}$$

- Il massimo della funzione si ottiene per $p = \frac{2}{5} = 0.4$, corrispondente a una capacità di:

  $$C_S \approx 0.322 \text{ bit/simbolo}$$

4.6 Tasso di Trasmissione e Criteri di Decodifica

Come anticipato, la codifica di canale affronta il problema dell'inaffidabilità introducendo ridondanza controllata, allo scopo di proteggere l'informazione dal rumore e abbattere la probabilità di errore a destinazione.

4.6.1 Codici di Canale e Tasso di Trasmissione

Per quantificare l'efficienza di questa operazione, definiamo alcune metriche fondamentali. Sia C un codice su un alfabeto q-ario:

- n è la lunghezza delle parole di codice (codeword) emesse sul canale.
- $M = |C|$ è la cardinalità del codice, ovvero il numero di codeword valide.

Poiché il numero totale di sequenze possibili di lunghezza n è q^n, per poter identificare e correggere gli errori è necessario scegliere un sottoinsieme ristretto di codeword valide, imponendo $M < q^n$ (e quindi $\log_q M < n$). Questo evita che il rumore trasformi inevitabilmente una codeword in un'altra codeword legittima.

Si definisce il **tasso di trasmissione del codice** (o *code rate*) R come:

$$R = \frac{\log_q M}{n} = \frac{k}{n} \quad \left[\frac{\text{simboli di sorgente}}{\text{simboli di canale}}\right]$$

dove $k = \log_q M$ rappresenta il numero equivalente di simboli di informazione pura trasportati da ciascuna codeword. A causa dell'aggiunta di ridondanza ($k < n$), il tasso risulta sempre $R < 1$.

L'approccio della ridondanza: esempio della decodifica a maggioranza. Per dimostrare matematicamente come l'aggiunta di ridondanza abbatta la probabilità di errore, consideriamo la trasmissione su un Canale Binario Simmetrico (BSC) con probabilità di errore sul singolo bit pari a ϵ. Senza alcuna codifica, la probabilità di ricevere un simbolo errato è banalmente $P_e = \epsilon$.

Introduciamo un semplice **codice a ripetizione** con $n = 3$: il simbolo di sorgente 0 viene codificato e trasmesso come 000, mentre il simbolo 1 come 111. Il tasso del codice scende drasticamente a $R = 1/3$.

In ricezione, applichiamo un criterio di **decodifica a maggioranza**: il blocco di 3 bit ricevuto viene decodificato nel simbolo che compare più volte. Di conseguenza, si verificherà un errore di decodifica solo se il canale altera 2 o 3 bit della codeword. Essendo il canale senza memoria, gli errori sui bit sono eventi statisticamente indipendenti. La probabilità di errore dell'intero blocco è data dalla somma dei termini della distribuzione binomiale:

$$P_e = \binom{3}{2}\epsilon^2(1-\epsilon) + \binom{3}{3}\epsilon^3 = 3\epsilon^2(1-\epsilon) + \epsilon^3$$

Ipotizzando un canale con $\epsilon = 10^{-2}$, la probabilità di errore post-decodifica scende a $P_e \approx 3 \cdot 10^{-4}$. L'inserimento della ridondanza ha sacrificato la velocità di trasmissione per ottenere un drastico guadagno sull'affidabilità.

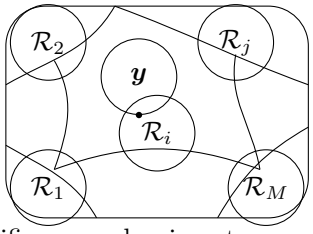
4.6.2 Il Decodificatore di Canale

Poiché, grazie alla ridondanza, lo spazio delle possibili sequenze in uscita Y è molto più ampio dell'insieme delle M parole di codice in ingresso X, il ricevitore non può operare una semplice corrispondenza biunivoca. Il decodificatore *raggruppa* le sequenze ricevute in M **classi di equivalenza** dello spazio Y.
Formalmente, si definiscono:

- **Funzione di decodifica** $\Psi : Y \to X$: associa ad ogni parola in uscita y, un elemento stimato $\tilde{x} \in X$.
- M **Regioni di decodifica**: R_i è la regione composta da tutti gli y tali che $\Psi(y) = x_i$.

Le M regioni $R_1, \ldots, R_M$ costituiscono classi di equivalenza che partizionano l'intero spazio di arrivo Y.

Introduce una **partizione** in M classi di equivalenza $\mathcal{R}_1, \ldots, \mathcal{R}_M$ su Y.



Con questa impostazione, un **errore di decodifica** si verifica quando viene trasmesso il simbolo $x_i \in X$, ma a causa del rumore si riceve una sequenza y che cade al di fuori della sua regione di competenza (cioè $y \notin R_i$). Per una data parola x_i, la probabilità che ciò avvenga è:

$$P_{err} = \sum_{i=1}^{M} p(x_i) P_{err}(x_i) = \sum_{i=1}^{M} p(x_i) \sum_{y \notin R_i} p(y|x_i)$$

Analizzando la formulazione finale, emergono considerazioni fondamentali per il progetto del sistema di codifica di canale:

- $p(x_i)$ dipende dalla sorgente
- $p(y|x_i)$ dipende dal canale

L'unico fattore su cui incide la codifica/decodifica è R_i. L'obiettivo sarà quindi scegliere le **regioni di decodifica** $R_1, \ldots, R_M$ per minimizzare P_{err}

4.6.3 Criteri di Decisione Ottimi

Come discusso in precedenza, l'obiettivo primario di un decodificatore è definire le regioni R_i in modo da rendere minima la probabilità di errata decodifica globale. I **criteri di decisione ottimi** forniscono proprio le regole matematiche per tracciare i confini di queste regioni nel modo più efficiente possibile, a seconda delle informazioni disponibili sul sistema.

Criterio dell'Osservatore Ideale

Il Criterio dell'Osservatore Ideale è la regola di decisione ottima in senso assoluto. L'obiettivo è **minimizzare la probabilità di errore media** del sistema, massimizzando di conseguenza la probabilità di **corretta decodifica a posteriori**.

Ad ogni sequenza ricevuta y è associata la probabilità di commettere un errore scegliendo una generica parola stimata $\tilde{x}$. Questa probabilità a posteriori è:

$$P_e(y) = Pr(\tilde{x} \neq x|y) = 1 - Pr(\tilde{x} = x|y)$$

Mediando su tutte le possibili sequenze in uscita, la probabilità di errore totale del sistema diventa:

$$P_e = \sum_j p(y_j) P_e(y_j) = 1 - \sum_j p(y_j) Pr(\tilde{x} = x|y_j)$$

La probabilità $p(y_j)$ di ricevere una determinata sequenza dipende unicamente dalla combinazione tra sorgente e canale ($p(y_j) = \sum_i p(x_i)p(y_j|x_i)$) e non dalla regola di decodifica. Pertanto, l'unico modo per **minimizzare** P_e è **massimizzare** la probabilità di corretta decodifica a posteriori $Pr(\tilde{x} = x|y_j)$ per ogni singola osservazione y_j.

Regola di decisione. Per ottenere questo obiettivo, il decodificatore seleziona la parola di codice x_i che risulta essere la più probabile *dopo* aver osservato l'uscita y. In formule, la parola stimata $\tilde{x}$ sarà:

$$\tilde{x} = \underset{x_i}{\operatorname{argmax}}\, p(x_i|y)$$

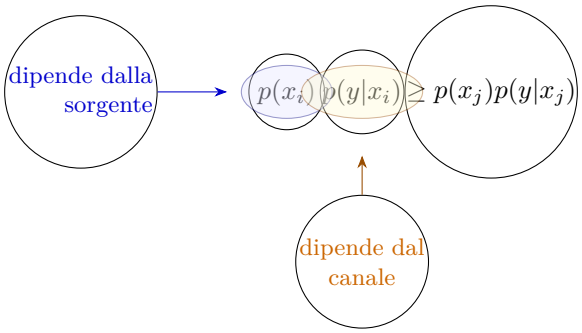
Questo significa che la funzione di decodifica assegnerà y a x_i, ovvero $\Psi(y) = x_i$, se e solo se:

$$p(x_i|y) \geq p(x_j|y) \quad \forall j \neq i$$

Applicando il Teorema di Bayes a questa disuguaglianza, otteniamo:

$$\frac{p(y|x_i)p(x_i)}{p(y)} \geq \frac{p(y|x_j)p(x_j)}{p(y)}$$

Poiché il denominatore $p(y)$ è comune a entrambi i membri ed è sempre strettamente positivo, possiamo eliderlo senza alterare la disuguaglianza. La regola di decisione operativa diventa quindi: si ha $\Psi(y) = x_i$ se $\forall j \neq i$ risulta



Inconveniente: il decodificatore è progettato tenendo conto delle probabilità a priori $p(x_i)$, dipende quindi dalla specifica sorgente. In caso di modifica delle probabilità a priori il decodificatore deve essere *tarato* nuovamente.

Criterio di Massima Verosimiglianza (ML)

Quando i messaggi in ingresso al canale sono equiprobabili (condizione che caratterizza un'uscita prodotta da una *buona* codifica di sorgente a monte), si ottiene una semplificazione del Criterio dell'Osservatore Ideale. In questo caso, la distribuzione di probabilità a priori è **uniforme**:

$$p(x_i) = \frac{1}{M} \quad \forall i$$

Applicando questa distribuzione di probabilità alla formula del Criterio dell'Osservatore Ideale ($p(y|x_i)p(x_i) \geq p(y|x_j)p(x_j)$) (sostituendo le probabilità a priori con $\frac{1}{M}$), il termine relativo alla sorgente è costante per ogni i e si elide da entrambi i membri. La scelta ottima ricade quindi unicamente sul massimizzare la probabilità dipendente dal canale.

Regola di decisione. Con questa premessa, il decodificatore, dalla sequenza ricevuta y, restituisce il messaggio x_i se la probabilità che il canale abbia prodotto proprio y a partire da x_i (ovvero la sua ***verosimiglianza***) è maggiore rispetto a qualsiasi altro messaggio x_j.

$$\Psi(y) = x_i \iff p(y|x_i) \geq p(y|x_j) \quad \forall j \neq i$$

La parola decodificata $\tilde{x} \in X$ sarà quindi tale che:

$$\tilde{x} = \underset{x_i}{\operatorname{argmax}}\, p(y|x_i)$$

Per progettare il decodificatore si lavora direttamente sulle colonne della matrice di transizione.
Nota sull'ottimalità: La sua ottimalità assoluta è garantita *solo* se i messaggi in ingresso sono perfettamente equiprobabili. Più la distribuzione effettiva si allontana dall'uniformità, più le prestazioni di questo criterio si allontanano dall'ottimo teorico.

Criterio dell'Errore Massimale

Questo criterio stabilisce a priori, per ogni codeword, un tetto massimo, δ, alla probabilità di errata decodifica.

$$P_{err} \leq \delta \quad \forall i \in X$$

La probabilità media di errore sarà:

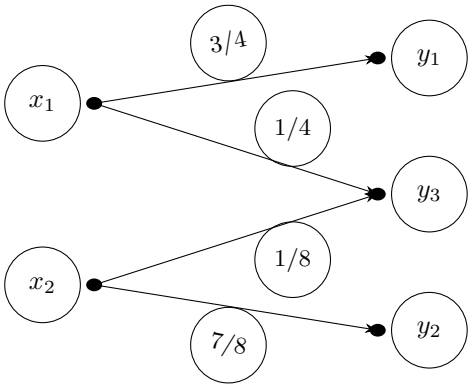
$$P_e = \sum_{i=1}^{M} P_{err}(x_i)p(x_i) \leq \delta$$

Non risulta però semplice implementare gli schemi di decodifica. Shannon ha usato questo criterio per *spingere* il tasso di trasmissione al limite della capacità di canale.

Esempi di criteri di decodifica

Esempio di osservatore ideale. Si consideri lo schema di decisione dell'osservatore ideale applicato a un canale con 2 ingressi (x_1, x_2) e 3 uscite (y_1, y_2, y_3). Il canale è completamente definito dalla sua matrice di transizione P:

$$P = \begin{bmatrix} \frac{3}{4} & 0 & \frac{1}{4} \\ 0 & \frac{7}{8} & \frac{1}{8} \end{bmatrix}$$



Vediamo come cambiano le regioni di decodifica al variare delle probabilità a priori della sorgente.

- **Caso 1: Sorgente equiprobabile** $(p(x_1) = p(x_2) = \frac{1}{2})$

  $$\Psi(y_1) = x_1, \Psi(y_2) = x_2, \Psi(y_3) = x_1 \quad R_1 = \{y_1, y_3\}, R_2 = \{y_2\}$$

  (Poiché la sorgente è equiprobabile, questo risultato coincide con il Criterio di Massima Verosimiglianza)

  $$P_{err} = \sum_{i=1}^{2} p(x_i) \sum_{y \notin R_i} p(y|x_i) = 0 + \frac{1}{2}\left(0 + \frac{1}{8}\right) = \frac{1}{16}$$

- **Caso 2: Sorgente sbilanciata** $(p(x_1) = \frac{1}{4}, \quad p(x_2) = \frac{3}{4})$

  $$\Psi(y_1) = x_1, \Psi(y_2) = x_2, \Psi(y_3) = x_2 \quad R_1 = \{y_1\}, R_2 = \{y_2, y_3\}$$

  $$P_{err} = \sum_{i=1}^{2} p(x_i) \sum_{y \notin R_i} p(y|x_i) = \frac{1}{4}\left(0 + \frac{1}{4}\right) + 0 = \frac{1}{16}$$

4.7 Secondo Teorema di Shannon

Abbiamo visto quali problemi comporta il manifestarsi di fenomeni di disturbo sul canale (introduzione di incertezza ed errori). Ci chiediamo quindi se e come si possa porre rimedio a questo fenomeno. A questa domanda ha risposto Claude Shannon con la formulazione del **Teorema di codifica di canale** (noto come *Secondo Teorema di Shannon*). Come vedremo, il risultato fondamentale è che, a certe condizioni, è possibile ottenere una trasmissione *affidabile* (con probabilità di errore tendente a zero) anche in presenza di rumore, senza dover rallentare la trasmissione fino a zero.

4.7.1 Enunciato del Teorema e Teorema Inverso

Shannon, con la sua teoria e le premesse enunciate in precedenza, si pone nell'ottica di rispondere alla necessità di definire in modo preciso e sistematico i limiti entro cui si può ottenere una trasmissione affidabile in un contesto *reale* affetto da rumore.

Il teorema delinea il compromesso fondamentale tra la probabilità di errata decodifica P_{err}, il tasso del codice R e la capacità del canale C. Il risultato sorprendente è che il rumore, di per sé, non pone

un limite alla *precisione* (cioè non impedisce di avere errore nullo), ma pone un limite alla *velocità* di trasmissione.

Teorema Fondamentale (*Conseguenze dirette*)

Dato un canale discreto senza memoria tempo-invariante di capacità $C_S \quad [\frac{bit}{simbolo}]$, per qualunque tasso di trasmissione $R < C_S$, è possibile individuare una sequenza di codici con lunghezza crescente n aventi tasso R e probabilità di errore $P_{err} \to 0$.

Questo risultato positivo garantisce che, finché rimaniamo "**sotto**" la velocità limite del canale, l'affidabilità perfetta è teoricamente raggiungibile. Tuttavia, il teorema ha anche un risvolto negativo invalicabile:

Teorema dell'Inverso (Debole e Forte)

1. **Inverso Debole:** Se $R > C$, la probabilità di errore è limitata inferiormente da una costante positiva. Non esistono sequenze di codici per cui $P_{err} \to 0$.
2. **Inverso Forte:** Se $R > C$, al crescere della lunghezza del codice ($n \to \infty$), la probabilità di errore tende necessariamente alla certezza ($P_{err} \to 1$).

Conseguenze del Secondo Teorema di Shannon. Il teorema fornisce delle *garanzie* sull'esistenza, o meno, di codifiche in grado di garantire una trasmissione affidabile a certe condizioni. Non fornisce indicazioni però su *come* realizzare tali codifiche; ci assicura solo sul fatto che, se $R < C$, una codifica che porti la probabilità di errore di decodifica ad essere arbitrariamente piccola **esiste**.

Nella pratica, però, il teorema richiede che la lunghezza delle parole di codice tenda all'infinito ($n \to \infty$) per garantire $P_{err} \to 0$ mantenendo un tasso R costante. Affinché il ricevitore possa applicare i criteri di decisione (come il MAP o l'ML), esso deve prima ricevere l'intera codeword di lunghezza n. Di conseguenza, per $n \to \infty$, il ritardo di decodifica (latenza) tende anch'esso all'infinito, rendendo i codici ottimi teorici inapplicabili ai sistemi di telecomunicazione real-time

Parte IV

Tecniche e Famiglie di Codici per il Controllo d'Errore

Capitolo 5

Strategie Generali di Controllo e Correzione d'Errore

Grazie al Secondo Teorema di Shannon, abbiamo un contesto teorico ben solido per stabilire *cosa* una codifica di canale può fare; rimane da analizzare *come* (aspetto che la teoria di Shannon non affronta esplicitamente). Per quanto concerne le capacità di un codice di canale, sappiamo dare un limite al numero di errori che potranno essere rilevati o corretti. Oltre una certa soglia, dovremo accettare il rischio, che calcoleremo analiticamente, che la decodifica non rilevi un errore o lo corregga in modo erroneo.

Per implementare questo controllo, si utilizzano i **Codici a Controllo d'Errore**, i quali introducono una ridondanza calcolata nel messaggio. A livello di sistema, questa ridondanza può essere sfruttata secondo tre strategie operative principali:

- **Correzione degli errori (FEC - Forward Error Correction):** Il ricevitore utilizza la ridondanza per individuare e correggere autonomamente gli errori, a patto che il loro numero non superi la capacità di correzione del codice. È l'unica via percorribile quando non esiste un canale di ritorno (es. comunicazioni spaziali, broadcasting TV). Si deve però accettare il rischio di non riconoscere alcuni errori o di effettuare *miscorrection* (correggere in modo errato, aggiungendo ulteriori errori a quelli del canale).
- **Rivelazione e Ritrasmissione (ARQ - Automatic Repeat Request):** Il ricevitore usa il codice solo per *accorgersi* della presenza di un errore. Se un blocco risulta errato, lo scarta e sfrutta un canale di ritorno per chiedere al trasmettitore di reinviarlo (es. protocollo TCP). È una tecnica che garantisce una probabilità di errore residuo molto minore rispetto al FEC, ma richiede un canale bidirezionale e introduce ritardi (latenza) che devono essere accettabili per l'applicazione.
- **Strategie Miste (Hybrid ARQ):** Uniscono i vantaggi delle due tecniche precedenti. Effettuano una correzione parziale (limitata a un numero di errori facilmente gestibile) e ricorrono alla richiesta di ritrasmissione solo quando il canale degrada pesantemente e gli errori superano la soglia di correzione sicura del codice.

Indipendentemente dalla strategia di rete adottata (ARQ o FEC), i codici a controllo d'errore si dividono matematicamente in due grandi famiglie in base a come elaborano il flusso di dati in ingresso:

- **Codici a Blocco:** Il flusso di dati viene diviso in blocchi di lunghezza fissa. A un blocco di k cifre di informazione vengono aggiunte $(n-k)$ cifre di controllo (parità), calcolate in modo deterministico esclusivamente sulla base di quelle k cifre. Non c'è memoria dei blocchi precedenti.
- **Codici Convoluzionali:** Elaborano l'informazione come un flusso continuo. Le cifre di parità vengono generate non solo in base all'ultimo bit di informazione arrivato, ma dipendono da un numero prefissato di bit precedenti. Sono, a tutti gli effetti, codici dotati di "memoria".
- **Codici Composti:** Combinano codici più semplici avvalendosi di tecniche di decodifica iterativa (es. codici concatenati e turbo-codici).

5.1 Capacità di Rivelazione e Correzione

Introduciamo due metriche operative che misurano le prestazioni di un codice, oltre alla *complessità realizzativa*.

- **Capacità di rilevazione:** numero massimo di errori che può rilevare in una singola codeword
- **Capacità di correzione:** numero massimo di errori che può correggere in una singola codeword
- **Efficienza:** equivale al **tasso del codice** $R = \frac{k}{n}$

Capitolo 6

Codici a Blocco

I **codici a blocco** costituiscono una delle due famiglie principali dei codici a controllo d'errore. La loro caratteristica fondamentale è l'elaborazione dell'informazione in segmenti di lunghezza fissa, trattando ciascun blocco in modo del tutto indipendente da quelli adiacenti.

6.1 Principi Fondamentali dei Codici a Blocco

In un codice a blocco, il flusso di dati viene suddiviso in segmenti di lunghezza fissa pari a k simboli (tipicamente bit). Il codificatore di canale associa a ciascun messaggio in ingresso un blocco di uscita di n simboli, chiamato **parola di codice** (*codeword*), dove $n > k$. Un codice di questo tipo viene indicato con la notazione $C(\mathbf{n}, \mathbf{k})$. Come già accennato, la composizione di un blocco di n bit non è influenzata dai dati elaborati in precedenza (**assenza di memoria**). Nella pratica si predilige l'uso dei cosiddetti **codici sistematici**. In un codice a blocco sistematico, i k bit di informazione originali non vengono mescolati o alterati, ma compaiono intatti (solitamente come prefisso o suffisso) all'interno della parola di codice lunga n, a cui vengono semplicemente accodati gli $(n - k)$ bit di parità.

6.2 Esempi Introduttivi di Codici a Blocco *Lineari*

Prima di introdurre formalmente i vari tipi di codice, analizziamo due esempi introduttivi di codici a blocco, entrambi rientranti nella categoria dei **codici lineari**. In sintesi, la proprietà fondamentale dei codici lineari è che **ogni combinazione lineare di parole di codice produce un'altra parola di codice valida**. All'atto pratico, questo significa che le cifre di controllo vengono generate tramite semplici combinazioni lineari (somme modulo 2) dei bit di informazione: grazie a questa caratteristica, l'implementazione in hardware (basata su porte logiche XOR) risulterà estremamente semplice ed efficiente.

6.2.1 Codice a Ripetizione

Il codice a ripetizione è il più banale dei codici lineari e sistematici. La ridondanza si ottiene costruendo le codeword ripetendo la trasmissione di un singolo bit di informazione n volte, ottenendo una parola di codice di lunghezza n.

$$0 \to \overbrace{000\ldots00}^{n}$$
$$1 \to 111\ldots11$$

Quindi per i codici a ripetizione:

- $k = 1$
- $q = n - k = n - 1$
- $R = \frac{1}{n}$

Alla ricezione di parole diverse da 000 e 111 si riconosce la presenza di errori, poi a seconda della strategia di codifica/decodifica scelta, è possibile:

- Rilevare errori e chiedere ritrasmissione (strategia **ARQ**);
- Rilevare errori e correggerli (strategia **FEC**);

Probabilità di errore. Assumendo che gli errori sui bit siano casuali con probabilità $P_{be} = \alpha (\alpha \ll 1)$, allora la probabilità che occorrano i errori in una parola di n bit è pari a:

$$P(i,n) = \binom{n}{i} \alpha^i (1-\alpha)^{n-i} \cong \binom{n}{i} \alpha^i$$

Quindi un codice a ripetizione sarà efficiente solo per α *sufficientemente piccolo* da verificare la condizione

$$P(i+1,n) \ll P(i,n)$$

Esempio di codice a ripetizione. Consideriamo un codice a ripetizione $C(3,1)$

- $n = 3$
- $k = 1$
- Codewords $< 000, 111 >$

In ricezione si possono attuare sia la strategia ARQ che FEC.

Messaggio	Parola di codice	Singolo errore	Doppio errore	Triplo errore
0	000	(001 010 100)	(011 110 101)	111
1	111	(011 101 110)	(100 010 001)	000

Il decodificatore si accorgerà di qualsiasi configurazione con 1 o 2 errori (poiché generano sequenze miste non appartenenti al dizionario). L'unico caso critico è l'errore triplo, che trasforma una parola valida nell'altra parola valida. Pertanto, la probabilità P_e di **non rilevare** la presenza di errori in una parola sarà

$$P_e = P(3,3) = \binom{3}{3} \alpha^3 \quad \textbf{Prob. di errata decodifica}$$

Correzione degli errori: Decodifica a Maggioranza

Se invece il sistema opera in modalità **FEC**, il ricevitore non scarta le parole errate, ma cerca di **stimare** il messaggio originale applicando il criterio di **Massima Verosimiglianza**. Per i codici a ripetizione su canale simmetrico, questo coincide con la **regola della maggioranza**: se una parola ricevuta non è una parola di codice, la si *trasforma* nella codeword ad essa più *vicina*, assumendo che almeno $\frac{(n+1)}{2}$*bit* siano corretti.

Con questa premessa possiamo correggere configurazioni con un singolo errore.

Pertanto, la probabilità di **errata decodifica** P_e sarà:

$$P_e = \binom{3}{2} \alpha^2 (1-\alpha) + \binom{3}{3} \alpha^3 = 3\alpha^2 (1-\alpha) + \alpha^3 \cong 3\alpha^2$$

6.2.2 Codici a Controllo di Parità Singola (CCP)

Altra semplice codifica a blocco (n,k) lineare e sistematica è quella dei codici a controllo di parità.

Definiamo prima il concetto di *parità*; una parola è detta con

- parità **pari** se contiene un numero **pari** di bit 1
- parità **dispari** se contiene un numero **dispari** di bit 1

In un $CCP(n, n-1)$ le parole di codice avranno $n-1$ bit di *informazione* e 1 bit di *parità*, per una lunghezza totale di n bit. Quest'ultimo bit viene scelto in modo tale che la parola di codice abbia la parità stabilita a priori per il codice.

All'estremo opposto del codice a ripetizione troviamo il Codice a Controllo di Parità (CCP). In questo schema, a un blocco di k bit di informazione viene aggiunto un singolo bit di ridondanza ($n = k+1$). Il tasso del codice risulta quindi molto alto: $R_c = \frac{k}{k+1}$.

Come si comporta un $CCP(3,2)$?

Messaggio	Parola di codice	Singolo errore	Doppio errore	Triplo errore
00	**000**	(001 010 100)	(011 110 101)	111
01	**011**	(010 001 111)	(000 101 110)	100
10	**101**	(100 111 001)	(110 011 000)	010
11	**110**	(111 100 010)	(101 000 011)	001

Nota sulle capacità di rilevazione:
Singolo e Triplo errore: Rilevati perché il numero dispari di errori altera la parità.
Doppio errore: Non rilevati perché due errori mantengono la parità inalterata (pari).

Probabilità di rivelazione e limiti di correzione. Un CCP è in grado di rilevare un **numero dispari di errori**. La probabilità P_{we} di **non** rilevare la presenza di errori in una parola, considerando una bassa probabilità di errore sui singoli bit ($\alpha \ll 1$), è quindi data da:

$$P_{we} = \sum_{\textbf{i pari}}^{n} P(i,n) \cong P(2,n) = \binom{n}{2}\alpha^2(1-\alpha)^{n-2} \cong \frac{n(n-1)}{2}\alpha^2$$

Un CCP così configurato **non può correggere errori**. Con T identifichiamo il messaggio trasmesso, con R il messaggio ricevuto dal canale:

- $T = 110001101000011010111010 \rightarrow CCP = 0$
- $R = 110001101100011010111010 \rightarrow CCP = 1$

Errore rilevato ma non corretto!

Se trasmettessimo un CCP per ogni bit trasmesso si avrebbe ad esempio:

$$CCP_T = 110001101000011010111010 = T \quad \text{doppia trasmissione di } T$$

Si corregge qualsiasi errore, ma solamente con l'irrealistica ipotesi che i bit di parità siano tutti ricevuti **sempre** correttamente. La conseguenza, però è che bisogna trasmettere il **doppio** dei bit!

Si può fare di meglio? La Parità Multidimensionale

I CCP che abbiamo introdotto offrono un'ottima efficienza, ma non possono correggere nulla. Per trovare un compromesso migliore, possiamo disporre i bit di informazione in modo strutturato, applicando controlli di parità multipli.

Controllo di Parità Bidimensionale: il messaggio T viene diviso in due *righe* e si calcolano i bit di parità per ogni colonna e per le due righe.

$$\begin{array}{rl} & 1\ 1\ 0\ 0\ 0\ 1\ 1\ 0\ 1\ 0\ 0\ 0 \quad \text{CCP} = 1 \text{ (12 bits)} \\ \oplus & 0\ 1\ 1\ 0\ 1\ 0\ 1\ 1\ 1\ 0\ 1\ 0 \quad \text{CCP} = 1 \text{ (12 bits)} \\ \hline \text{CCP} = & 1\ 0\ 0\ 0\ 1\ 1\ 0\ 1\ 0\ 0\ 1\ 0 \end{array}$$

CCP(38,14): $R = \frac{24}{38} \approx 63.1\%$***:*** *Se alla prima riga si riceve* 110001101100*, CCP della prima riga dovrebbe essere* 0 *per la prima riga e* 1 *nella decima colonna, l'errore è stato quindi individuato.*

Si può migliorare ulteriormente? Sì, impostando il messaggio come una matrice il *più quadrata* possibile.

$$\begin{array}{rl} & 1\ 1\ 0\ 0\ 0\ 1 \quad \text{CCP } 1 \\ \oplus & 1\ 0\ 1\ 0\ 0\ 0 \quad \text{CCP } 0 \\ \oplus & 0\ 1\ 1\ 0\ 1\ 0 \quad \text{CCP } 1 \\ \oplus & 1\ 1\ 1\ 0\ 1\ 0 \quad \text{CCP } 0 \\ \hline \text{CCP} = & 1\ 1\ 1\ 0\ 0\ 1 \end{array}$$

CCP(34,10): $R = \frac{24}{34} \approx 70.5\%$***:*** *I 24 bit dati generano 4 bit di parità per le righe e 6 per le colonne. La ridondanza scende a* $q = 10$*, e l'efficienza del codice è migliorata rispetto all'esempio precedente.*

6.3 Struttura dei Codici a Blocco

Per poter analizzare e progettare codici complessi e più efficaci, è necessario definire formalmente i parametri di un sistema di codifica. Abbiamo già visto che la **ridondanza** è l'ingrediente fondamentale per garantire una maggiore resistenza al rumore. Questo pone una nuova problematica: *come gestire* questa ridondanza?

Da uno spazio, potenzialmente *enorme* di possibili q^n codeword, in un alfabeto con q simboli, se ne scelgono solamente M, con $M < q^n$ come parole di codice legittime.

Possiamo pensare di scegliere un valore di M molto piccolo; questo migliorerà la capacità di correzione, ma la ridondanza risulterà penalizzante per l'efficienza del codice. Al contrario possiamo scegliere di favorire un buon tasso di codice R, ma questo penalizza la ridondanza e quindi la capacità di rilevare e correggere errori. Ma c'è un altro fattore, oltre alle dimensioni, di un codice che determina le sue prestazioni: la sua *struttura*, ovvero **come** si scelgono le parole che lo compongono. Questo perché?

Abbiamo visto come gli errori, in certi casi, portino il decodificatore a scambiare una codeword affetta da errori per un'altra legittima. Si capisce quindi come, per alleviare questo problema, si debbano *distanziare* il più possibile le parole per evitare il rischio di errata decodifica.

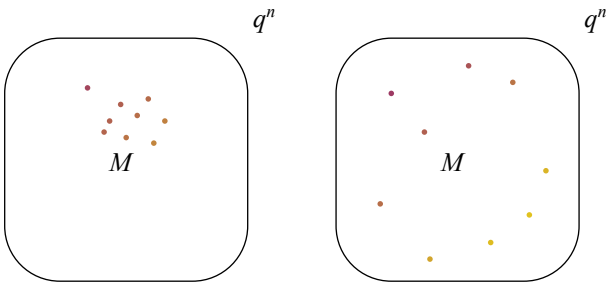


Figura 6.1: Distribuzione parole su q^n

Esempio:

$$q = 2, n = 100, R = \frac{1}{2};$$

Spazio delle **possibili** parole $\{0,1\}^n \Rightarrow 2^{100}$ parole. Quante parole di codice occorrono?

$$M = 2^{nR} = 2^{50} \approx 10^{15}$$

È necessario che le codeword siano legate da una struttura!

6.4 I Codici Algebrici

L'implementazione pratica di un codice a blocco sicuro ed efficiente non può basarsi su scelte casuali o sulla memorizzazione di dizionari sterminati. La soluzione risiede nell'introduzione di una rigorosa **struttura matematica**: nascono così i **codici algebrici**. L'idea fondamentale alla base di questi codici è dotare l'alfabeto dei simboli e l'insieme delle parole di codice di specifiche proprietà matematiche. In questo modo, le codeword faranno parte di strutture ben definite (come gruppi, campi o spazi vettoriali), e le operazioni di codifica e decodifica si trasformano in operazioni **algebriche** (come moltiplicazioni matriciali o calcoli polinomiali), facilmente implementabili in hardware o algoritmi software.

Si assume che l'**alfabeto di canale** $\mathcal{A}$ sia un **campo finito (di Galois)**, indicato con $GF(q)$, dove $q = p^k$, con p **numero primo** e $k \geq 1$. Il campo ha associate le operazioni di prodotto e somma tra i suoi elementi:

$$\mathcal{A} = \{a_1, a_2, \ldots, a_q\} \quad (\mathcal{A}, +, *)$$

Esempio: $GF(2): \quad \mathcal{A} = \{\prime, \infty\}, q = 2$

+	0	1
0	0	1
1	1	0

*	0	1
0	0	0
1	0	1

L'insieme $\mathcal{A}^n$ può essere dotato di una struttura di spazio vettoriale sul campo $GF(q)$

- Prodotto:
$$s, u_i \in \mathcal{A} : s \cdot (u_1, \ldots, u_n) = (s * u_1, \ldots, s * u_n) \in \mathcal{A}$$
- Somma:
$$u_i, v_i \in \mathcal{A} : (u_1, \ldots, u_n) + (v_1, \ldots, v_n) = (u_1 + v_1, \ldots, u_n + v_n) \in \mathcal{A}$$

Esempio: $\mathcal{A} = \{0, 1\}, q = 1, n = 3$

$$x = 001, y = 101 \in \mathcal{A}^3$$
$$x + y = 100$$
$$1 \cdot 101 = 101$$
$$0 \cdot 101 = 000$$

6.4.1 Richiami di Strutture Algebriche

6.5 Geometria dei Codici: Spazio e Distanza di Hamming

Con spazio di **Hamming** si identifica lo spazio vettoriale delle sequenze $\mathcal{A}^n$. Nello spazio di Hamming differenti stringhe lunghe n si considerano **adiacenti** se differiscono in una delle n posizioni. La **Distanza di Hamming** $d_H(x, y)$ indica per quante posizioni le stringhe x e y differiscono.

$$d_H(x, y) :=| \{i : x_i \neq y_i\} |$$

È una metrica:

1. $d_H(x, y) = 0 \iff x = y$
2. $d_H(x, y) = d_H(y, x)$
3. $d_H(x, y) + d_H(y, z) \geq d_H(x, z)$

Esempio: Distanze di Hamming

$$d_H(00101, 10101) = 1$$
$$d_H(214013, 244114) = 3$$

Il Vettore di Errore. Consideriamo i vettori di ingresso e di uscita dal canale. Se trasmettiamo una parola $\vec{x}$ e il canale introduce del rumore indipendente sui singoli bit, riceveremo una sequenza alterata $\vec{y}$. Sia $\vec{e}$ il segnale di rumore (dove $e_i = 0$ se il simbolo è ricevuto correttamente). Definiamo il **vettore di errore** $\vec{e}$ tale che:

$$\vec{y} = \vec{x} + \vec{e} \implies \vec{e} = \vec{y} - \vec{x}$$

Esempio: Alfabeto Binario, lunghezza 5

Consideriamo un alfabeto binario $\mathcal{A} = \{0, 1\}$ con parole di lunghezza $n = 5$.

- Trasmettiamo: $x = 01101$
- Riceviamo: $y = 10101$
- Il vettore errore risulta: $e = y - x = 11000$

Peso di Hamming. Strettamente legato al concetto di distanza è il **Peso di Hamming**, indicato con $wt(x)$, che rappresenta semplicemente il numero di elementi non nulli presenti in un vettore x. Applicando questa definizione al vettore di errore $e = y - x$, emerge una proprietà fondamentale per l'analisi e la progettazione dei codici: **il peso del vettore errore, $wt(e)$, indica esattamente il numero di errori commessi dal canale** durante la trasmissione.

$$d_H(x, y) = wt(y - x) = wt(e)$$

6.6 Criterio di decodifica a distanza minima

Con i concetti introdotti, si introduce il criterio di decodifica a distanza minima. Dato un messaggio $y \in \mathcal{A}^n$ in uscita dal canale, si decodifica con la parola $x \in \mathcal{A}$ tale per cui la distanza di Hamming tra le due risulta minima.

Proposizione: Il criterio di decodifica a distanza minima corrisponde a un *criterio di massima verosimiglianza*. **Dim:** Supponendo di usare lo schema a massima verosimiglianza in un canale BSC, si *stima* $\tilde{x} = x$ se $p(y|x) = \max_{1 \le i \le M} p(y|x_i)$. Si ha che

$$p(y|x) = p(x + e|x) = p(e|x) = p(e)$$

Sia $wt(e)$ il peso relativo al vettore di errore, allora:

$$p(e) = \epsilon^{wt(e)}(1 - \epsilon)^{n - wt(e)} = (1 - \epsilon)^n \left(\frac{\epsilon}{1 - \epsilon}\right)^{wt(e)}$$

Essendo $(1-\epsilon)^n$ una costante indipendente dal vettore d'errore, la massimizzazione dipende esclusivamente dal termine $\left(\frac{\epsilon}{1-\epsilon}\right)^{wt(e)}$. Assumendo che il canale sia "buono", ovvero che la probabilità di errore sul singolo bit sia $\epsilon < \frac{1}{2}$, si ottiene che la base dell'esponenziale è strettamente minore di 1 $\left(\frac{\epsilon}{1-\epsilon} < 1\right)$. Poiché stiamo elevando a potenza un numero minore di uno, per **massimizzare** l'intero termine (e quindi $p(y|x)$), è matematicamente necessario **minimizzare l'esponente** $wt(e)$.

Ma $wt(e) = wt(y - x) = d_H(x, y)$; quindi bisogna **minimizzare** $d_H(x, y)$ rispetto a x.

Osservazione: la probabilità $p(e)$ è decrescente col peso di e; vettori di errore più *leggeri* sono più probabili.

Esempio: Codice a ripetizione binario $n = 3$, $\mathcal{A} = \{0, 1\}$

In questo codice, lo spazio di Hamming è un cubo tridimensionale avente $2^3 = 8$ vertici:

$$\mathcal{A}^3 = \{000, 001, 010, 011, 100, 101, 110, 111\}, \quad C = \{000, 111\}$$

La decodifica a distanza minima partiziona questo cubo in due regioni R_1 e R_2, associando ogni vertice (sequenza ricevuta) alla parola di codice più vicina.

6.7 Capacità di correzione del codice

Possiamo ora valutare le capacità di un codice. Introduciamo il concetto di **distanza minima del codice**: si tratta della distanza di Hamming minima tra tutte le coppie di codeword.

$$d_{min} = \min_{x \neq x' \in C} d_H(x, x')$$

Consideriamo quindi $S_t(x)$, la **sfera centrata** in x:

$$S_t(x) = \{y \in q^n : d_H(x, y) \leq t\}$$

In un codice *ben progettato* le sfere di due parole saranno disgiunte:

$$x \neq x' \implies S_t(x) \cap S_t(x') = \emptyset$$

L'idea è quella di avere le sfere centrate su ogni parola il più grandi possibile, in modo da *intercettare* parole affette da errore ma sempre riconducibili alla parola inviata in origine sul canale. Un limite è però dato dal fatto che non si devono sovrapporre le sfere. Con t indichiamo il raggio delle sfere, avremo quindi che:

$$2t + 1 \leq d_{min} \implies S_t(x) \cap S_t(x') = \emptyset$$

Il vincolo è quindi determinato dal raggio t di ogni sfera, ed è soddisfatto quando:

$$t \leq \left\lfloor \frac{d_{min} - 1}{2} \right\rfloor$$

Se soddisfatto, il vincolo garantisce che se $wt(e) \leq t$, allora la decodifica a distanza minima risulta corretta.

Se un codice C ha distanza minima d_{min}, può correggere fino a t errori, con

$$t \leq \left\lfloor \frac{d_{min} - 1}{2} \right\rfloor$$

Viceversa, se a un codice è richiesto di correggere **fino a** t errori, allora deve valere:

$$d_{min} \geq 2t + 1$$

L'aggiunta di una unità alla somma dei raggi garantisce la disgiunzione tra le sfere.

Dato C, codice binario, con distanza minima d_{min}, si ha che:

1. $d_{min} = 2t + 1 \implies C$ corregge fino a t errori.
2. $d_{min} = 2t \implies C$ rileva t errori e corregge **almeno** $t - 1$ errori.

Esempio: Codice a ripetizione su canale BSC $n = 3 \quad \mathcal{A} = \{0, 1\}$

- $$\mathcal{A}^3 = \{000, 001, 010, 011, 100, 101, 110, 111\}$$
- $$d_{min} = 3$$
- $\left\lfloor \frac{d_{min}-1}{2} \right\rfloor = 1 \implies$ il codice corregge **tutte** le configurazioni di **un solo** errore

e	wt(e)	P(e)	P(e) se $\epsilon = 0.1$
000	0	$(1-\epsilon)^3$	0.729
001	1	$\epsilon(1-\epsilon)^2$	0.081
010	1	$\epsilon(1-\epsilon)^2$	0.081
100	1	$\epsilon(1-\epsilon)^2$	0.081
011	2	$(1-\epsilon)\epsilon^2$	0.009
101	2	$(1-\epsilon)\epsilon^2$	0.009
110	2	$(1-\epsilon)\epsilon^2$	0.009
111	3	ϵ^3	0.001

$P_C = 0.972$ – probabilità di **CORRETTA** decodifica
$P_e = 0.028$ – probabilità di **ERRORE** in decodifica

Generalizzando, per un codice a ripetizione, la probabilità di errata decodifica è data da:

$$P_e = Pr\{wt(e) \geq t+1\} = \sum_{i=t+1}^{n} \binom{n}{i} \epsilon^i (1-\epsilon)^{n-i}$$

- $\binom{n}{i}$ quantifica le configurazioni di i errori su una sequenza di n simboli
- $\epsilon^i(1-\epsilon)^{n-i}$ la probabilità che si verifichi una specifica sequenza di errore di peso i

Un codice a ripetizione corregge tutte e sole le $\sum_{i=1}^{t} \binom{n}{i}$ configurazioni di peso minore o uguale a t.
Prestazioni scadenti: con tasso di codice $k = 1$

$$R = \frac{k}{n} = \frac{1}{n} \to 0$$

Capacità di rilevazione ottima: Rapportando la distanza minima alla lunghezza del blocco, il codice rileva il massimo numero possibile di errori (se si riceve una parola non apppartenente al codice, si sa con certezza che c'è stato errore).

$$\lambda = \frac{d_{min}}{n} = \frac{n}{n} = 1$$

6.8 Codici perfetti

L'obiettivo nella progettazione di un codice è massimizzare la distanza minima d_{min} (e quindi la capacità di correzione t) a parità di tasso del codice R e lunghezza n, inserendo al contempo il maggior numero possibile M di parole valide all'interno dello spazio q^n.

Analizziamo il caso in cui lo spazio delle *sfere* è sfruttato al massimo, quando si progetta un codice ***perfetto*** **Definizione**: Un codice si definisce **perfetto** se l'unione di tutte le sfere di raggio t, delle M parole di codice, riempie **esattamente** e **totalmente** l'intero spazio q^n senza lasciare alcuna sequenza fuori da tali sfere e senza alcuna sovrapposizione tra di esse.

$$\bigcup_{x \in C} S_t(x) \implies M \left(\sum_{i=0}^{t} \binom{n}{i} (q-1)^i \right) = q^n$$

Nello specifico, il termine all'interno della sommatoria, $\binom{n}{i}(q-1)^i$, calcola il numero di parole che si trovano all'interno della sfera di raggio i. Sono le parole di lunghezza n in cui esattamente i posizioni sono state modificate, scegliendo per ciascuna di esse uno dei restanti $(q-1)$ simboli possibili.

6.9 I Codici Lineari a Blocco

Fino ad ora abbiamo descritto i codici a blocco in modo generale, definendoli come un sottoinsieme C di M parole valide all'interno di uno spazio più ampio q^n. Tuttavia, affinché un codice sia implementabile tramite circuiti logici, deve possedere una struttura matematica rigorosa. Nasce così la famiglia più importante dei codici a blocco: i **Codici Lineari**.

6.9.1 Definizione e Proprietà Fondamentali

Un codice a blocco $C(n, k)$ su un alfabeto (*campo*) con q simboli si definisce **lineare** se le sue $M = q^k$ parole di codice formano un **sottospazio vettoriale** di dimensione k all'interno dello spazio totale di dimensione n. Di un codice lineare possiamo definire anche come parametro la distanza minima richiesta, con $C(n, k, d_{min})$.

- n: lunghezza codeword
- k: dimensione (bit di informazione)
- d_{min} distanza minima (*la notazione può essere estesa per includere esplicitamente la distanza minima*)

La *linearità* garantisce proprietà molto importanti e *utili*: la *chiusura* rispetto alle operazioni di somma e prodotto.

- la **somma** di due parole di codice è ancora una parola di codice
- il **prodotto** di una parola di codice per un elemento di $\mathcal{A}$ è ancora una parola di codice
- Una conseguenza della chiusura rispetto a somma e prodotto è il fatto che la codeword *nulla*, composta solo di simboli 0, deve far parte del codice

Esempio: Codice NON Lineare

$$C = \{010, 111\} \quad 010 + 111 = 101 \notin C \implies \text{codice } \textbf{non} \text{ lineare}$$

Esempio: Codice Lineare C(3,1)

$$C = \{000, 111\}$$

- $000 + 111 = 111 \in C$
- $000 + 000 = 000 \in C$
- $111 + 111 = 000 \in C$
- $000 \cdot 0 = 000 \in C$
- $111 \cdot 0 = 000 \in C$
- $000 \cdot 1 = 000 \in C$
- $111 \cdot 1 = 111 \in C$

$\implies C$ è un codice lineare.

6.9.2 La Matrice Generatrice e la Codifica

Poiché il codice costituisce un sottospazio vettoriale di dimensione k, lo si può definire tramite una **base** composta da k **vettori linearmente indipendenti** di lunghezza n: $\vec{g}_1, \vec{g}_2, \ldots, \vec{g}_k$. Disponendo gli elementi della base come righe di una matrice, otteniamo la **matrice generatrice del codice** $G_{k\times n}$.

$$G = \begin{pmatrix} g_{11} & g_{12} & \cdots & g_{1n} \\ g_{21} & g_{22} & \cdots & g_{2n} \\ & \vdots & & \\ g_{k1} & g_{k2} & \cdots & g_{kn} \end{pmatrix}$$

Il nome deriva dal fatto che, tramite combinazioni lineari tra righe, *genera* parole di codice. Permutando le righe, si ottiene un codice *equivalente*.

Esempio: $C(5,1)$

$$G = (1 \quad 1 \quad 1 \quad 1 \quad 1) \implies C = \{11111, 00000\}$$

Si tratta di un codice a ripetizione con $d_{min} = 5$ che corregge fino a due errori.

Esempio: $C(5,3)$

$$G = \begin{pmatrix} 1 & 1 & 1 & 0 & 0 \\ 0 & 0 & 1 & 1 & 0 \\ 1 & 1 & 1 & 1 & 1 \end{pmatrix}$$

Le parole di codice sono $q^k = 2^3 = 8$:

00000111000011011111

11010000111100100101

La distanza minima è $d_{min} = 2$, quindi il codice può rilevare errori di peso $d_{min} - 1 = 1$, e non può correggere alcuna configurazione di errore di peso 1 (per farlo servirebbe $d_{min} = 3$).

Esempio: $C(7,4)$

$$G = \left(\begin{array}{cccc|ccc} 1 & 0 & 0 & 0 & 0 & 1 & 1 \\ 0 & 1 & 0 & 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 0 & 1 & 1 & 0 \\ 0 & 0 & 0 & 1 & 1 & 1 & 1 \end{array}\right)$$

$$\downarrow$$

Sottomatrice di rango 4 (Identità I_4)

Le parole di codice sono $q^k = 2^4 = 16$. La matrice G in questo caso ha una forma particolare, $([I_k \mid A])$.

Matrice Sistematica

Se la matrice generatrice è della forma $([I_k \mid A])$, con I_k la *matrice identità* di ordine k, e $A_{k\times(n-k)}$, allora si dice **sistematica** o **in forma canonica**. La *sistematicità* implica che, a seguito della codifica, i primi k bit delle codeword saranno identici ai bit del messaggio originale (il *payload* lungo k). Questo rende l'estrazione dell'informazione una operazione banale per il ricevitore.

6.9.3 Codifica di un codice Lineare

Le proprietà dei codici lineari semplificano il processo di codifica dei dati; si riduce la codifica a un prodotto tra vettore (dato in ingresso) e matrice (la matrice generatrice). Per codificare un vettore in

ingresso $u = \langle u_1, \ldots, u_k \rangle$ in una parola di codice $x = \langle x_1, \ldots, x_n \rangle$ si calcola

$$x = u \cdot G$$

Questo corrisponde a una trasformazione lineare da uno spazio di dimensione k a uno di dimensione n. Così viene *aggiunta* la **ridondanza** necessaria per correggere gli errori.

Esempio: $C(5,3)$

Riprendiamo la matrice generatrice $G_{3\times 5}$ vista in precedenza:

$$G = \begin{pmatrix} 1 & 1 & 1 & 0 & 0 \\ 0 & 0 & 1 & 1 & 0 \\ 1 & 1 & 1 & 1 & 1 \end{pmatrix}$$

Per codificare il messaggio $u =< u_1, u_2, u_3 >$ si calcola

$$\begin{aligned} x = u \cdot G = \\ &= (u_1, u_2, u_3) \cdot \begin{pmatrix} 1 & 1 & 1 & 0 & 0 \\ 0 & 0 & 1 & 1 & 0 \\ 1 & 1 & 1 & 1 & 1 \end{pmatrix} = \\ &= (u_1 + u_3, u_1 + u_3, u_1 + u_2 + u_3, u_2 + u_3, u_3) \end{aligned}$$

- $u = 101 \implies 00011$
- $u = 001 \implies 11111$

Esempio: Codifica in forma sistematica $C(7,4)$

$$G = \left(\begin{array}{cccc|ccc} 1 & 0 & 0 & 0 & 0 & 1 & 1 \\ 0 & 1 & 0 & 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 0 & 1 & 1 & 0 \\ 0 & 0 & 0 & 1 & 1 & 1 & 1 \end{array}\right)$$

$$x = (u_1, u_2, u_3, u_4) \cdot G = (\, u_1, u_2, u_3, u_4 \,,\, u_2 + u_3 + u_4, u_1 + u_3 + u_4, u_1 + u_2 + u_4 \,)$$

k simboli informativi $\quad$ $n - k$ simboli di controllo

Ad esempio con $u = \langle 1010 \rangle$, $x = u \cdot G = \langle 1010101 \rangle$.

6.9.4 La Matrice di Controllo della Parità (H)

Fino ad ora abbiamo analizzato il codice dal punto di vista del trasmettitore; la matrice generatrice G è lo strumento per mappare un messaggio in una parola di codice valida $\vec{x}$. Ora analizziamo il problema inverso: come si sfruttano le proprietà algebriche dei codici lineari per stabilire se la sequenza $\vec{y}$ in uscita dal canale corrisponde a una parola del codice C *legittima* o se è stata corrotta da un errore?

La linearità ci permette di definire un **test di validità algebrico** e ci evita un confronto sistematico di $\vec{y}$ con l'intero dizionario del codice. La decodifica ci permetterà quindi di riconoscere messaggi validi, o di identificare e in certi casi correggere eventuali errori. Per realizzare la decodifica serve introdurre il concetto di codice duale derivante dagli spazi ortogonali.

Codice Duale

Dato un codice lineare $C(n,k)$, definiamo il suo **sottospazio ortogonale** (o **Codice Duale**) $C^{\perp}$, come l'insieme dei vettori di $\mathcal{A}^n$ ortogonali a tutte le parole di C:

$$\begin{aligned} C^{\perp}(n,k) &= \{y \colon y \perp x, \forall x \in C\} \\ &= \{y \colon y \cdot x^T = 0, \ \textit{per ogni parola di codice}\} \end{aligned}$$

Dall'algebra lineare, essendo il codice C un **sottospazio vettoriale** di dimensione k, esisterà sempre uno spazio ad esso ***ortogonale*** di dimensioni $n-k$. I vettori di questo sottospazio $C^{\perp}$ godono di una proprietà importante: un vettore y farà parte di $C^{\perp}$ se il suo prodotto scalare con qualsiasi vettore di C (quindi qualsiasi parola di codice) restituisce esattamente lo scalare **zero**. Da questa proprietà algebrica si deriva la relazione detta **equazione di controllo** che, nei fatti, ai nostri fini realizza il controllo di validità sui messaggi ricevuti dal canale:

$$x_1 \cdot y_1 + x_2 \cdot y_2 + \cdots + x_n \cdot y_n = 0$$

Si deduce che un codice $C(n,k)$ avrà s^{n-k} equazioni di controllo.

Il Codice Duale e la Matrice H

Il codice duale $C^{\perp}$ è composto da $n-k$ vettori linearmente indipendenti, che generano il codice duale stesso; sono quindi una **base** per $C^{\perp}$. Raggruppando *per righe* i vettori di questa base, otteniamo la **Matrice di Controllo della Parità di** C **H**, che avrà quindi dimensioni $(n-k) \times n$.

$$H = \begin{pmatrix} h_{11} & h_{12} & \dots & h_{1n} \\ h_{21} & h_{22} & \dots & h_{2n} \\ & \vdots & \dots & \\ h_{(n-k)1} & h_{(n-k)2} & \dots & h_{(n-k)n} \end{pmatrix}$$

Essendo costruita con i vettori dello spazio ortogonale $C^{\perp}$, H condensa in un'unica equazione matriciale tutte le equazioni di controllo. Quindi, se $\vec{x}$ è una parola di codice, vale l'equazione:

$$H \cdot x^T = \begin{pmatrix} h_{11} & h_{12} & \dots & h_{1n} \\ h_{21} & h_{22} & \dots & h_{2n} \\ & \vdots & & \\ h_{(n-k)1} & h_{(n-k)2} & \dots & h_{(n-k)n} \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{pmatrix} = \begin{pmatrix} 0 \\ 0 \\ \vdots \\ 0 \end{pmatrix}$$

Se x non è una parola di codice, si avrà invece che:

$$H \cdot x^T \neq 0 \implies x \notin C \implies \vec{x} = \vec{z} + \vec{w} \quad (z \in C, w \in C^{\perp})$$

Matrice di controllo in forma canonica. Se la matrice generatrice G è in forma canonica ($G = [I_k \mid A]$), conseguentemente anche H si costruisce in forma sistematica:

$$H = [-A^T | I_{n-k}]_{n-k \times n}$$

Questa costruzione non è casuale; è l'unica che garantisce la proprietà fondamentale $G \cdot H^T = 0$. Possiamo fare delle osservazioni:

- H ha rango $n-k$
- In un campo $GF(2)$, lavoriamo in aritmetica *modulo 2* (dove $-1 \equiv 1$), quindi il segno meno di $-A^T$ nei fatti *sparisce*, ottenendo $H = [A^T \mid I_{n-k}]$.
- $\forall x \in C, H \cdot x^T = 0$

Ricapitolando: ogni messaggio u di k bit viene codificato in una parola di codice x tramite l'operazione $x = u \cdot G$; quindi, per come abbiamo costruito le matrici G ed H, vale la relazione $H \cdot x^T = 0$ che definisce ogni codeword **valida**. Infatti:

$$x^T = G^T \cdot u^T \implies H \cdot x^T = H \cdot G^T \cdot u^T \implies H \cdot x^T = \vec{0}_{n-k}$$

Questo poiché, per costruzione, sappiamo che $H \cdot G^T = \mathbf{0}$.

Matrice di Controllo e Distanza Minima

I vincoli nella costruzione di H consentono di utilizzarla per inferire la distanza minima del codice. Scrivendo H *per colonne*, $H = [\vec{h_1} \mid \vec{h_2} \mid \cdots \mid \vec{h_n}]$, si può riscrivere il prodotto $H \cdot x^T$, il *controllo di correttezza di* x, come somma di colonne:

$$\sum_{i=1}^{n} x_i \cdot \vec{h_i}$$

Il numero di colonne effettivamente sommate dipende dal numero di bit di x di valore 1, cioè dal **peso** $wt(x)$. Poiché una parola x appartiene a C se e solo se questa somma dà come risultato il **vettore nullo** ($\mathbf{0}$), stiamo di fatto sommando tra loro esattamente $wt(x)$ colonne della matrice. Per i principi dell'algebra lineare, se la somma di $wt(x)$ colonne (con $wt(x) > 0$) dà il vettore nullo, significa per definizione che quelle $wt(x)$ colonne sono **linearmente dipendenti** tra loro. Viceversa, per ogni combinazione lineare nulla di w colonne di H, esiste una parola x' tale per cui $wt(x') = w$.

Il legame tra il peso delle parole e le colonne di H fornisce lo strumento per calcolare la prestazione principale del codice: la sua **distanza minima** (d_{min}). Il nesso logico si basa sulla proprietà di linearità:

- La distanza di Hamming tra due parole di codice qualsiasi x e y equivale al peso della loro differenza (o somma, in modulo 2): $d(x, y) = wt(x + y)$.
- Poiché il codice è lineare, la somma di due parole di codice genera *sempre* un'altra parola di codice valida $z \in C$. Quindi: $d(x, y) = wt(z)$.

La distanza minima tra due parole equivale quindi a cercare la singola parola di codice (diversa da quella nulla) con il peso minore in assoluto. Si giunge così all'equivalenza fondamentale per i codici lineari:

$$d_{min} = \min_{x,y \in C, x \neq y} d_H(x, y) = \min_{x' \in C, x' \neq 0} d_H(x', 0) = \min_{x' \in C, x' \neq 0} wt(x')$$

La distanza minima di un codice lineare coincide con il peso minimo delle sue parole diverse da zero.

Questa equivalenza e quanto dimostrato in precedenza sulle colonne di H, portano ad enunciare il teorema finale: se la parola valida più "leggera" del codice ha peso w_{min}, significa che il numero più piccolo di colonne di H che sommate danno il vettore nullo è esattamente w_{min}. Di conseguenza:

$$d_{min} = \text{numero minimo di colonne di } H \text{ linearmente dipendenti}$$

Proprietà dei codici lineari a blocchi (sistematici)

Ricapitoliamo le proprietà fondamentali dei codici lineari a blocchi:

1. Il codice a blocchi è costituito dalla parola nulla (tutti zeri) e da tutte le possibili combinazioni lineari (somme) delle righe della matrice generatrice G.
2. La distanza di Hamming minima d_{min} coincide con il peso minimo delle codeword diverse dalla parola nulla.
3. **Distribuzione dei pesi**: è l'elenco di tutti i possibili pesi assunti dalle parole del codice, associato al numero di parole che possiedono ciascun peso.
4. **Codici equivalenti**: due codici si dicono equivalenti se le loro matrici generatrici (G e G') sono ottenibili l'una dall'altra tramite operazioni elementari sulle righe e permutazioni sulle colonne.
5. Due codici equivalenti condividono le stesse prestazioni a livello di blocco: hanno la stessa distribuzione dei pesi e la stessa probabilità di individuare una parola errata.
6. Tuttavia, a parità di canale, due codici equivalenti avranno *probabilità di errore sui singoli bit* diverse, poiché utilizzano una diversa mappatura tra messaggio e codeword (i codificatori sono diversi).
7. **Teorema di equivalenza**: ogni codice lineare a blocchi (n, k) non sistematico è sempre equivalente a un codice lineare a blocchi (n, k) in forma sistematica.

Per i codici che si presentano in forma **sistematica**, valgono in aggiunta le seguenti proprietà:

1. La matrice generatrice è sempre riconducibile alla forma *canonica* $G = [I_k \mid A]$.

2. Nel caso binario, i primi k bit della parola coincidono esattamente con i bit del messaggio, mentre i rimanenti $n-k$ sono i bit di controllo (o di parità).

3. una parola di codice sistematico può essere indicata in due modi:

$$u = (u_1, \ldots, u_k) \quad x = (u_1, \ldots, u_k, x_{k+1}, \ldots, x_n)$$

Esempio: Esempio in forma sistematica

$$\mathcal{A} = \{0,1\}, C = (7,4)$$

$$G = \begin{pmatrix} 1 & 0 & 0 & 0 & 0 & 1 & 1 \\ 0 & 1 & 0 & 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 0 & 1 & 1 & 0 \\ 0 & 0 & 0 & 1 & 1 & 1 & 1 \end{pmatrix} \qquad H = \begin{pmatrix} 0 & 1 & 1 & 1 & 1 & 0 & 0 \\ 1 & 0 & 1 & 1 & 0 & 1 & 0 \\ 1 & 1 & 0 & 1 & 0 & 0 & 1 \end{pmatrix}$$

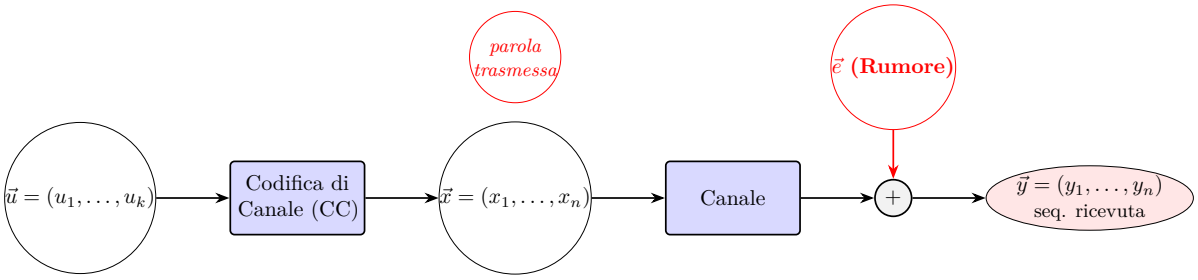
I_4 A A^T I_3

Applicando il teorema precedente alla matrice di controllo H possiamo dedurre la distanza minima del codice in modo puramente *visivo*:

- Il peso minimo **NON** può essere 1: servirebbe una singola colonna $\vec{h_i}$ linearmente dipendente da sola, il che implica che $\vec{h_i} = \mathbf{0}$; guardando H, non ci sono colonne nulle
- Il peso minimo **NON** può essere 2: servirebbero due colonne che sommate danno zero ($\vec{h_i} + \vec{h_j} = \mathbf{0}$). In aritmetica modulo 2, questo accade solo se $\vec{h_i} = \vec{h_j}$. Guardando H, non esistono due colonne identiche.
- Il peso minimo **PUÒ** essere 3: è infatti facile trovare combinazioni di tre colonne che sommate in modulo 2 si annullano (es. le prime tre colonne: $\vec{h_1} + \vec{h_2} + \vec{h_3} = \mathbf{0}$).
- è un esempio di codice di Hamming

6.9.5 Decodifica Algebrica: Sindrome e Tabella di Slepian

La codifica aggiunge ridondanza *strutturata* al messaggio originale; nella trasmissione il canale rumoroso *può aggiungere* una quota di rumore, alterando i bit in ingresso al canale. La decodifica ha quindi il duplice compito di **rivelare** se il rumore ha modificato il messaggio e, nei limiti delle capacità del codice, **correggerlo**. In termini vettoriali, il modello del canale introduce un **vettore d'errore** $\vec{e}$ (che presenta degli "1" in corrispondenza delle posizioni alterate dal rumore). La sequenza ricevuta $\vec{y}$ sarà quindi:



Da cui il vettore di errore risulta matematicamente definito come la differenza (equivalente alla somma in aritmetica modulo 2) tra il vettore ricevuto e la parola originariamente trasmessa:

$$\vec{e} = \vec{y} - \vec{x} \quad \text{(ovvero } \vec{e} = \vec{y} + \vec{x} \text{ in } GF(2))$$

Tuttavia, il decodificatore conosce *esclusivamente* la sequenza ricevuta $\vec{y}$. Non conoscendo né la parola corretta $\vec{x}$ né l'errore $\vec{e}$, l'equazione appena scritta non è direttamente risolvibile. Serve un ulteriore oggetto matematico per risolvere il problema, la **sindrome di** y. in grado di analizzare $\vec{y}$, "filtrare via" l'incognita $\vec{x}$ e far emergere solo le informazioni relative all'errore $\vec{e}$. Questo strumento è la chiave della decodifica algebrica e si basa sull'uso della matrice di controllo H.

Il Vettore Sindrome

Si definisce **sindrome di** y il vettore ottenuto moltiplicando la matrice di controllo H per il trasposto del vettore ricevuto dal canale:

$$S(y) = H \cdot y^T$$

In questa fase, H *adempie* alla sua funzione di **controllo**: i singoli zeri presenti nel vettore sindrome corrispondono alle equazioni di parità che sono state soddisfatte. Pertanto:

$$S(y) = \mathbf{0} \iff y \in C \iff x + e \in C$$

La sindrome ha dimensione $n - k$, pari al numero di equazioni di controllo. Una proprietà fondamentale è che la sequenza ricevuta y e il vettore di errore e hanno la **stessa sindrome**. Infatti, poiché x è una parola di codice valida ($H \cdot x^T = \mathbf{0}$), si ha:

$$S(y) = H \cdot y^T = H \cdot (x^T + e^T) = H \cdot x^T + H \cdot e^T = \mathbf{0} + H \cdot e^T = S(e)$$

La sindrome, come suggerisce il nome medico, *diagnostica* la presenza e la configurazione dell'errore. Dipendendo esclusivamente da quest'ultimo ($S(y) = S(e)$), permette di "isolare" l'errore dal messaggio trasmesso, ignorando x, per cercare, se possibile, di correggerlo.

Rivelazione di errori tramite Sindrome. Sfruttando le proprietà appena definite, il processo di decodifica **algebrica** si articola in una sequenza di passi:

1. **Calcolo:** Ricevuto il vettore y, il decodificatore calcola la sindrome $S(y) = H \cdot y^T$.
2. **Rivelazione:** Si verifica il valore di $S(y)$:
   - Se $S(y) = \mathbf{0}$, si assume che non si siano verificati errori (o che si sia verificato un errore non rilevabile). La parola viene accettata così com'è: la stima della parola trasmessa è $\hat{x} = y$.
   - Se $S(y) \neq \mathbf{0}$, viene rilevato un errore.
3. **Correzione (se necessaria):** Una volta rilevato l'errore, si procede a stimarne la configurazione più probabile per poter correggere la parola ricevuta.

6.9.6 Correzione di errori tramite Sindrome e Tabella di Slepian

Per poter correggere un errore, il ricevitore deve associare la sindrome calcolata $S(y)$ al vettore d'errore $\vec{e}$ che l'ha generata. Tuttavia, sorge un problema pratico: a una singola sindrome corrispondono esattamente s^k vettori di errore possibili.

Data un'ipotetica configurazione di errore $\vec{e}$, se la sommiamo a una qualsiasi parola di codice $\vec{x}_i \in C$ (con $1 \leq i \leq s^k$), la linearità della sindrome porta a:

$$S(x_i + e) = S(x_i) + S(e) = \mathbf{0} + S(e) = S(e)$$

Questo dimostra che esistono esattamente s^k sequenze diverse che condividono la **stessa identica sindrome**. Quando il ricevitore ottiene $S(y)$, ha quindi s^k "candidati" possibili per l'errore. Come sceglie quello giusto?

Si partiziona l'intero spazio vettoriale delle sequenze ricevibili ($\mathcal{A}^n$) costruendo la **Tabella Standard di Slepian** (o *Standard Array*).

La tabella distribuisce le s^n possibili sequenze ricevibili in una griglia strutturata in questo modo:

- La tabella è costituita da s^{n-k} **righe**, tante quante sono le possibili configurazioni della sindrome.
- Ogni riga contiene s^k **elementi** (colonne), tutti accomunati dall'avere la stessa sindrome. In algebra, ciascuna di queste righe prende il nome di **classe laterale** (o *coset*) del sottogruppo C.
- **La prima riga** (quella associata alla sindrome nulla $\mathbf{0}$) contiene esattamente le s^k parole di codice valide $\vec{x}_i \in C$.

$\vec{x}_0 = \vec{e}_0 = (0, \dots, 0)$	$\vec{x}_1$	$\vec{x}_2$	$\dots$	$\vec{x}_{2^k-1}$
$\vec{e}_1$	$\vec{x}_1 + \vec{e}_1$	$\vec{x}_2 + \vec{e}_1$	$\dots$	$\vec{x}_{2^k-1} + \vec{e}_1$
$\vec{e}_2$	$\vec{x}_1 + \vec{e}_2$	$\vec{x}_2 + \vec{e}_2$	$\dots$	$\vec{x}_{2^k-1} + \vec{e}_2$
$\vdots$	$\vdots$	$\vdots$	$\ddots$	$\vdots$
$\vec{e}_R$	$\vec{x}_1 + \vec{e}_R$	$\vec{x}_2 + \vec{e}_R$	$\dots$	$\vec{x}_{2^k-1} + \vec{e}_R$

Sindromi
$S(0) = \mathbf{0}$
$S(e_1)$
$S(e_2)$
$\vdots$
$S(e_R)$

Il Processo di Decodifica. Alla ricezione di una sequenza y, il decodificatore calcola la sua sindrome e la cerca all'interno della Tabella Standard. La **stima** della parola trasmessa ($\hat{x}$) si legge guardando in cima alla tabella: sarà la parola di codice situata nella **prima riga** e nella **stessa colonna** di y.

Matematicamente, questo procedimento equivale a stimare come vettore d'errore ($\hat{e}$) l'elemento che si trova nella primissima colonna della riga in cui risiede y.

Questo elemento, che "genera" l'intera riga, prende il nome di **Coset Leader** (o Capoclasse). La **qualità** e l'affidabilità della decodifica dipendono interamente dalla scelta di questi capiclasse. Per minimizzare la probabilità di errore (criterio di Massima Verosimiglianza):

*Ogni riga deve avere come Coset Leader il vettore con il **peso minimo** tra tutti quelli appartenenti a quella specifica classe laterale.*

Esempio: $C(4, 2)$

Consideriamo un codice con $s = 2$, $n = 4$, $k = 2$.

$$G = \begin{pmatrix} 0 & 1 & 1 & 0 \\ 1 & 0 & 1 & 1 \end{pmatrix} \implies C = \{0000, 0110, 1011, 1101\}$$

Costruiamo la matrice di controllo H tale che $H \cdot G^T = \mathbf{0}$:

$$H = \begin{pmatrix} 1 & 0 & 0 & 1 \\ 0 & 1 & 1 & 1 \end{pmatrix}$$

Confrontiamo ora due possibili costruzioni della Tabella di Slepian: una non ottimale, e una ottimale.

1. Tabella di Slepian NON ottimale (Scelta errata dei Leader):

Leader ($\hat{e}$)	**0000**	**0110**	**1011**	**1101**
1111	1111	1001	0100	0010
1000	1000	1110	0011	0101
0111	0111	0001	1100	1010

Sindrome
00
01
10
11

In questo scenario, ipotizziamo di trasmettere $x = 0110$ e che il canale introduca un singolo errore ($e = 0001$). Il ricevitore ottiene $y = 0111$.

- Il ricevitore cerca 0111 nella tabella: lo trova nell'ultima riga, prima colonna.
- La decodifica restituisce $\hat{x} = 0000$ (in cima alla colonna) e calcola come errore stimato $\hat{e} = 0111$ (il leader della riga).
- **Risultato:** Decodifica errata! Il sistema ha preferito credere che si siano verificati 3 errori contemporanei ($\hat{e} = 0111$) piuttosto che un errore singolo ($e = 0001$).

2. Tabella di Slepian Ottimale (Leader a peso minimo):

Leader ($\hat{e}$)	**0000**	**0110**	**1011**	**1101**
0100	0100	0010	1111	1001
1000	1000	1110	0011	0101
0001	0001	0111	1010	1100

Sindrome
00
01
10
11

Ripetiamo lo stesso scenario: trasmettiamo $x = 0110$, l'errore è $e = 0001$, riceviamo $y = 0111$.

- Il ricevitore cerca 0111 nella tabella: lo trova nell'ultima riga, seconda colonna.
- La decodifica restituisce la corretta codeword in cima alla colonna ($\hat{x} = 0110$) identificando correttamente l'errore a peso minimo ($\hat{e} = 0001$) dal Coset Leader.
- **Risultato:** Decodifica perfetta, massimizzando la probabilità di successo.

6.9.7 Probabilità di Corretta Decodifica

Come valutare le *prestazioni* di una tabella di Slepian? Una decodifica avrà successo **se e solo se** il *pattern* di errore generato dal canale corrisponde esattamente a uno dei Coset Leader scelti. Se invece il canale introduce un errore che appartiene al coset ma **non** è il suo leader, il decodificatore (che *scommette* sempre sul leader) effettuerà una correzione errata.

La probabilità di corretta decodifica P_C si calcola sommando la probabilità di occorrenza di ogni errore $\vec{e}$ scelto come Coset Leader ($P_r(e)$). Su un canale BSC con probabilità di transizione errata ϵ, si ha:

$$P_C = \sum_{e \in Leaders} P_r(e) = \sum_{e \in Leaders} \epsilon^{wt(e)}(1-\epsilon)^{n-wt(e)} = \sum_{e \in Leaders} (1-\epsilon)^n \left(\frac{\epsilon}{1-\epsilon}\right)^{wt(e)}$$

La probabilità di corretta decodifica sarà *alta* se si massimizzano i termini all'interno della sommatoria. Questo si ottiene minimizzando l'esponente:

$$\min wt(e) \implies \max \left(\frac{\epsilon}{1-\epsilon}\right)^{wt(e)}$$

Questo perché, si assume $\epsilon < \frac{1}{2} \implies \frac{\epsilon}{1-\epsilon} < 1$

Tutto quindi dipende dai **pesi** dei Coset Leader: leader con pesi minori ($wt(e)$ basso) saranno ordini di grandezza più probabili, garantendo una *migliore* decodifica. Se si progetta un codice **lineare perfetto** che corregge fino a t errori, **tutte e sole** le sequenze di errore di peso **fino a** t (con peso $0 \leq w \leq t$) devono comparire tra i Coset Leader (generatori).

Affinché questo avvenga, la somma di tutte queste sequenze possibili deve coincidere esattamente con il numero totale delle classi laterali (ovvero le s^{n-k} righe della tabella di Slepian):

$$\sum_{w=0}^{t} \binom{n}{w} (s-1)^w = s^{n-k}$$

Esempio: P_C per un codice perfetto $C(5,1)$

Consideriamo il codice a ripetizione di lunghezza 5 su canale binario ($s = 2$).

$$G = (1 \quad 1 \quad 1 \quad 1 \quad 1) \implies C = \{00000, 11111\}$$

Verifichiamo che è un codice perfetto per $t = 2$ errori. Moltiplicando le $2^k = 2$ codeword per il numero di Coset Leader di peso fino a 2, si ottiene l'intero spazio delle $2^n = 32$ sequenze ricevibili:

$$2^1 \cdot \left[\binom{5}{0} + \binom{5}{1} + \binom{5}{2}\right] = 2 \cdot (1 + 5 + 10) = 32 = 2^5$$

La Tabella di Slepian risulta:

$\binom{5}{0}$	00000	11111
$\binom{5}{1}$	10000 ⋮ 00001	01111 ⋮ 11110
$\binom{5}{2}$	11000 ⋮ 00011	00111 ⋮ 11100

Con gli errori di peso $w \leq 2$ come leader, la probabilità di corretta decodifica P_C, con probabilità di errore ϵ, diventa la somma delle probabilità di questi tre gruppi:

$$P_C = (1-\epsilon)^5 + \binom{5}{1}\epsilon(1-\epsilon)^4 + \binom{5}{2}\epsilon^2(1-\epsilon)^3$$

6.10 I Codici di Hamming (Binari)

I codici introdotti da Richard Hamming nel 1950 sono il primo esempio di uso di tecniche algebriche per la codifica di canale. Sono codici in grado di correggere **tutte** le configurazioni di **un singolo errore** e di rilevare fino a due errori. Rappresentano un classico esempio di **codice a blocchi sistematico**.

6.10.1 Costruzione

Sia $m = n - k, \forall m = 2, 3, 4, \dots$, si costruisce una matrice di controllo H tale che non abbia coppie di colonne linearmente dipendenti, in questo modo la distanza minima sarà $d_{min} = 3$.

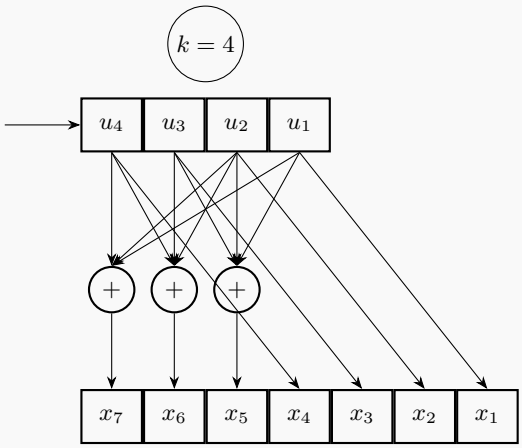
Lavorando con un alfabeto binario, si usano tutti i possibili $n = 2^m - 1$ vettori distinti (diversi dal vettore nullo) e di lunghezza m come colonne di H.

Il codice lineare $C(n, k) = C(2^m - 1, 2^m - 1 - m)$ è detto **codice di Hamming binario**.

Esempio: $C(7,4) \quad m = 7 - 4 = 3$

Il codice avrà nei primi $k = 4$ bit, gli stessi bit del dato in ingresso, nei rimanenti $n - k = 3$ delle combinazioni dei bit in ingresso:

- $x_i = u_i \quad \text{per } i = 1, 2, 3, 4$
- $x_5 = u_1 + u_2 + u_3$
- $x_6 = u_2 + u_3 + u_4$
- $x_7 = u_1 + u_2 + u_4$



La matrice generatrice e quella di controllo:

$$G = \left(\begin{array}{cccc|ccc} 1 & 0 & 0 & 0 & 1 & 0 & 1 \\ 0 & 1 & 0 & 0 & 1 & 1 & 1 \\ 0 & 0 & 1 & 0 & 1 & 1 & 0 \\ 0 & 0 & 0 & 1 & 0 & 1 & 1 \end{array}\right) \qquad H = \left(\begin{array}{ccc|ccc} 1 & 1 & 1 & 0 & 1 & 0 & 0 \\ 0 & 1 & 1 & 1 & 0 & 1 & 0 \\ 1 & 1 & 0 & 1 & 0 & 0 & 1 \end{array}\right)$$

I_4 A $-A^T$ I_3

Prestazioni e Limiti Asintotici. L'efficienza e le prestazioni di un codice di Hamming, sono indicate da due metriche di prestazione, che derivano direttamente dalla scelta del parametro costruttivo $m = n-k$ (il numero di bit di ridondanza): il **tasso del codice** (R) e il **tasso di correzione** (λ). Abbiamo che il **tasso** di un codice di Hamming è dato da:

$$R = \frac{k}{n} = \frac{2^m - 1 - m}{2^m - 1}$$

Alcuni esempi

- $m = 2 \implies R = \frac{1}{3}$ codice a ripetizione di lunghezza 3 ($C(3,1)$)
- $m = 3 \implies R = \frac{4}{7}$ (codice Hamming C(7,4)
- $m \to \infty \implies R = 1 - \frac{m}{2^m - 1} \to 1$

Il tasso di correzione è invece:

$$\lambda = \frac{d_{min}}{n} = \frac{3}{2^m - 1}$$

Alcuni esempi:

- $m = 2 \implies \lambda = \frac{3}{3} = 1$
- $m = 3 \implies \lambda = \frac{3}{7}$
- $m \to \infty \implies \lambda = 0$

6.10.2 Decodifica a Sindrome (Correzione di un singolo errore)

Per la decodifica dei codici di Hamming binari, ricordiamo la proprietà fondamentale su cui si basano: usiamo tutti i possibili $n = 2^m - 1$ vettori non nulli di lunghezza m come colonne per formare la matrice di controllo H.

$$H = (h_1, h_2, \ldots, h_n) \quad h_i \neq 0, h_i \in \{0,1\}^m$$

Assumiamo di aver trasmesso una parola di codice x e di ricevere la sequenza $y = x + e$, dove e è un vettore d'errore. I Codici di Hamming **correggono un SOLO errore**, questo implica che possiamo assumere che il peso del vettore di errore sia $wt(e) \leq 1$. Significa in sostanza che il vettore di errore avrà **al più una sola posizione** i tale che $e_i = 1$, mentre tutti gli altri bit saranno nulli.
Alla ricezione di un vettore y, la **sindrome** sarà la combinazione lineare delle colonne in corrispondenza degli errori:

$$S(y) = H \cdot y^T = H \cdot (x^T + e^T) = \mathbf{0} + H \cdot e^T = \sum_{j=1}^{n} h_j \cdot e_j = h_i$$

Essendo e un vettore con un solo 1 nella specifica posizione i, tutti i termini della sommatoria si annullano tranne l'i-esimo. Si ottiene quindi:

$$S(y) = h_i$$

La grande *utilità* della sindrome è data dal fatto che **coincide esattamente con la colonna di H associata al bit corrotto**. In che modo sfruttiamo questa proprietà all'atto pratico?

Procedura di Correzione. Poiché lavoriamo tipicamente con codici in forma **sistematica** (in cui le colonne di H non seguono un ordine numerico progressivo in base 2), la decodifica si effettua per semplice confronto con le colonne della matrice H (come se questa fosse una *Lookup Table*):

1. Si calcola la sindrome $S(y) = y \cdot H^T$.
2. Se $S(y) = \mathbf{0}$, si assume che la trasmissione sia priva di errori e si stima $\hat{x} = y$.
3. Se $S(y) \neq \mathbf{0}$, si cerca il vettore sindrome appena calcolato tra le colonne della matrice H. Trovata la colonna h_i identica alla sindrome, l'errore si trova con certezza nella posizione i.
4. Si corregge il messaggio invertendo l'i-esimo bit della sequenza ricevuta y.

Osservazione sui limiti: Errori Doppi. Cosa succede se il canale è particolarmente rumoroso e introduce **due errori** (peso $wt(e) = 2$ in posizioni i e j)? Per la linearità della sindrome, si avrà:

$$S(y) = S(e_i + e_j) = h_i^T + h_j^T$$

Poiché le colonne di H esauriscono *tutte* le combinazioni binarie non nulle, la loro somma modulo 2 produrrà inevitabilmente una **terza colonna valida** h_r^T fisicamente presente in H.

Il decodificatore, ottenendo la sindrome h_r^T, crederà erroneamente di trovarsi di fronte a un singolo errore nella posizione r. Andrà quindi a correggere il bit r-esimo, **peggiorando la situazione** e trasformando un errore doppio in un errore triplo. Questo dimostra perché un codice con $d_{min} = 3$ fallisce se si supera la sua capacità di correzione garantita ($t = 1$).

6.11 Limiti dei Codici a Blocchi e Codici Perfetti

Abbiamo già visto che la distanza minima d_{min} di un codice, ne determina le capacità correttive. È emerso anche il compromesso fondamentale che governa la codifica di canale: per aumentare d_{min} e garantire una maggiore robustezza al rumore, è necessario aumentare il numero di cifre di ridondanza, il che riduce inesorabilmente il tasso informativo ($R = k/n$) e l'efficienza della trasmissione. Nasce quindi spontanea una domanda cruciale: fissata la lunghezza del blocco n e la distanza minima desiderata d_{min}, qual è il **numero massimo di parole di codice** (e quindi la dimensione k massima) che possiamo "impacchettare" nello spazio vettoriale senza che le loro sfere di decodifica si sovrappongano?

6.11.1 Il Limite di Singleton

Il primo limite alla dimensione dei codici è dato dal Limite di Singleton: è la disuguaglianza più *diretta* e generale che lega i parametri fondamentali di un codice a blocchi.

Sia $C(n, k)$ un codice a blocchi con distanza minima d_{min} su alfabeto di cardinalità s, allora:

$$|C| \leq s^{n-d_{min}+1}$$

Dimostrazione: se $|C|$ fosse maggiore del limite, allora **almeno** due parole di codice x_1 e x_2 coinciderebbero sulle prime $n - d_{min} + 1$ cifre. Questo comporterebbe che la distanza minima tra le due parole sarebbe

$$d_H(x_1, x_2) \leq n - (n - d_{min} + 1) = d_{min} - 1 \implies \textbf{impossibile!}$$

Visualizziamo il ragionamento come analogia del ***Principio dei cassetti***:

1. *Eliminiamo* le ultime $d_{min} - 1$ cifre da tutte le parole.
2. rimangono *tronconi* di parole lunghi $n - (d_{min} - 1) = n - d_{min} + 1$.
3. le combinazioni diverse (*i cassetti*) per parole lunghe $n - d_{min} + 1$ su un alfabeto di cardinalità s sono $s^{n-d_{min}+1}$
4. ma il codice di partenza ha s^n parole, e $s^k > s^{n-d_{min}+1}$
5. questo implica che avremmo due parole di codice in partenza diverse che finiscono nello stesso *cassetto* (ovvero hanno i primi $n - d_{min} + 1$ simboli identici). Essendo identiche in questa prima parte, l'unico punto in cui possono differire sono esattamente le $d_{min} - 1$ cifre che avevamo *eliminato*. Ma se differiscono al massimo in $d_{min} - 1$ posizioni, la loro distanza reciproca sarà minore o uguale a $d_{min} - 1$, violando così la definizione stessa di distanza minima del codice!
6. per evitare questo assurdo, si deve imporre che il numero di parole sia minore delle combinazioni possibili
7. deve valere
$$|C| \leq s^{n-d_{min}+1}$$

Quindi, limitare la cardinalità del codice o la distanza minima delle sue parole sono due facce della stessa medaglia.

Nel caso specifico dei codici lineari su un alfabeto di cardinalità s, il numero delle parole di codice (ovvero il numero totale da codificare) è dato da $|C| = s^k$. Sostituendo questo valore nella disuguaglianza generale del limite di Singleton, si ottiene:

$$s^k \leq s^{n-d_{min}+1}$$

Applicando il logaritmo in base s (con $s \geq 2$) a entrambi i membri, la disuguaglianza mantiene il proprio verso poiché la funzione logaritmo con base maggiore di 1 è strettamente crescente:

$$\log_s(s^k) \leq \log_s(s^{n-d_{min}+1})$$

Dalle proprietà dei logaritmi ($\log_b(b^x) = x$), gli esponenti si "abbassano":

$$k \leq n - d_{min} + 1 \iff d_{min} \leq n - k + 1$$

Quindi per i codici lineari validi, la distanza minima è dettata dal valore di k scelto.

6.11.2 I Limiti di Hamming (Sphere Packing Bound)

Se il Singleton bound è un limite *esistenziale* per i codici lineari, ora analizziamo i limiti legati al *volume geometrico* dello spazio di Hamming, calcolando quante parole di codice possiamo *impacchettare* al suo interno senza che le loro sfere di decodifica si sovrappongano, inficiando così una corretta decodifica.

Prima evidenziamo come sono in relazione il numero di codeword (su alfabeto binario) di un codice e l'*informazione* generata da una sorgente che emette R simboli binari al secondo: per trasmettere adeguatamente l'informazione generata, se un canale può trasmettere **un solo simbolo** al secondo, servirà un insieme di 2^{nR} parole di codice.

Da ciò si intuisce come per un alto tasso di trasmissione è necessario un numero di parole di codice *grande*.

Ma quante parole di codice possono esserci in un codice affinché si possano correggere configurazioni con al più t errori per parola trasmessa? In altre parole quanto grande può essere l'insieme di parole del codice, e quindi quanta informazione può passare in un canale, volendo però poter correggere un numero congruo di possibili errori?

Hamming Upper Bound (Numero di parole di codice)

Consideriamo ogni codeword come il centro della propria *sfera di correzione*, all'interno dello spazio di Hamming. Considerando un codice **binario**, avremo $M = 2^k$ sequenze binarie lunghe n, in grado di correggere tutte le configurazioni fino a t errori. Per evitare sovrapposizioni tra sfere/sequenze deve valere la relazione:

$$\textbf{Limite di Hamming:} 2^k = 2^{n\frac{k}{n}} = 2^{nR} \leq \frac{2^n}{\sum_{i=0}^{t}\binom{n}{i}}$$

In dettaglio si ha che:

- 2^n è la dimensione dello spazio per **tutte** le sequenze binarie lunghe n.
- $$\sum_{i=0}^{t}\binom{n}{i}$$
  quantifica quante sequenze differiscono da w_i fino a t posizioni

Dimostrazione:

- $$w_1, \ldots, w_M$$
  parole di codice **binarie**
- Ad ogni parola è associata una sfera con *centro* w_i, e *raggio* t
- ognuna delle M *sfere* (partizioni) è l'insieme di **tutte** le sequenze binarie v tali che $d_H(w_i, v) \leq t$
- ogni sequenza che ricade dentro una sfera i dista al massimo t dal suo centro (parola con cui viene decodificata)
- ogni sfera *contiene* $1 + \binom{n}{1} + \cdots + \binom{n}{t} = \sum_{i=0}^{t}\binom{n}{i}$ sequenze
- il totale delle sequenze incluse nelle sfere è quindi $M\sum_{i=0}^{t}\binom{n}{i}$
- il numero di sequenze contenute nelle sfere però **non può eccedere il numero totale di sequenze binarie** in uno spazio di dimensione n:

$$M\sum_{i=0}^{t}\binom{n}{i} \leq 2^n \implies M = 2^k \leq \frac{2^n}{\sum_{i=0}^{t}\binom{n}{i}}$$

Il valore di M è un rapporto tra lo spazio e il numero di configurazioni lunghe fino a t.

Il limite di Hamming è una **condizione necessaria ma NON sufficiente** per un codice che corregge fino a t errori.

Esempio:

$$t = 1, n = 4 \implies M \leq \frac{2^n}{(1+n)} = \frac{16}{5}$$

Il numero intero più grande che soddisfa il limite di Hamming è $M = 3$. Ma non esistono codici che possano correggere singoli errori con più di 2 parole di lunghezza 4, perché la distanza minima per correggere un errore è $d_{min} = 3$

Esempio:

Possiamo correggere singoli errori, con parole di lunghezza 5 con un codice di 4 parole binarie $(00000, 10011, 11100, 01111)$.

Esempio:

Se vogliamo correggere errori doppi mantenendo parole di lunghezza 5, non possiamo usare più di due parole $(00000, 11111)$.

Hamming Lower Bound (Numero di cifre di controllo)

Con la precedente analisi si è data una limitazione alla dimensione del codice, con questa ulteriore valutazione si impone un limite inferiore al numero $q = n - k$ di cifre di controllo.

$$2^q = 2^{n-k} \geq \sum_{i=0}^{t} \binom{n}{i}$$

Dimostrazione: Se un codice a controllo di parità è capace di correggere **fino a** t errori, allora ogni combinazione di t **o meno** errori deve risultare in un **singolo correttore** (sindrome distinta).

- Il numero di possibili vettori di errore di lunghezza n contenenti esattamente i errori è dato dal coefficiente binomiale $\binom{n}{i}$.
- I possibili correttori (sindromi) sono invece $2^{n-k} = 2^q$.
- Per garantire l'unicità di ogni correttore per ogni combinazione di errore deve valere la formulazione del limite di Hamming:

$$2^{n-k} \geq 1 + \binom{n}{1} + \cdots + \binom{n}{t} = \sum_{i=0}^{t} \binom{n}{i}$$

- il termine 1 nella sommatoria è presente nel caso in cui **non si verifichino** errori (ovvero il vettore di errore nullo, a cui è associata la sindrome nulla);

L'Hamming Lower Bound è una **condizione necessaria MA NON sufficiente** per costruire codici a blocchi che correggano fino a t errori. Potrebbero esistere valori di q_0, inteso come il più piccolo intero positivo che soddisfi il Lower Bound, per valori di n e t, ma potrebbero **non** esistere codici validi con q_0 cifre di controllo.

Esempio: Lower Bound soddisfatto, codice impossibile

$$n = 10 \quad t = 2 \implies 2^q = 2^{n-k} \geq \sum_{i=0}^{t} \binom{n}{i} \implies q_0 = 6$$

Non esistono codici lineari binari che possano correggere un doppio errore con meno di 7 cifre di controllo, perché la distanza minima per correggere un doppio errore è 5!

Servirà un ulteriore teorema per dare un limite superiore al numero di cifre di controllo.

Hamming Upper e Lower Bound. Il limite *inferiore* sul **numero di cifre di controllo** (q) e il limite *superiore* sul **numero delle parole di codice** (k) sono, nei fatti, due facce della stessa medaglia. Partendo dalla formulazione del Lower Bound sulle cifre di controllo $q = n - k$:

$$2^{n-k} \geq \sum_{i=0}^{t} \binom{n}{i}$$

Sfruttando le proprietà delle potenze $2^{n-k} = \frac{2^n}{2^k}$):

$$\frac{2^n}{2^k} \geq \sum_{i=0}^{t} \binom{n}{i}$$

A questo punto, moltiplicando entrambi i membri per 2^k e dividendo per la sommatoria, ricaviamo esattamente la formula dell'Upper Bound:

$$2^k \leq \frac{2^n}{\sum_{i=0}^{t} \binom{n}{i}}$$

Questo dimostra che limitare lo spazio per le parole di codice o garantire spazio sufficiente per le sindromi d'errore sono lo stesso identico vincolo matematico.

6.11.3 Il Limite VGS (Varshamov-Gilbert-Sacks)

Il limite di Hamming definisce una condizione *necessaria* sul numero di cifre di controllo q, ma non ci garantisce che un codice con tali parametri sia effettivamente costruibile.

Adesso introduciamo una condizione **sufficiente, ma NON necessaria** per la costruibilità del codice. Un limite *sufficiente* (ma non necessario) per la costruzione di un codice è fornito dal teorema di Varshamov-Gilbert-Sacks (VGS); afferma che:

Un codice binario con parole lunghe n che corregge t errori **può essere costruito** se il numero delle cifre di controllo soddisfa la seguente relazione:

$$2^q = 2^{n-k} > \sum_{i=0}^{2t-1} \binom{n-1}{i}$$

Se per un dato n e un dato t, troviamo il più piccolo intero q_0 che soddisfa questa relazione, il limite VGS ci dà la certezza matematica che **esiste** almeno un codice lineare con q_0 cifre di controllo in grado di correggere t errori.

Esempio:

$$n = 10, t = 2 \implies 2^q > \sum_{i=0}^{2t-1} \binom{n-1}{i} \implies q_0 = 8$$

Tuttavia esistono codici che correggono errori doppi con un $q = 7$ cifre di controllo.

Confronto tra i limiti di Hamming e VGS. Applicando il logaritmo in base 2 alle disuguaglianze viste precedentemente, possiamo isolare il numero di cifre di controllo q ed evidenziare il ruolo complementare dei due limiti.

Limite di Hamming
(Condizione Necessaria)

$$q \geq \log_2 \left[\sum_{i=0}^{t} \binom{n}{i} \right]$$

Velocità di trasmissione:

$$R \leq 1 - \frac{1}{n} \log_2 \left[\sum_{i=0}^{t} \binom{n}{i} \right]$$

$\Downarrow$

Limite superiore sulla ridondanza minima: tutti i codici che correggono t errori devono rispettare questo limite.

Limite VGS
(Condizione Sufficiente)

$$q > \log_2 \left[\sum_{i=0}^{2t-1} \binom{n-1}{i} \right]$$

$\Downarrow$

Limite inferiore sufficiente sulla ridondanza minima: esiste almeno un codice che corregge t errori se si rispetta questo limite.

6.11.4 Codici Perfetti

Esiste tuttavia una classe ristretta e molto speciale di codici in cui il limite di Hamming viene soddisfatto con l'**uguaglianza esatta**:

$$2^k \sum_{i=0}^{t} \binom{n}{i} = 2^n$$

I codici che soddisfano questa equazione sono definiti **Codici Perfetti**.

Dal punto di vista geometrico, in un codice perfetto le sfere di decodifica di raggio t realizzano una partizione esatta dell'intero spazio di Hamming: riempiono tutto il volume disponibile senza sovrapporsi e **senza lasciare alcuno "spazio vuoto"**.

Di conseguenza, *qualsiasi* sequenza binaria y di lunghezza n che il ricevitore possa captare, ricadrà matematicamente dentro una (e una sola) sfera di decodifica. Questo implica che i vettori di errore più pesanti ($wt(e) > t$) non genereranno mai una richiesta di ritrasmissione, ma subiranno inesorabilmente una **decodifica errata certa**.

Esempio: I

Codici di Hamming (che correggono un singolo errore, $t = 1$) sono l'esempio per eccellenza di codici perfetti. Un'altra classe famosa e molto rara in teoria dell'informazione è rappresentata dai *Codici di Golay* (con $n = 23, k = 12, t = 3$). Oltre a questi, esistono solo codici perfetti "banali" (come i codici a ripetizione di lunghezza dispari).

6.12 Codici Ciclici

Dai teoremi sullo *sphere packing* si conclude che per avere un'elevata capacità correttiva ($t \geq 1$) e mantenere un'alta efficienza ($R \cong 1$), è necessario utilizzare codici a blocchi molto lunghi ($n \gg 1, k \gg 1$). Tuttavia, all'aumentare della lunghezza delle parole di codice, la complessità hardware necessaria per codificare e decodificare tramite matrici diventa troppo elevata. Per superare questo ostacolo, si introducono i **Codici Ciclici**, una sottoclasse dei codici binari lineari a blocchi la cui struttura matematica riduce drasticamente la complessità di codifica e decodifica, anche con parole di elevata lunghezza.

6.12.1 Definizione e Rappresentazione Polinomiale

Un codice lineare si definisce **ciclico** se qualsiasi rotazione ciclica (*shift*) di una parola di codice è a sua volta una parola di codice valida. Se il vettore $(x_0, x_1, \ldots, x_{n-1})$ appartiene al codice, allora vi apparterrà anche la sua rotazione verso destra $(x_{n-1}, x_0, \ldots, x_{n-2})$. Un esempio di rotazione a sinistra:

$$x_i, x_{i+1}, \ldots, x_{n-1+i} \in C \implies x_{i+1}, \ldots, x_{n-1+i}, x_i \in C$$

Esempio: Rotazione ciclica

$$1001 \in C \implies \{0011, 0110, 1100\} \in C$$

Tale proprietà permette di *abbandonare* le matrici per definire e operare sui codici, e associare la composizione delle parole di all'anello dei polinomi con coefficienti in $GF(s)$. A ciascuna parola di codice $\vec{x}$ viene associato un polinomio $X(p)$ nell'incognita p, di grado non superiore a $n-1$, in cui i simboli della parola fungono da coefficienti:

$$\vec{x} = (x_0, \ldots, x_{n-1}) \iff X(p) = x_0 + x_1 p + x_2 p^2 + \cdots + x_{n-1} p^{n-1}$$

L'indice i della potenza p^i corrisponde alla posizione i nella parola di codice del coefficiente associato a p^i.

Notazione: $X(p) \in C$ indica che $X(p)$ è il polinomio associato alla parola di codice x, lunga n.

6.12.2 Operazioni su codici ciclici

Le operazioni su questi polinomi seguono regole di un'aritmetica speciale (l'anello dei polinomi modulo $p^n - 1$):

- **Somma:** È la normale somma tra polinomi a coefficienti binari (corrisponde a un semplice XOR bit a bit tra le parole, senza riporti).
- **Prodotto tra polinomi:** La moltiplicazione tra due polinomi si esegue con le regole dell'algebra classica, ma il risultato deve essere ridotto **modulo $p^n - 1$**. In pratica, si divide il risultato per $(p^n - 1)$ e si tiene solo il resto, garantendo che il grado non superi mai $n-1$.
- **Rotazione (Shift ciclico):** Una proprietà fondamentale di questa aritmetica è che moltiplicare un polinomio $X(p)$ per la sola incognita p (sempre modulo $p^n - 1$) corrisponde matematicamente a effettuare una **rotazione ciclica di una posizione** verso destra della parola di codice originale.

Traslare di i posizioni a destra equivale a moltiplicare per $p^i \pmod{d(p)}$

Nota algebrica sulla rotazione. Per dimostrare che moltiplicare per p equivale a ruotare, calcoliamo la differenza tra il polinomio moltiplicato $pX(p)$ e il polinomio della parola ruotata "a mano" $X'(p)$. Sottraendoli, i termini intermedi si annullano:

$$pX(p) - X'(p) = x_{n-1}p^n - x_{n-1} = x_{n-1}(p^n - 1)$$

Poiché la loro differenza è un multiplo esatto di $(p^n - 1)$, nell'aritmetica modulare essa equivale a zero:

$$pX(p) - X'(p) \equiv 0 \pmod{p^n - 1}$$

Di conseguenza, portando $X'(p)$ a destra dell'uguale, si dimostra che i due polinomi coincidono:

$$pX(p) \equiv X'(p) \pmod{p^n - 1}$$

Esempio: I

l codice $C = \{000, 110, 101, 011\}$ è ciclico.

$$x_1 = [000] \quad x_2 = [110] \quad x_3 = [101] \quad x_4 = [011]$$

Ad esempio x_4 si ottiene con due scorrimenti a destra da x_3. Passando ai polinomi, moltiplichiamo $X_3(p)$ per p^2:

$$p^2 \cdot X_3(p) = p^2 \cdot (1 + p^2) = p^2 + p^4$$

Calcolando il risultato modulo $d(p) = p^3 + 1$ (ricordando che $p^3 \equiv 1$), si ottiene:

$$p^2 + p^4 = p^2 + p \cdot p^3 \equiv p^2 + p \pmod{p^3 + 1}$$

Il polinomio risultante $p + p^2$ corrisponde esattamente alla parola $x_4 = [011]$.

6.12.3 Polinomio generatore

Enunciamo ora un teorema che lega i polinomi alle codeword dei codici lineari ciclici.

Sia $G(p) \in C$ **IL polinomio monico di grado minimo** q. Ogni polinomio (parola) $X(p) \in C$ è **multiplo** di $G(p)$.

Dimostrazione: Dall'algoritmo di Euclide, esistono un polinomio quoziente $Q(p)$ e un resto $R(p)$ tali che:

$$X(p) = Q(p)G(p) + R(p) \implies \text{con il grado del resto minore di quello del } \textbf{divisore } G(p)$$

Poiché $G(p) \in C$ e il codice è lineare, anche qualsiasi sua rotazione/moltiplicazione lo è, quindi $Q(p)G(p) \in C$. Poiché la differenza di due parole appartiene al codice, si ha che $X(p) - Q(p)G(p) \in C$. Questo implica che anche $R(p) \in C$. Tuttavia, $R(p)$ ha un grado strettamente inferiore a $G(p)$, che per definizione è il minimo possibile per il codice. L'unica soluzione accettabile è che $R(p) = 0$ (polinomio nullo). Dunque $X(p) = Q(p)G(p)$, dimostrando che $X(p)$ è un multiplo esatto.

6.12.4 Caratterizzazione con i polinomi

Per definire un codice ciclico di lunghezza n, è quindi sufficiente individuare un **qualunque** polinomio generatore $G(p)$ che sia un **divisore di** $p^n - 1$.

Le tre regole fondamentali per la caratterizzazione sono:

- Ogni polinomio $G(p)$ di grado q che sia un **divisore esatto** di $(p^n - 1)$ genera univocamente un codice ciclico valido di lunghezza n.
- La dimensione del codice generato dipenderà direttamente dal grado del divisore scelto: si avranno q cifre di controllo e $k = n - q$ cifre di informazione.
- L'operazione di codifica si riduce a un prodotto polinomiale: la parola di codice $X(p)$ si ottiene moltiplicando il messaggio $M(p)$ per il generatore $G(p)$.

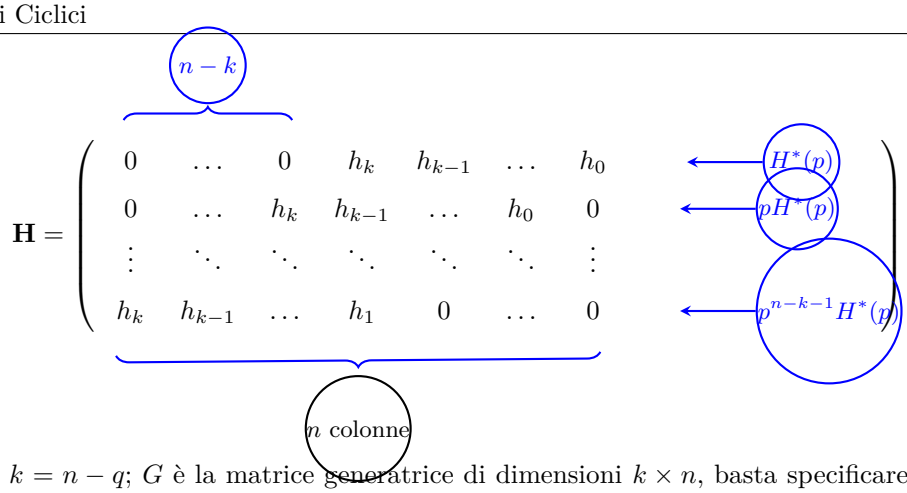
Esempio: Codice ciclico binario di 5 bit

Tutti i possibili codici ciclici con parole di 5 bit sono ricavabili dalla fattorizzazione di $p^5 - 1 = (1 + p)(1 + p + p^2 + p^3 + p^4)$

6.12.5 Matrice Generatrice

Da queste considerazioni, passiamo a ricavare la matrice generatrice G di un codice ciclico (di dimensione $k \times n$) a partire dal polinomio generatore $G(p)$ tramite le sue rotazioni cicliche $(pG(p), p^2G(p), \dots)$.

$$G(p) = g_0 + g_1 p + g_2 p^2 + \cdots + g_q p^q \quad g_q = 1$$



Poniamo $k = n - q$; G è la matrice generatrice di dimensioni $k \times n$, basta specificare il polinomio generatore invece che tutti i $k \times n$ elementi della matrice!

Esempio di Matrice Generatrice per C(7,4). Abbiamo il codice binario ciclico $C(7,4)$, e come polinomio generatore si è scelto $G(p) = 1 + p + p^3$, la matrice generatrice sarà quindi:

$$G = \begin{pmatrix} 1 & 1 & 0 & 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 1 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 & 1 & 0 & 1 \end{pmatrix}$$

La **codifica** di un messaggio $M = (m_0, m_1, \ldots, m_{k-1})$ nella parola di codice $c = MG$, equivale a codificare il messaggio *polinomiale* $M(p) = \sum_{i=0}^{k} m_i p^i$ (i primi k bit della codeword che corrispondono ai bit di informazione) nel codice polinomiale $c(p) = M(p)G(p)$

6.12.6 Scelta del polinomio generatore

Un codice binario ciclico $C(n,k)$ è definito dal polinomio generatore nella forma

$$G(p) = p^{n-k} + g_{n-k-1}p^{n-k-1} + \cdots + g_1 p + g_0$$

I coefficienti $g_i \in GF(2)$ sono tali per cui $G(p)$ sia un fattore del polinomio $p^n - 1$, e, il grado minimo $q = n - k$ del polinomio indica quante cifre di controllo sono presenti. Conseguentemente, **tutti gli altri** $2^k - 1$ polinomi di codice sono **multipli** di $G(p)$. Ogni polinomio di grado uguale o inferiore a $n - 1$, divisibile per $G(p)$ deve essere un polinomio di codice. Quindi, le parole di codice sono tante quante i polinomi di grado minore di $k = n - q$.

Il polinomio generatore di un codice ciclico è il polinomio di codice con **grado minimo**, ad esclusione del polinomio nullo.

6.12.7 Polinomio di controllo

Partendo dal polinomio generatore $G(p)$, come possiamo determinare un polinomio di controllo H? Partendo dalla proprietà fondamentale di $G(p)$, cioè che si tratta di un polinomio di grado q divisore di $p^n - 1$, questo fatto implica automaticamente l'esistenza di un altro polinomio univoco per quel codice, chiamato **polinomio di controllo parità** $h(p)$.

Definiamo $H(p)$ il quoziente della divisione esatta tra $p^n - 1$ e il polinomio generatore $g(p)$. L'equazione fondamentale che lega i due polinomi è quindi:

$$H(p) \cdot G(p) = p^n - 1 = 0$$

Per qualsiasi codeword $X(p) \in C$, si ha che $X(p) = G(p)b(p)$ per qualche $b(p)$. Segue che:

$$H(p) \cdot X(p) = H(p) \cdot G(p)b(p) = 0 \quad \text{con } H(p) \cdot G(p) = 0$$

L'equazione $H(p) \cdot G(p) = 0$ svolge il ruolo di **equazione di controllo**, con $H(p)$ (che non è unico) detto quindi **polinomio di controllo**.

Sia $G(p)$ polinomio generatore e $H(p)$ **un** polinomio di controllo, si ha che:

$$pH(p) * G(p) = H(p) * \underbrace{pG(p)}_{\in C} = 0$$
$$p^2H(p) * G(p) = H(p) * \underbrace{p^2G(p)}_{\in C} = 0$$
$$\vdots$$
$$p^{n-k-1}H(p) * G(p) = H(p) * \underbrace{p^{n-k-1}G(p)}_{\in C} = 0$$

Quindi, ogni permutazione ciclica di $H(p)$ è ancora un polinomio di controllo!

6.12.8 Matrice di controllo e Codice Duale

Mentre il polinomio generatore $g(p)$ definisce il codice C, il polinomio di controllo $h(p) = (p^n - 1)/g(p)$ è legato al **codice duale** $C^\perp$.

Tuttavia, i coefficienti di $h(p)$ non possono essere usati *direttamente* nello stesso ordine per costruire le righe della matrice di controllo H. Affinché H definisca correttamente lo spazio nullo di G, le sue righe devono essere **ortogonali** a quelle di G (ovvero deve valere $G \cdot H^T = 0$).

A causa del modo in cui operano le rotazioni cicliche, per far sì che i termini si annullino a vicenda durante il prodotto scalare, è necessario *invertire* l'ordine dei coefficienti di $h(p)$. Il polinomio ottenuto da questa inversione prende il nome di **Polinomio Reciproco** $h^*(p)$, ed è definito matematicamente come:

$$H^*(p) = p^k H\left(\frac{1}{p}\right)$$

La matrice di controllo H si costruisce quindi inserendo sulle righe le traslazioni cicliche dei coefficienti del polinomio reciproco $H^*(p)$.

La matrice di controllo H è quindi data da:

$$\mathbf{H} = \overbrace{\begin{pmatrix} 0 & \dots & 0 & h_k & h_{k-1} & \dots & h_0 \\ 0 & \dots & h_k & h_{k-1} & \dots & h_0 & 0 \\ \vdots & \ddots & \ddots & \ddots & \ddots & \ddots & \vdots \\ h_k & h_{k-1} & \dots & h_1 & 0 & \dots & 0 \end{pmatrix}}^{n-k}$$

$H^*(p)$

$pH^*(p)$

$p^{n-k-1}H^*(p)$

Esempio: Codice ciclico

Sia $\mathcal{A} = \{0, 1\}, n = 5$. Abbiamo già visto come per codici con parole da 5 bit, sono possibili tre codici ciclici con i seguenti polinomi generatori:

1. $$G(p) = 1$$
   Composto da tutte le parole di 5 bit, che saranno tutti bit di informazione e quindi 0 bit di ridondanza.
2. $$G(p) = 1 + p$$
   Questo codice ha $k = 4$ bit di informazione e una sola cifra di controllo. Le parole saranno quindi $2^k = 2^4 = 16$:
   $$\mathcal{C} = \begin{Bmatrix} 00000, 11000, 01100, 00110, 00011, 10001, 10100, 01010, \\ 00101, 10010, 01001, 11110, 01111, 10111, 11011, 11101 \end{Bmatrix}$$

$$\mathbf{G} = \begin{pmatrix} 1 & 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 0 & 0 \\ 0 & 0 & 1 & 1 & 0 \\ 0 & 0 & 0 & 1 & 1 \end{pmatrix}$$

$$H(p) * G(p) = p^5 + 1$$
$$H(p) = p^4 + p^3 + p^2 + p + 1$$
$$H^*(p) = p^4 H\left(\frac{1}{p}\right) = H(p)$$
$$\mathbf{H} = \begin{pmatrix} 1 & 1 & 1 & 1 & 1 \end{pmatrix}$$

3.

$$G(p) = 1 + p + p^2 + p^3 + p^4$$

$$H(p) = 1 + p \qquad H^*(p) = pH\left(\frac{1}{p}\right) = 1 + p$$

$$\mathbf{G} = \begin{pmatrix} 1 & 1 & 1 & 1 & 1 \end{pmatrix} \qquad \mathbf{H} = \begin{pmatrix} 0 & 0 & 0 & 1 & 1 \\ 0 & 0 & 1 & 1 & 0 \\ 0 & 1 & 1 & 0 & 0 \\ 1 & 1 & 0 & 0 & 0 \end{pmatrix}$$

6.12.9 Codici ciclici Sistematici

In generale i codici ciclici possono essere sia sistematici che non, ma concentreremo l'analisi sui codici sistematici.

Considerando un generico codice ciclico, definiamo i polinomi

- $M(p) = m_{k-1}p^{k-1} + \cdots + m_1 p + m_o$: il polinomio che identifica i bit di informazione.
- $C(p) = c_{q-1}p^{q-1} + \cdots + c_1 p + c_0$: il polinomio che identifica i bit di controllo.

Per avere un codice **sistematico**, la parola di codice $X(p)$ generata a partire da $M(p)$, sarà espressa come :

$$X(p) = M(p) + p^k C(p)$$

Esisterà quindi un polinomio quoziente $Q_M(p)$ tale che:

$$X(p) = Q_M(p)G(p) = M(p) + p^k C(p)$$

Da ciò si ricava che:

$$\frac{p^{n-k}M(p) + C(p)}{G(p)} = p^{n-k}Q_M(p) \Rightarrow \frac{p^{n-k}M(p)}{G(p)} = Q'_{M(p)} + \frac{C(p)}{G(p)}$$

Il polinomio dei bit di controllo si può quindi calcolare come **resto** della divisione di $p^{n-k}M(p)$ con $G(p)$:

$$C(p) = [p^{n-k}M(p)] \pmod{G(p)}$$

6.12.10 Codici Ciclici binari - Algoritmi di Codifica

Primo algoritmo di codifica (NON sistematico)

Per un codice ciclico (n, k), il polinomio di codice $X(p)$ per un messaggio $M(p)$, può essere ottenuto moltiplicando quest'ultimo per il polinomio generatore $G(p)$.

$$X(p) = M(p)G(p)$$

Ogni fattore di $(p^n + 1)$ di grado $q = n - k$ è un polinomio generatore *valido*, ma non tutti i fattori genereranno un *buon codice*. Questo algoritmo è più computazionalmente più semplice (facendo uso di operazioni su polinomi invece che matrici), ma non identifica un codificatore sistematico.

Partendo da un messaggio $M = (m_0, m_1, \ldots, m_{k-1})$ si associa il polinomio

$$m_0 + m_1 p + \cdots + m_{k-1}p^{k-1}$$

Quest'ultimo verrà codificato con la parola $X(p)$:

$$X(p) = M(p)G(p)$$

Si potrà quindi utilizzare il polinomio di controllo $H(p)$ per verificare se una parola ricevuta è una parola di codice

$$X(p)H(p) = 0 \implies X \in C$$

I codici ciclici più usati sono i codici di Hamming, i codici di Golay, i codici BCH. Il codice di Hamming $(7, 4)$ è ciclico:
Per la parola di codice 0111010, ogni sua rotazione *a sinistra* (1110100, 1101001, 1010011, 0100111, 1001110, 0011101) fa parte del codice, e questa proprietà vale per tutte le codeword.

Esempio: Codice non sistematico

Consideriamo un polinomio generatore $G(p) = 1 + p^2 + p^3$ e un messaggio $M = 1101$. Il polinomio associato al messaggio è $M(p) = 1 + p + p^3$, pertanto la parola codificata $X(p)$ risulta:

$$X(p) = (p^3 + p^2 + 1)(p^3 + p + 1)$$

$$X(p) = p^6 + p^4 + p^3 + p^5 + p^3 + p^2 + p^3 + p + 1 = p^6 + p^5 + p^4 + p^3 + p^2 + p + 1$$

Si fa notare che la codeword così ottenuta è 1111111, quindi il codificatore **non** è sistematico.

Secondo algoritmo di codifica

Si può modificare la codifica basata sulla operazione $M(p)G(p)$ per ottenere un codice ciclico **sistematico**.
Si moltiplica il messaggio $M(p)$ per p^{n-k} e si divide il prodotto per il generatore $G(p)$

$$p^{n-k}M(p) = Q(p)G(p) + R(p)$$

Il polinomio resto $R(p)$ avrà grado **non maggiore** di $n - k - 1$, in quanto il polinomio generatore è di grado $n - k$. Si ricava quindi:

$$p^{n-k}M(p) + R(p) = Q(p)G(p)$$

Essendo il secondo membro un polinomio di grado minore o uguale a $(n - 1)$ e **multiplo** del polinomio generatore, allora il primo membro identifica un **polinomio di codice** $X(p)$. In forma esplicita:

$$p^{n-k}(m_0 + m_1 p + \cdots + m_{k-1}p^{k-1}) + r_0 + \cdots + r_{n-k-1}p^{n-k-1} \in C$$

$$\underbrace{(m_0 + m_1 p + \cdots + m_{k-1}p^{k-1})}_{\text{k bit informativi}} + \underbrace{r_0 p^k + \cdots + r_{n-k-1}p^{n-1}}_{\text{n-k bit controllo parità}} \in C$$

In questo modo il codice risulta sistematico!

La sezione di controllo di parità di ogni codeword di un codice ciclico sistematico (n, k) si ottiene come **resto** della divisione

$$\frac{p^{n-k}M(p)}{G(p)}$$

Riassumiamo l'algoritmo di codifica per ottenere un codice ciclico binario sistematico:

1. **Si moltiplica** il messaggio: $M(p)p^{n-k}$
2. **Si divide** $M(p)p^{n-k}$ per $G(p)$, ottenendo il resto $R(p)$
3. **Si somma** $M(p)$ con il resto $R(p)$ moltiplicato per p^k e si ottiene quindi il polinomio di codice $X(p)$

$$X(p) = M(p) + p^k R(p), \quad R(p) = [p^{n-k}M(p)] \pmod{G(p)}$$

Esempio: Algoritmo di codifica - Hamming(7,4)

Dato il codice di Hamming$(7,4)$ ciclico, con polinomio generatore $G(p) = 1 + p^2 + p^3$. Si vuole codificare il messaggio 1101 con l'algoritmo di codifica sistematico, con $q = n - k = 3$.

1. **Moltiplicazione:**
$$M(p) = 1 + p + p^3 \Rightarrow p^3 M(p) = p^3 + p^4 + p^6$$
2. **divisione per** $G(p)$

p^6		$+p^4$	$+p^3$	$p^3 + p^2 + 1$
				$p^3 + p^2$
p^6	$+p^5$		$+p^3$	
	$+p^5$	$+p^4$	$+p^3$	
	$+p^5$	$+p^4$	$+p^2$	
			$+p^2$ → R(p)	

3. **Si somma**: $M(p) + p^k R(p)$
$$X_M(p) = M(p) + p^4 R(p) = 1 + p + p^3 + p^6$$
$$X = 1101001$$

6.12.11 Codici Ciclici - Algoritmi di decodifica e rilevazione di errore

Ad ogni messaggio inviato X, associamo il polinomio $Y(p)$ associato al vettore ricevuto al canale. Per individuare un eventuale errore sarà necessario calcolare il polinomio **sindrome** $s(p)$ di grado minore di $(n-k)$ associato a $Y(p)$. La sindrome si ottiene come **resto** della divisione polinomiale tra $Y(p)$ e $G(p)$.

$$Y(p) = Q(p)G(p) + s(p)$$

Solamente in caso di **sindrome nulla**, la parola di codice è valida, in quanto il polinomio generatore $G(p)$ è un **fattore** di $Y(p)$.

Ciò è conseguenza del seguente teorema:

Teorema: Sia C un codice ciclico (n,k), con polinomio di controllo di parità $H(p)$. Un polinomio $Y(p)$ è un polinomio di codice se e soltanto se:

$$Y(p)H(p) \equiv 0 \pmod{p^n - 1}$$

Per un codice (n,k) e capacità correttiva pari a t, il vettore di errore $E(p)$ sarà tale che:

$$X(p) = Y(p) + E(p)$$

Teorema: Se la sindrome $S(p)$, calcolata rispetto al polinomio $Y(p)$, ha **peso** $\leq t$, allora il vettore di errore coincide con la sindrome $E(p) = S(p)$.

Questa conseguenza è applicabile per la correzione **solamente se gli errori sono presenti nei primi** q **bit del messaggio**. Si può però *shiftare ciclicamente* $Y(p)$ fino ad ottenere, dopo m shift, una sindrome $S_m(p)$ di peso minore o uguale a t. Il *nuovo* vettore di errore $E(p)$ sarà quindi pari alla sindrome shiftata di m posizioni verso destra:

$$E(p) = p^m S_m(p) \pmod{(p^n - 1)}$$

Esempio: Algoritmo di decodifica

Codice di Hamming ciclico $(7,4)$, polinomio generatore $G(p) = 1 + p^2 + p^3$.

- $t = 1$, si riceve la sequenza $Y = 1100001$, polinomio associato $Y(p) = 1 + p + p^6$.
- Polinomio sindrome $S(p) = p^2 + 1$ con peso $2 > t$.
- Si opera uno shift a sinistra di una posizione $m = 1$

$$Y' = 1000011 \implies Y'(p) = 1 + p^5 + p^6$$

- Il *nuovo* polinomio sindrome è $S_1(p) = p^2$ ha peso $1 = t$, e dunque:

$$E(p) = p^m S_1(p) = p^3$$

$$X(p) = Y(p) + E(p) = 1 + p + p^3 + p^6$$

Codici BCH

I codici **BCH** (Bose – Chaudhuri – Hocquenghem) sono codici ciclici, *non necessariamente binari*, che costituiscono una generalizzazione dei codici di Hamming con $t \geq 1$. Per $m \geq 3$ *intero*, esiste un codice BCH binario con parametri:

$$n = 2^m - 1, \quad n - k \leq mt, \quad d_{min} \geq 2t + 1$$

Può correggere t errori. Il corrispondente polinomio generatore si può costruire dai fattori di:

$$p^n + 1 = p^{2^m - 1} + 1$$

Esempi di polinomi generatori

Codice	n	k	R_c	d_{min}	$G(p)$
Hamming	7	4	0.57	3	1 011
	15	11	0.73	3	10 011
	31	26	0.84	3	100 101
BCH	15	7	0.46	5	111 010 001
	31	21	0.68	5	11 101 101 001
	63	45	0.71	7	1 111 000 001 011 001 111
Golay	23	12	0.52	7	101 011 100 011

Esempio di codice ciclico binario

Si considera un codice ciclico binario di lunghezza 7.

$$x^7 - 1 = (x - 1)(x^3 + x + 1)(x^3 + x^2 + 1)$$

Lavorando con codici binari, i segni "+" e "−" sono intercambiabili:

$$x^7 + 1 = (x + 1)(x^3 + x + 1)(x^3 + x^2 + 1)$$

Abbiamo quindi tre fattori irriducibili e, di conseguenza, $2^3 = 8$ codici ciclici (inclusi $g(x) = 1$ e $g(x) = x^7 + 1$):

$$\begin{aligned}
&(i) && 1 = 1 \\
&(ii) && x + 1 = x + 1 \\
&(iii) && x^3 + x + 1 = x^3 + x + 1 \\
&(iv) && x^3 + x^2 + 1 = x^3 + x^2 + 1 \\
&(v) && (x + 1)(x^3 + x + 1) = x^4 + x^3 + x^2 + 1 \\
&(vi) && (x + 1)(x^3 + x^2 + 1) = x^4 + x^2 + x + 1 \\
&(vii) && (x^3 + x + 1)(x^3 + x^2 + 1) = \\
& && \quad x^6 + x^5 + x^4 + x^3 + x^2 + x + 1 \\
&(viii) && (x + 1)(x^3 + x + 1)(x^3 + x^2 + 1) = x^7 + 1
\end{aligned}$$

- (i) $g(x) = 1$ genera tutte le parole di lunghezza 7.
- (ii) $g(x) = 1 + x$ genera un codice di controllo con una cifra di parità.
- (vii) $g(x) = 1 + x + x^2 + x^3 + x^4 + x^5 + x^6$ genera un codice a ripetizione con $k = 1$ (un solo bit di informazione ripetuto 7 volte)
- $(viii)$ $g(x) = 1 + x^7$ si ha il codice 0
- (iii) $g(x) = x^3 + x + 1$ ha grado 3, può generare codici di Hamming$(7, 4)$.
- (iv) $g(x) = x^3 + x^2 + 1$ ha grado 3, può generare codici di Hamming$(7, 4)$.
- (v) $g(x) = (x + 1)(x^3 + x + 1) = x^4 + x^3 + x^2 + 1$ genera il codice duale di Hamming.
- (vi) $g(x) = (x + 1)(x^3 + x^2 + 1) = x^4 + x^2 + x + 1$ genera il codice duale di Hamming.

6.12.12 Vantaggi dei codici ciclici

Sono tre i vantaggi garantiti dall'impiego di codici ciclici:

1. semplicità di codifica;
2. semplicità nel calcolo della sindrome;
3. una struttura matematica precisa che consente di sviluppare metodi di decodifica molto efficienti, in quanto non è più necessario *memorizzare informazione*; è quindi possibile implementare codici con

$$n >> 1, k >> 1 \implies R_c \approxeq 1$$

Osservazioni

- Se il polinomio $G(p)$ divide sia $(p^m + 1)$ che $(p^n + 1)$, con $m < n$, allora $(p^m + 1)$ è una codeword generata dal polinomio generatore.
- $p^m + 1$ è il polinomio della codeword di peso 2 per il codice (n, k), il quale ha quindi di conseguenza distanza minima $d_{min} \leq 2$
- Conseguentemente tale codice potrà rilevare **al massimo** un singolo errore
- Per aggirare questa limitazione il parametro n deve essere il più piccolo intero tale che $(p^n + 1)$ è **multiplo** di $G(p)$.

Capitolo 7

Codici Convoluzionali

Così come i codici a blocchi, i codici convoluzionali nascono per fronteggiare le condizioni avverse dei canali trasmissivi, introducendo un'opportuna ridondanza in grado di proteggere l'informazione dagli errori.

Tuttavia, mentre i codici a blocchi frammentano il flusso dei dati in segmenti rigidi e indipendenti (ogni parola di k bit genera una codeword di n bit senza alcun legame con i blocchi adiacenti), i codici convoluzionali operano in modo profondamente diverso.

Sebbene siano anch'essi codici *lineari* (strutturati ad albero), il loro codificatore è dotato di ***memoria***. È proprio grazie a questa memoria che l'operazione di **convoluzione** può "sparpagliare" il flusso di dati: la stringa in uscita in un dato istante non dipende solo dai bit attualmente in ingresso, ma è influenzata anche dallo storico dei bit precedenti. Questo meccanismo produce un flusso continuo in cui informazione e bit di controllo risultano indissolubilmente mescolati.

7.1 Principi e Struttura dei Codici Convoluzionali

7.1.1 L'Operazione di Convoluzione e la Memoria

Il termine *convoluzionale* deriva dall'omonima operazione matematica ed è il meccanismo centrale tramite il quale questi codici producono l'uscita. Prendendo spunto dall'etimologia della parola, **avvolgere insieme** è difatti ciò che fanno questi sistemi: aggiungono ridondanza e la intrecciano intimamente con i dati da codificare. Così come in matematica la convoluzione "fonde" due funzioni facendone scorrere una sull'altra, qui il flusso continuo dei dati scorre e si "fonde" con la struttura logica del codificatore.

Da un punto di vista implementativo, se consideriamo u come il flusso di dati in ingresso e X come il flusso codificato in uscita, il processo avviene in modo continuo attraverso questi passaggi:

- **Ingresso:** Dal flusso u vengono letti k bit per volta.
- **Scorrimento (Memoria):** Questi bit entrano in un **registro a scorrimento** (*shift register*). All'interno di questo registro sono ancora trattenuti i bit elaborati negli istanti precedenti, i quali implementano fisicamente la *memoria* del codificatore.
- **Combinazione:** Una **funzione combinatoria** (tipicamente una rete di porte logiche XOR) collegata ai vari stadi del registro va a mescolare i nuovi k bit appena entrati con quelli vecchi. Questa operazione produce infine gli n bit in uscita, che andranno a formare il flusso codificato X da trasmettere sul canale.

7.1.2 Parametri e notazioni

Elenchiamo parametri e notazioni per definire compiutamente un codificatore **binario convoluzionale** (n, k)

- **Data-frame (k):** Il gruppo di k bit di informazione che entra nel codificatore in un singolo istante di tempo.
- **Code-frame (n):** Il gruppo di n bit prodotto in uscita dalla rete logica in ogni istante. Per introdurre ridondanza, si ha sempre $n > k$.

- **Frequenza di codifica ($R_c = \frac{k}{n}$):** Detta anche *Code Rate*, indica l'efficienza del codice ed è definita dal rapporto tra i bit di informazione e i bit totali del code-frame
- **Lunghezza di vincolo (N):** Indica quanti data-frame influenzano la generazione del code-frame. Rappresenta l'ampiezza della "finestra temporale" in cui un bit in ingresso continua a produrre effetti sull'uscita.
- **Memoria del codificatore $v = (N-1)k$:** Rappresenta il numero totale di bit *passati* memorizzati nei registri a scorrimento. Poiché il vincolo totale è N, gli stadi puramente di memoria sono $N-1$.

7.1.3 Codificatore Convoluzionale

Un codificatore convoluzionale può quindi essere progettato come una **macchina sequenziale a stati**, con registri a scorrimento con Nk posizioni, raggruppati in N stadi, ciascuno di k elementi. A loro volta i registri sono collegati a un circuito combinatorio collegato al registro di uscita; insieme al vincolo, il modo in cui è progettato il circuito determina il comportamento del codificatore.

Per determinare il funzionamento del codificatore, faremo uso dei *generatori*.

7.1.4 Sequenze Generatrici

Mentre nei codici lineari a blocchi, la codifica è determinata da matrici generatrici o da polinomi generatori, nei codici convoluzionali la codifica è specificata dalle **sequenze generatrici** (*generatori*).

Per determinare il comportamento di un codice convoluzionale (n, k, N) ci si affida a nk *vettori di bit*, di lunghezza N, che identificano matematicamente (e *fisicamente* identificando quali commutatori attivare) il codificatore. Definiscono in modo univoco il legame tra il flusso di dati in ingresso e i bit prodotti in uscita.

Sia $[m_1, m_2, m_3, \dots]$ il flusso di messaggi in ingresso al codificatore.

I generatori relativi all'uscita i-esima del codificatore sono indicati con:

$$g_{ij} = [g_{ij}^1, g_{ij}^2, \dots, g_{ij}^N] \quad i = 1, \dots, n \quad j = 1, \dots, k$$

Sia $u_j^{(t)}$ il j-esimo ingresso al tempo t, con $j = 1, \dots, k$

Sia $x_i^{(t)}$ la i-esima uscita al tempo t, con $i = 1, \dots, n$

$$x_i^{(t)} = \sum_{j=1}^{k} \sum_{l=1}^{N} g_{ij}^l u_j^{(t-l)} \quad i = 1, \dots, n$$

Per un codificatore con vincolo di dimensione N, i generatori sono definiti dalle n definizioni:

$$g_i = [g_i^1, g_i^2, \dots, g_i^N] \quad i = 1, \dots, n$$

Il valore del coefficiente g_i^l è pari a 1 quando lo stadio l-esimo del registro in ingresso è ***connesso*** al sommatore, la cui uscita è a sua volta connessa allo stadio i-esimo del registro in uscita. La sequenza in uscita all'i-esimo sommatore (assumendo $k = 1$ singolo bit in ingresso), è quindi data dalla convoluzione:

$$x_i^{(t)} = \sum_{l=1}^{N} g_i^l u^{(t-l)} \quad i = 1, \dots, n$$

7.1.5 Rappresentazioni Grafiche

Si possono usare diverse rappresentazioni grafiche per rappresentare un codificatore convoluzionale:

- rappresentazione *ad albero*
- rappresentazione *a traliccio*
- diagramma degli stati

Diagramma a Stati

Il diagramma degli stati è un **grafo** composto da $2^v = 2^{k(N-1)}$ nodi, ognuno dei quali è uno dei possibili stati in cui può trovarsi il codificatore.
Con il grafo si può modellare completamente il comportamento di un codice convoluzionale.
Ogni stato è identificato dagli i bit nei registri a scorrimento, e dai k bit in ingresso, che di concerto determinano gli n bit in uscita.
I nodi sono connessi tra loro tramite archi, ognuno dei quali identifica un data-frame in ingresso. Per data-frame di dimensione k, si avranno 2^k rami uscenti per ogni nodo
La transizione da uno stato ad un altro è associata alla emissione di un code-frame di n bit.

Diagramma a Traliccio (Trellis)

Il diagramma **a traliccio** può essere considerato come un'estensione *temporale* di un diagramma a stati; aggiunge infatti un indice temporale i. È uno strumento utile per analizzare la decodifica.
Per ogni indice temporale, il numero degli stati è determinato dalla memoria v del codificatore; gli stati saranno quindi $2^v = 2^{k(N-1)}$.
Nel passaggio da un indice temporale al successivo, gli stati sono connessi tramite gli archi delle possibili transizioni di stato, ognuna delle quali è determinata da una possibile sequenza in ingresso.

Esempio: Codice Convoluzionale $(2, 1, 3)$

Consideriamo un codificatore caratterizzato da $n = 2$, $k = 1$, $N = 3$. I due generatori associati alle uscite sono $g_1 = [110]$ e $g_2 = [011]$. Le equazioni per i due bit di uscita all'istante i risultano:

$$x_1 = u_i + u_{i-1}$$
$$x_2 = u_{i-1} + u_{i-2}$$

Lo stato del sistema è determinato dagli ultimi due bit in ingresso memorizzati nei registri (u_{i-1}, u_{i-2}). Al variare dell'ingresso u_i, la macchina transita tra i 4 stati possibili $(00, 10, 01, 11)$, emettendo i relativi bit di uscita x_1x_2 e tracciando un percorso specifico sul diagramma a stati e sul traliccio.

7.1.6 Capacità Correttiva: Distanze del Codice

Le capacità di rivelazione e correzione degli errori di un codice convoluzionale sono vincolate alla stessa metrica fondamentale utilizzata per i codici a blocchi: la **distanza di Hamming**.

Tuttavia, emerge una differenza sostanziale nel modo in cui questa metrica viene valutata. Non elaborando *singole codeword* isolate, ma operando su *flussi continui*, è necessario considerare la distanza tra le **intere sequenze** codificate X, generate a partire dai diversi flussi di informazione in ingresso.

Poiché i codici convoluzionali mantengono la proprietà di **linearità,** calcolare la distanza minima tra due sequenze valide qualsiasi equivale semplicemente a valutare il **peso di Hamming** dell'intera sequenza (ovvero la sua distanza rispetto alla sequenza nulla composta da soli zeri). Non cambia quindi il paradigma che guida la decodifica, il criterio di **Massima Verosimiglianza** (*Maximum Likelihood*).

Si introducono due metriche specifiche per i codici convoluzionali, la distanza colonna e la distanza libera.

Distanza Colonna

Consideriamo due sequenze codificate fino all'istante i in cui divergono alla prima diramazione del traliccio.

La distanza colonna $d_c(i)$ è la misura della minima distanza di Hamming tra **tutte le coppie** di sequenze di questo tipo.

Per calcolare $d_c(i)$ si considerano le distanze rispetto alla **sequenza di tutti zeri**. In questo modo, sfruttando la linearità del codice, la distanza colonna coincide con il **peso minimo** delle sequenze di codice che divergono alla prima ramificazione dalla *sequenza zero.*

La $d_c(i)$ è una funzione **non decrescente** della profondità i, ovvero:

$$d_c(i) \leq d_c(i+1)$$

L'apparente limitazione di valutare la distanza rispetto alla sola **sequenza nulla** (tutti zeri) è giustificata dalla **linearità** del codice. Poiché la distanza di Hamming tra due sequenze codificate A e B

è identica al peso di Hamming della loro somma modulo-2 ($A \oplus B$), e dato che in un codice lineare tale somma è anch'essa una sequenza valida, la sequenza di tutti zeri funge da **riferimento universale**.

Trovare il percorso che diverge e accumula il minor peso rispetto allo zero equivale a trovare la distanza minima tra **qualsiasi** coppia di sequenze del codice, senza dover confrontare ogni possibile combinazione di messaggi.

Distanza Libera

Estendendo l'analisi della distanza colonna verso una profondità infinita, definiamo la **distanza libera** come il valore limite:

$$d_l = \lim_{i \to \infty} d_c(i)$$

Rappresenta la minima distanza di Hamming tra due sequenze codificate di lunghezza arbitraria (*infinitamente lunghe*). Grazie alla linearità, d_l coincide con il peso minimo di un percorso che:

1. Diverge dal percorso nullo (stato tutto zero) al primo istante.
2. Attraversa il traliccio *accumulando peso*.
3. Si ricongiunge *alla fine* allo stato nullo iniziale.

Il calcolo della distanza libera può quindi essere effettuato senza dover considerare il peso w di tutte le possibili sequenze $w(X)$:

Si identifica sul traliccio il percorso a **peso minimo** che si *separa* dal percorso *nullo*, per poi ricongiungersi, *stabilmente* con quest'ultimo.

Come già detto, d_l sarà il **minimo peso** di questo insieme di sequenze.

Operativamente si suppone di terminare una sequenza con una serie di bit 0 in modo da riportare il registro a scorrimento al suo stato iniziale, composto di tutti 0. In questo modo si eliminano alcuni rami dalle L transizioni finali nel traliccio del codice.

In fase di progettazione, fissati il rapporto di codifica $R_c = k/n$ e la lunghezza del vincolo N, il codice **ottimo** è quello che garantisce la **massima distanza libera** possibile, minimizzando così la probabilità di errore residua dopo la decodifica.

7.1.7 Funzione Generatrice

Mentre la distanza libera indica il *caso peggiore*, si introduce la **funzione generatrice** per fornire l'intera distribuzione dei pesi del codice. Essa può essere vista come la funzione *di trasferimento* del sistema, valutata rispetto alle transizioni tra gli stati del codificatore.

La funzione generatrice fornisce indicazioni fondamentali sia sulla distanza libera, sia sulle probabilità di errore che si possono incontrare in fase di decodifica.

7.2 Decodifica dei Codici Convoluzionali

Sono tre i metodi di decodifica principali per i codici convoluzionali:

- **Algoritmo di Viterbi:** implementa una decodifica ottima a massima verosimiglianza, richiede hardware *complesso*;
- **Decodifica Sequenziale:** realizza un compromesso tra complessità implementativa e prestazioni della decodifica;
- **Decodifica a Retroazione:** decodifica che non richiede hardware *complesso*, ma offre prestazioni inferiori;

7.2.1 Metriche per la Decodifica a Massima Verosimiglianza su Traliccio

Similmente ai codici lineari a blocchi, anche per i codici convoluzionali il criterio di **massima verosimiglianza** viene espletato selezionando di volta in volta la sequenza di codice a **distanza minima**.

Essendo la *possibile sequenza codificata* un cammino sul traliccio del codice, il decodificatore individua il cammino a distanza minima dalla sequenza ricevuta.

Si possono utilizzare due metriche:

- **Distanza di Hamming:** nel caso di decodifica *hard*;
- **Distanza Euclidea:** nel caso di decodifica *soft*;

7.2.2 L'Algoritmo di Viterbi

In *linea teorica*, un decodificatore a massima verosimiglianza dovrebbe analizzare l'intera sequenza ricevuta Y, trovando il percorso *ammissibile* a minore distanza di Hamming da Y, ma in casi reali non è una strategia percorribile, il costo sarebbe proibitivo perché il numero di percorsi possibili cresce esponenzialmente con la lunghezza della sequenza.

L'algoritmo proposto da Andrew Viterbi nel 1967, rende possibile la ricerca del cammino ottimale senza ricorrere a una ricerca esaustiva.

Questo risultato si ottiene attraverso due pilastri fondamentali:

- L'introduzione della **metrica di ramo** (*branch metric*), che permette di valutare la distanza locale ad ogni transizione;
- La selezione sistematica dei **percorsi sopravvissuti**, che consiste nel *potare* precocemente i cammini che non possono più ambire ad essere ottimali, evitando così di esplorare rami inutili.

Metrica di Ramo e Metrica di Percorso

Consideriamo il caso in cui in ingresso al registro a scorrimento arriva un solo bit ($k = 1$).

Ogni nodo del traliccio al generico stato j, all'istante i,è indicato come:

$$S_i^j \quad j = 1, 2, \ldots, 2^{N-1}$$

Metrica del ramo:

$$\lambda(S_i^j, S_{i+1}^l)$$

La distanza (di Hamming o euclidea) tra gli n bit **ricevuti** al tempo i-esimo e gli n bit **emessi** con la transizione dallo stato j-esimo allo stato l-esimo:

$$S_i^j \Rightarrow S_{i+1}^l$$

La metrica del ramo è ovviamente **additiva**, infatti ogni percorso P composto da M transizioni avrà la **metrica di percorso** (lunghezza del cammino) definita come **somma delle metriche di ramo calcolate per ogni transizione di stato che lo compone**:

$$\Gamma(P) = \sum_{k=0}^{M-1} \lambda\left(S_{i+k}^{j_k}, S_{i+k+1}^{j_{k+1}}\right)$$

Concetto di Percorsi Sopravvissuti

Serve un ultimo strumento per l'algoritmo di Viterbi utile per il processo di *scrematura* dei percorsi implicito nell'algoritmo stesso. Osservando il traliccio si nota che in ogni istante temporale, in un singolo nodo (stato) convergono sempre 2^k rami provenienti da stati precedenti. Poiché questi percorsi, da questo specifico stato in poi, dovranno necessariamente condividere lo stesso identico futuro, solo il cammino che ha accumulato finora la **metrica di percorso minore** (distanza minima) può *matematicamente* fare parte del cammino ottimo globale.

Il **percorso sopravvivente** $\hat{P}(S_i^j)$ sarà, tra quelli che giungono in quel nodo, quello con la minore lunghezza del cammino.

Procedura Operativa dell'Algoritmo

1. **Inizializzazione:** $i = 0$ e $\Gamma(S_i^j), j = 1, 2, \ldots, 2^{N-1}$; si assume che lo stato iniziale e quello finale dopo M passi siano composti da tutti *zeri* (reset dei registri).
2. **Calcolo metriche di ramo:** Per $i = i + 1$ si calcolano le 2^{N-1} metriche di ramo $\lambda(S_{i-1}^j, S_i^l)$ per ognuno dei 2^{N-1} stati.
3. **Selezione dei percorsi sopravviventi**: per ogni stato S_i^j si calcolano le lunghezze dei percorsi incidenti e si memorizza il percorso sopravvivente $\hat{P}(S_i^j)$ e la nuova metrica di stato:

$$\Gamma(S_i^j) = \Gamma(\hat{P}(S_i^j))$$

4. **Iterazione:** si ripetono i passi precedenti fino al passo M.

5. **Decodifica:** si decodifica la sequenza ricevuta come la sequenza associata alle transizioni di stato dell'unico percorso sopravvivente.

7.2.3 Limiti e Complessità Computazionale

Nonostante l'algoritmo di Viterbi offra una decodifica ottima a massima verosimiglianza, la sua applicazione pratica è fortemente limitata dalla complessità computazionale e dai requisiti di memoria, che crescono esponenzialmente al crescere dei parametri del codificatore. Solamente di recente sono state realizzate implementazioni hardware ad-hoc.

Per un codice convoluzionale (n, k, N), ad ogni passo (istante temporale) sul traliccio:

- Si hanno $2^{k(N-1)}$ stati possibili. Il decodificatore deve mantenere attivi e memorizzare $2^{k(N-1)}$ **cammini sopravviventi** (con relative sequenze di bit) e i corrispondenti $2^{k(N-1)}$ **valori di metrica di percorso**.

- Per ogni stato si hanno 2^k rami provenienti dall'istante precedente. Il decodificatore deve quindi calcolare 2^k **metriche di ramo** per ogni singolo stato, aggiornare le metriche di percorso e confrontarle per selezionare il percorso sopravvivente.

A causa di questa dipendenza esponenziale sia dal numero di bit in ingresso k, sia dalla lunghezza del vincolo N, l'implementazione hardware dell'algoritmo di Viterbi diventa rapidamente proibitiva. Per questo motivo, nelle applicazioni reali (come le comunicazioni spaziali o satellitari), l'utilizzo di Viterbi è limitato a codici con valori di k e N dell'ordine di grandezza di 10. Per codici più complessi si ricorre solitamente a metodi sub-ottimi, come la decodifica sequenziale.